

Temporal Coherence and Narrative Pacing in High-Dynamic-Range Timelapses

A Formal Treatment of Stochastic Noise Reduction, Monotonic Interpolation, and 16-Bit Linear Pipelines

Simon Knight

Simon Knight Research

March 15, 2026 • Version 2.0.0

Last updated: March 16, 2026

Abstract. This report provides a comprehensive formal framework for the construction of high-dynamic-range (HDR) timelapses. We address the primary technical hurdle: high-frequency stochastic luminance noise (flicker) introduced by mechanical inconsistencies in camera shutter and aperture assemblies. The luminance time-series is modelled as a non-stationary stochastic process, and we derive a temporal smoothing operator grounded in CIE 1931 photometric weights with sigma-clipping for robust estimation. We present a “Holy Grail” log-linear transform for seamless day-to-night exposure de-stepping, and a narrative control engine utilising Fritsch–Carlson monotonicity-preserving Hermite splines to ensure organic pacing without temporal overshoot. The resulting 16-bit linear pipeline, implemented in Rust with data-parallel analysis, achieves professional-grade fidelity through stochastic dithering, LRU-cached frame blending, and XMP sidecar export for round-trip integration with professional RAW processors.

Contents

I FOUNDATIONAL MATHEMATICS

1	Physics of Mechanical Stochastic Noise	1
1.1	Aperture Diaphragm Friction	1
1.2	Shutter Speed Jitter	2
1.3	Combined Noise Model	2
2	CIE 1931 Photometric Normalisation	3
2.1	The Luminous Efficiency Function	3
2.2	CIE Luminance Coefficients	3
2.3	Linear vs Gamma-Encoded Space	4
3	Temporal Smoothing Operator	4
3.1	Rolling Mean Kernel	4
3.2	Boundary Handling	4
3.3	Correction Factor	4
3.4	Pseudocode	5
3.5	Chromatic Deflickering	5
3.6	Window Size Selection	6
3.7	Frequency Response of the Boxcar Filter	7
4	Sigma-Clipping for Robust Estimation	7
4.1	Two-Pass Algorithm	7
4.2	Why Single-Iteration Suffices	8
4.3	Temporal Confidence Smoothing	8
4.4	Spatiotemporal Stability Map	8
4.5	Radial Vignette Normalisation	10
5	Holy Grail Log-Linear Transform	10
5.1	Cumulative EV Shift	10
5.2	Inverse Compensation	10
5.3	Worked Example	11
5.4	Deadband Filtering	11
6	Monotonic PCHIP Interpolation	11
6.1	The Fritsch–Carlson Theorem (1980)	11
6.2	Weighted Harmonic Mean Tangents	12
6.3	Boundary Conditions	12
6.4	Hermite Basis Evaluation	12
6.5	Interpolation Comparison	13
6.6	Algorithm Pseudocode	13
7	Stochastic Dithering Theory	13
7.1	The Quantisation Problem	13
7.2	RPDF Dithering	13
7.3	RPDF vs TPDF	14

II NARRATIVE CONTROL & OPERATIONS

8	Narrative Warping and Temporal Beats	14
8.1	YAML Schema Reference	14
8.2	Frame Index to PTS Mapping	15
8.3	Instantaneous Speed Profile	15
9	Synthetic Motion Blur via Temporal Convolution	16
9.1	Temporal Convolution Kernel	16
9.2	Kernel Selection: Gaussian vs Linear	16
9.3	Linear-Space Blending	17
9.4	Memory Implications	17
10	Optical-Flow Guided Blending	17
10.1	Farneback Dense Optical Flow	17
10.2	Bidirectional Warp and Bilinear Interpolation	17
10.3	Feature Gating and Graceful Fallback	18

11	Narrative Audio Time-Stretching	18
11.1	Audio Pipeline Architecture	18
11.2	Stretch Ratio Calculation	18
11.3	Pitch-Preserved Resampling	18
12	Professional Ecosystem Integration	19
12.1	XMP Sidecar Anatomy	19
12.2	Workflow Integration	19
12.3	Field Mapping	19
12.4	HDR Bracket Detection	20
12.5	XMEML Timeline Export	20
12.6	DaVinci Resolve CSV Export	20
13	CLI Reference	20
13.1	Flag Reference	21
13.2	Common Workflows	21
III	IMPLEMENTATION ARCHITECTURE	
14	System Architecture	21
14.1	Crate Dependency Map	22
14.2	Stage Descriptions	22
14.3	Parallelism Architecture	23
15	Data Model	23
15.1	ImageFrame Structure	23
15.2	NarrativePlan and NarrativeKeyframe	24
15.3	Frame Lifecycle	24
16	LRU Frame Caching	24
16.1	Cache Sequence	24
16.2	Capacity Formula	24
16.3	Sliding Window Advancement	25
16.4	Memory Budget	25
17	Persistent Metadata Caching	25
17.1	Cache Format	25
17.2	Cache Invalidation	26
17.3	Performance Impact	26
18	Full Project Persistence	26
18.1	ProjectState Structure	26
18.2	Load/Save Lifecycle	27
18.3	Partial Re-Rendering	27
19	16-Bit Linear Pipeline	27
19.1	RAW Sensor Data and Demosaicing	27
19.2	Precision Through the Pipeline	27
19.3	Quantisation Error Analysis	28
19.4	Array Memory Layout	28
20	Encoding and Dithering Implementation	29
20.1	H.264 YUV 4:2:0 Setup	29
20.2	Per-Pixel Correction Application	29
20.3	Dithering Code Path	29
IV	DIAGNOSTICS & VALIDATION	
21	Diagnostic CSV Export	30
21.1	Column Definitions	30
21.2	Luminance Curve Visualisation	30
22	Visual Diagnostic Report	30
22.1	Pane Layout	30
23	Validation Methodology	31
23.1	Deflicker Quality Verification	31

23.2	PCHIP Monotonicity Validation	31
23.3	EV Compensation Accuracy	32
23.4	Before/After Correction Quality	32
24	Future Directions	33
24.1	AI-Assisted Composition Analysis	33
24.2	Sequence Assembly & Flow	35
24.3	Visualisation & Temporal Summary	36
24.4	Editing Assistance & Narrative Intelligence	38
24.5	Optical Flow & Motion Analysis	39
24.6	Stabilisation & Geometric Correction	41
24.7	Surveillance & Video Analytics Techniques	42
24.8	Deep Learning Extensions	43
24.9	Computational Photography	44
24.10	Colour & Exposure Extensions	45
24.11	Sonification & Audio Synthesis	46
24.12	360° & Spatial Video	47
24.13	Generative AI Extensions	48
24.14	Scientific & Environmental Measurement	49
24.15	Social Media & Delivery Optimisation	51
24.16	Multi-Camera Fusion	52
24.17	Incremental & Live Processing	53
24.18	Timestamp & Clock Correction	54
24.19	Film & Colour Science	56
24.20	Drone & Aerial Processing	56
24.21	Extreme Environment Correction	57
24.22	Artistic & Glitch Effects	58
24.23	Extended Astronomical Analysis	59
24.24	Plugin & Pipeline Extensibility	61
24.25	Forensic & Archival	62
24.26	Panoramic & Multi-Frame Spatial	63
24.27	Biological & Microscopy	64
24.28	Construction & Architecture	65
24.29	Energy & Infrastructure Monitoring	66
24.30	Thermal & Multispectral Imaging	67
24.31	Accessibility & Documentation	68
24.32	Creative Compositing	69
24.33	Temporal Bokeh & Focus-on-Change	70
24.34	Object Detection & Event Intelligence	72
24.35	Attention & Narrative Intelligence	74
24.36	Performance & Quality Extensions	75
25	Conclusion	76

FOUNDATIONAL MATHEMATICS

1 Physics of Mechanical Stochastic Noise

In timelapse acquisition, even with “locked” exposure settings, the observed luminance $Y[n]$ is subject to physical perturbations. These arise from two primary mechanical sources.

1.1 Aperture Diaphragm Friction

The physical leaves of the iris diaphragm do not return to the exact same geometric position for every actuation cycle. Minor friction, spring fatigue, and thermal expansion of the leaf assembly lead to an effective aperture diameter

$$D_{\text{eff}} = D_{\text{set}} + \delta_D, \quad \delta_D \sim \mathcal{N}(0, \sigma_D^2) \quad (1)$$

where D_{set} is the commanded diaphragm diameter and δ_D is a zero-mean stochastic perturbation. Because the f-number $N = f/D$, a perturbation δ_D produces a fractional exposure error proportional to $(D_{\text{set}} + \delta_D)^2 / D_{\text{set}}^2$, which to first order is $\approx 1 + 2\delta_D / D_{\text{set}}$.

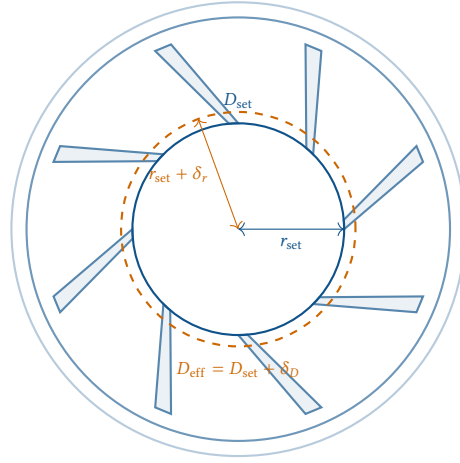


Figure 1. Aperture blade geometry. The solid circle shows the commanded aperture D_{set} ; the dashed circle shows the effective aperture D_{eff} after mechanical perturbation δ_D .

1.2 Shutter Speed Jitter

The electronic-mechanical timing of the shutter curtains fluctuates based on the physical state of the solenoid, ambient temperature, and battery voltage. This leads to an exposure time

$$t_{\text{eff}} = t_{\text{set}} + \delta_t, \quad \delta_t \sim \mathcal{N}(0, \sigma_t^2) \quad (2)$$

The variance σ_t^2 is inversely correlated with the commanded shutter speed: faster shutter speeds exhibit proportionally larger relative jitter because the absolute δ_t remains roughly constant (dominated by the curtain release solenoid latency, typically 50–200 μs).

Shutter Curtain Timing Jitter

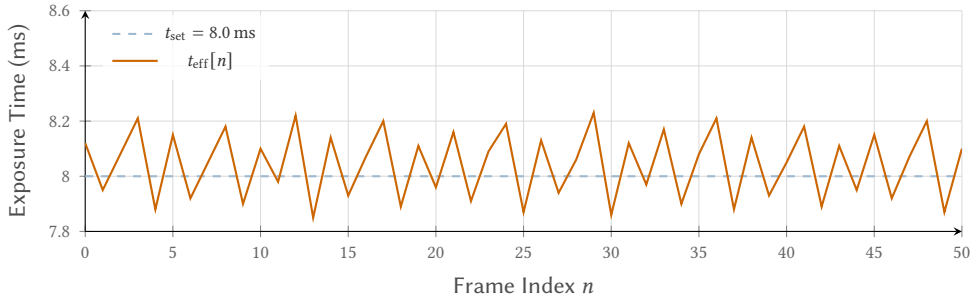


Figure 2. Shutter curtain timing jitter for a commanded 1/125 s exposure. The effective exposure t_{eff} fluctuates stochastically around the set-point with $\sigma_t \approx 0.12$ ms.

1.3 Combined Noise Model

The combined effect of aperture and shutter perturbations yields an observed luminance:

$$Y_{\text{obs}}[n] = Y_{\text{true}}[n] \cdot (1 + \epsilon[n]), \quad \epsilon[n] \sim \mathcal{N}(0, \sigma_{\text{mech}}^2) \quad (3)$$

where $Y_{\text{true}}[n]$ is the underlying photometric trend and $\sigma_{\text{mech}}^2 = \sigma_D^2 + \sigma_t^2$ (to first order, assuming independence). The variance σ_{mech}^2 is typically in the range of 0.01 to 0.05 stops, which is perceptually significant as high-frequency flicker in the rendered video.

Insight:
The noise $\epsilon[n]$ is multiplicative, not additive. This is why deflickering must operate in the ratio domain (smoothed/observed) rather than subtracting an offset.

Multiplicative Noise: True Signal vs Observed

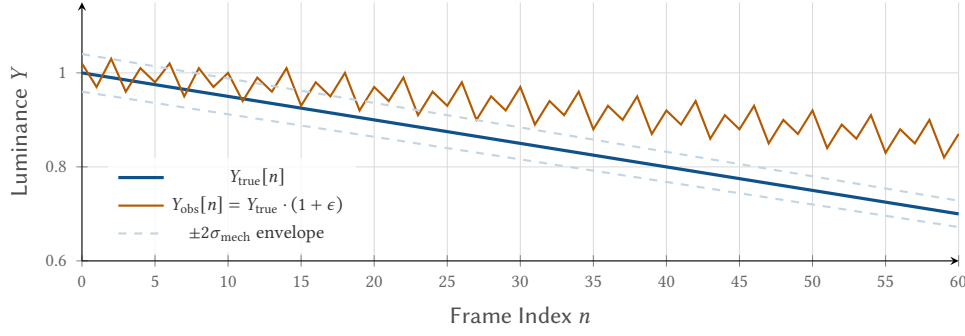


Figure 3. The multiplicative noise model in action. The true signal (blue) descends smoothly as daylight fades; the observed signal (orange) fluctuates within the $\pm 2\sigma$ envelope. Note that the noise *amplitude* shrinks with the signal—a hallmark of multiplicative (not additive) noise.

2 CIE 1931 Photometric Normalisation

To ensure absolute photometric fidelity, the engine performs luminance analysis in the linear energy domain using the 1931 CIE Standard Observer weights. These weights are derived from the photopic luminous efficiency function $V(\lambda)$, which represents the spectral sensitivity of the human visual system under photopic (daylight) conditions.

2.1 The Luminous Efficiency Function

$V(\lambda)$ peaks at 555 nm (yellow-green) and defines the weighting by which each wavelength contributes to perceived brightness. When projected onto the CIE xyz colour matching functions, the luminance channel Y is defined by the second row of the tristimulus transformation matrix.

Insight:
Standard sRGB coefficients (0.2126, 0.7152, 0.0722) assume a specific display gamma and D65 white point. Applying them to linear RAW sensor data introduces a systematic bias in luminance estimation.

CIE $V(\lambda)$ vs Rec.709 Luma Coefficients

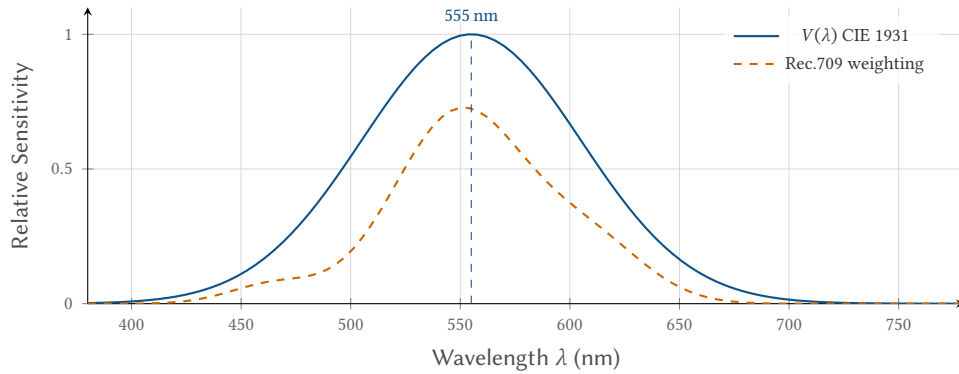


Figure 4. Comparison of the CIE 1931 photopic luminous efficiency function $V(\lambda)$ with the effective spectral sensitivity implied by Rec.709 luma coefficients. The CIE function is a continuous spectral weighting; Rec.709 approximates it through three discrete channel weights.

2.2 CIE Luminance Coefficients

Given the tristimulus values for the CIE primary monochromatic lights at 700.0 nm (R), 546.1 nm (G), and 435.8 nm (B), the relative luminance Y is defined as:

$$Y = 0.17697 R + 0.81240 G + 0.01063 B \quad (4)$$

This normalisation ensures that the deflickering engine operates on the “physical” brightness of the scene—the quantity that the human eye perceives as luminance—rather than on a display-referred approximation.

2.3 Linear vs Gamma-Encoded Space

Critical Distinction

The CIE coefficients in eq. (4) are only valid when applied to *linear* (scene-referred) data. RAW sensor values are inherently linear; however, many image processing tools silently apply a gamma curve ($\gamma \approx 2.2$ or sRGB transfer function) during demosaicing. The engine ensures that all analysis is performed *before* any non-linear transformation by operating on the 16-bit linear output of the RAW processor.

The distinction matters quantitatively. For a mid-grey value $Y_{\text{linear}} = 0.18$ (18% reflectance), the sRGB-encoded value is $Y_{\text{sRGB}} = 0.4615$ (via the sRGB EOTF). Computing weighted luminance on gamma-encoded data would overestimate shadow brightness by a factor of $\sim 2.5\times$, leading to systematically incorrect deflicker corrections in dark scenes.

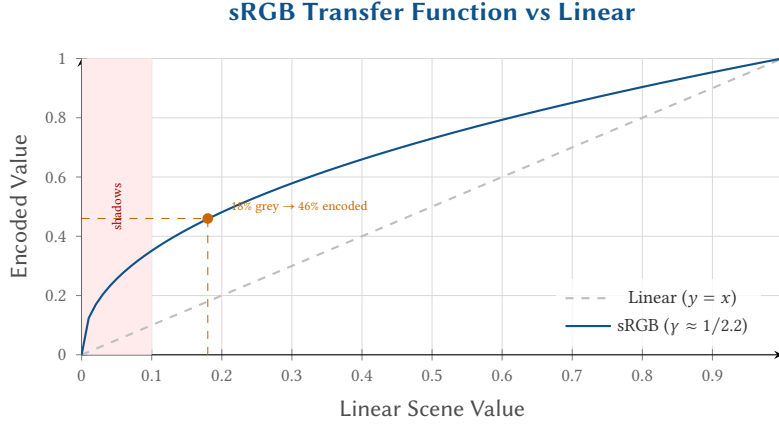


Figure 5. The sRGB transfer function (blue curve) compresses highlights and expands shadows relative to linear light. The shaded red region shows where gamma-encoded analysis would introduce the largest luminance estimation errors—precisely the region most important for nighttime timelapse deflickering.

3 Temporal Smoothing Operator

The deflickering algorithm suppresses the multiplicative noise $\epsilon[n]$ from eq. (3) by replacing the raw luminance series with a temporally smoothed estimate.

3.1 Rolling Mean Kernel

We define the temporal smoothing operator S_W as a symmetric centred boxcar (moving average) with half-width $W/2$:

$$\hat{Y}[n] = (S_W * Y)[n] = \frac{1}{|K_n|} \sum_{k \in K_n} Y[k] \quad (5)$$

where $K_n = \{k : |k - n| \leq \lfloor W/2 \rfloor, 0 \leq k < N\}$ is the set of indices within the window centred at n , clamped to the valid range $[0, N)$.

3.2 Boundary Handling

Near the sequence boundaries, the window is necessarily asymmetric:

$$|K_n| = \min(n, \lfloor W/2 \rfloor) + \min(N - 1 - n, \lfloor W/2 \rfloor) + 1 \quad (6)$$

This asymmetric truncation avoids introducing artificial edge transients. The implementation uses `ndarray::ArrayBase::mean()` over the valid slice, which handles the variable-length window implicitly.

3.3 Correction Factor

The per-frame correction factor is the ratio of the smoothed trend to the observed value:

$$C[n] = \frac{\hat{Y}[n]}{Y_{\text{obs}}[n]} \quad (7)$$

This factor is multiplicative—consistent with the multiplicative noise model of eq. (3). When applied to the 16-bit pixel data, it scales every pixel in the frame by the same scalar, effectively normalising the frame’s overall exposure to match the smooth trend.

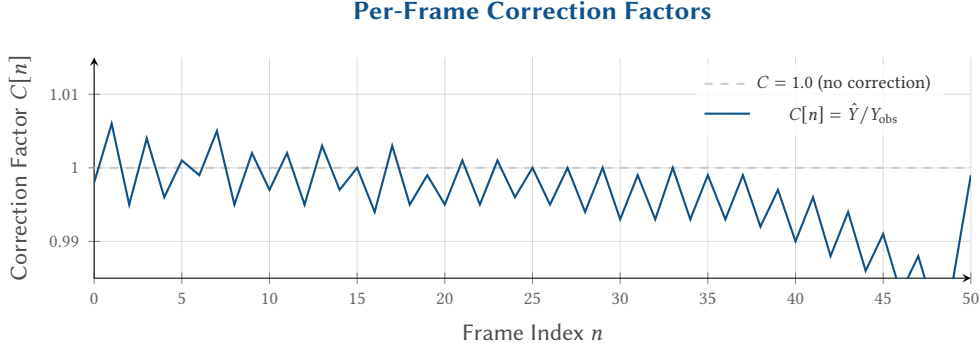


Figure 6. Per-frame correction factors oscillate around unity. Values > 1 brighten (frame was too dark), values < 1 dim (frame was too bright). The downward drift beyond frame 40 reflects the smoothing operator “anticipating” the twilight descent.

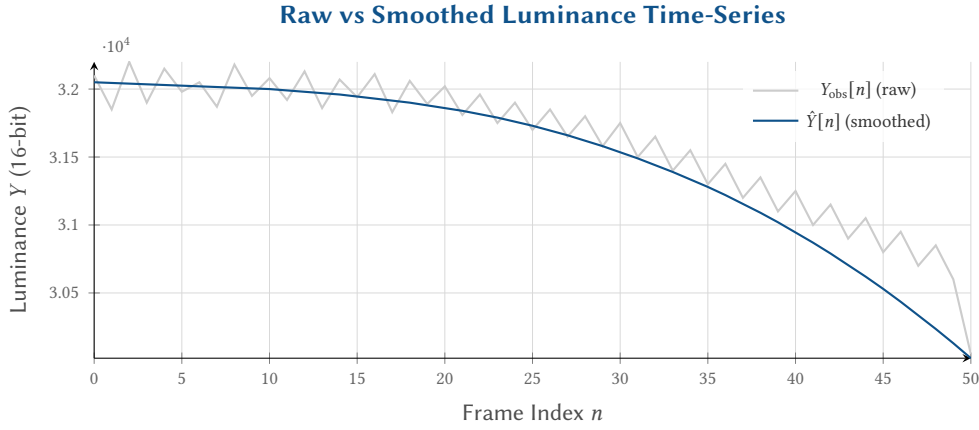


Figure 7. First 50 frames of a Holy Grail sequence. The raw signal (grey) exhibits high-frequency stochastic flicker; the smoothed trend (blue) captures the underlying photometric descent into twilight.

3.4 Pseudocode

Algorithm 1 Temporal Luminance Smoothing

Require: Frame sequence $\{Y_{\text{obs}}[n]\}_{n=0}^{N-1}$, window size W

Ensure: Smoothed sequence $\{\hat{Y}[n]\}$, correction factors $\{C[n]\}$

```

1:  $half \leftarrow \lfloor W/2 \rfloor$ 
2: for  $n \leftarrow 0$  to  $N - 1$  do
3:    $lo \leftarrow \max(0, n - half)$ 
4:    $hi \leftarrow \min(N - 1, n + half)$ 
5:    $\hat{Y}[n] \leftarrow \text{mean}(Y_{\text{obs}}[lo \dots hi])$ 
6: end for
7: for  $n \leftarrow 0$  to  $N - 1$  do
8:    $C[n] \leftarrow \hat{Y}[n] / Y_{\text{obs}}[n]$   $\triangleright$  Falls back to 1.0 if  $Y_{\text{obs}}[n] = 0$ 
9: end for

```

3.5 Chromatic Deflickering

In addition to luminance flicker, timelapse sequences suffer from *chromatic flicker*—subtle but perceptible fluctuations in colour temperature caused by the same mechanical tolerances that affect brightness. Because aperture blade geometry affects the relative transmission of different wavelengths (particularly at the periphery of the diaphragm), the R, G, and B channels exhibit independent stochastic noise.

The engine applies the same rolling mean operator $\mathcal{S}_W$ independently to each colour channel:

$$\hat{R}[n] = (\mathcal{S}_W * R)[n], \quad C_R[n] = \hat{R}[n] / R_{\text{obs}}[n] \quad (8)$$

$$\hat{G}[n] = (\mathcal{S}_W * G)[n], \quad C_G[n] = \hat{G}[n] / G_{\text{obs}}[n] \quad (9)$$

$$\hat{B}[n] = (\mathcal{S}_W * B)[n], \quad C_B[n] = \hat{B}[n] / B_{\text{obs}}[n] \quad (10)$$

where R_{obs} , G_{obs} , B_{obs} are the sigma-clipped per-channel means from section 4. These per-channel correction factors are stored in `color_correction`: `[f64; 3]` and applied during the pixel correction step alongside the luminance correction and EV offset.

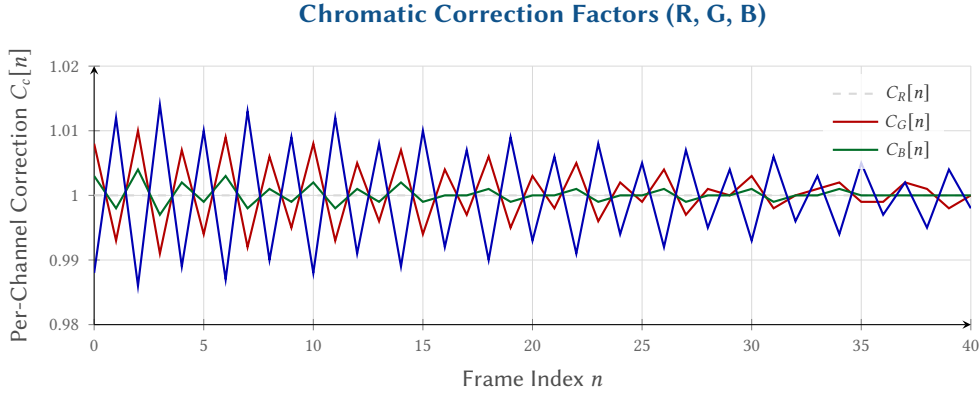


Figure 8. Per-channel chromatic correction factors. The blue channel (bottom trace) exhibits the largest variance due to its low CIE weight (0.01063), making it most sensitive to aperture blade position. The green channel (middle) is most stable, consistent with its dominant 0.81240 weighting.

Independence of Luminance and Chromatic Correction

The luminance correction factor $C[n]$ and the per-channel colour corrections C_R , C_G , C_B are computed independently and applied multiplicatively. This means a frame that is both too bright *and* too blue will receive both a dimming factor ($C < 1$) and a blue channel attenuation ($C_B < 1$), with the combined effect being physically correct in the linear domain.

3.6 Window Size Selection

The smoothing window size W (controlled by `-window`, default 10) represents a trade-off between noise suppression and temporal fidelity:

Window W	Character	Best For
5–10	Light smoothing	Short sequences, fast-changing light
10–20	Moderate smoothing	Most timelapse scenarios
20–50	Heavy smoothing	Long static scenes, severe flicker

A window that is too wide will attenuate legitimate luminance changes (e.g., a cloud shadow passing over the scene), while a window that is too narrow will fail to suppress the mechanical noise. The diagnostics CSV (section 21) enables empirical tuning by visualising the raw-vs-smoothed curves at different window sizes.

Effect of Window Size on Smoothing Fidelity

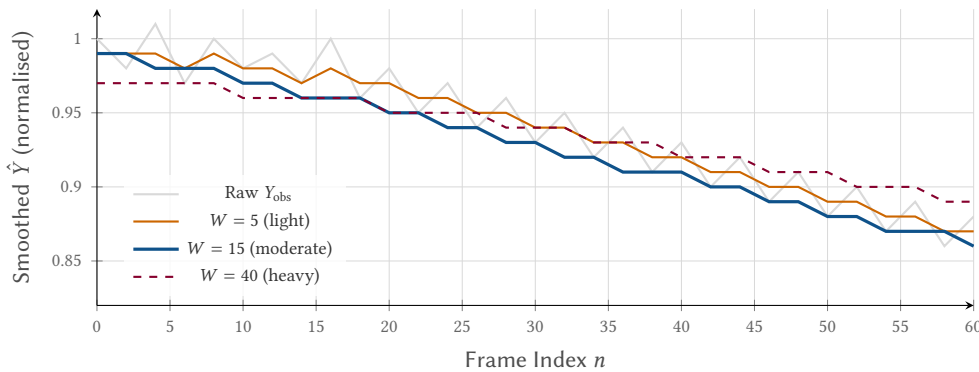


Figure 9. Window size comparison on the same sequence. $W = 5$ retains residual flicker; $W = 15$ balances noise suppression and trend fidelity; $W = 40$ over-smooths, lagging the true descent and underestimating the early frames while overestimating the late ones.

3.7 Frequency Response of the Boxcar Filter

The boxcar (rolling mean) kernel of width W has a normalised discrete-time frequency response given by the Dirichlet sinc:

$$H(f) = \frac{1}{W} \frac{\sin(\pi f W)}{\sin(\pi f)}, \quad f \in [0, 0.5] \quad (11)$$

where f is the normalised frequency in cycles per frame.

Frequency Response: Boxcar Filter

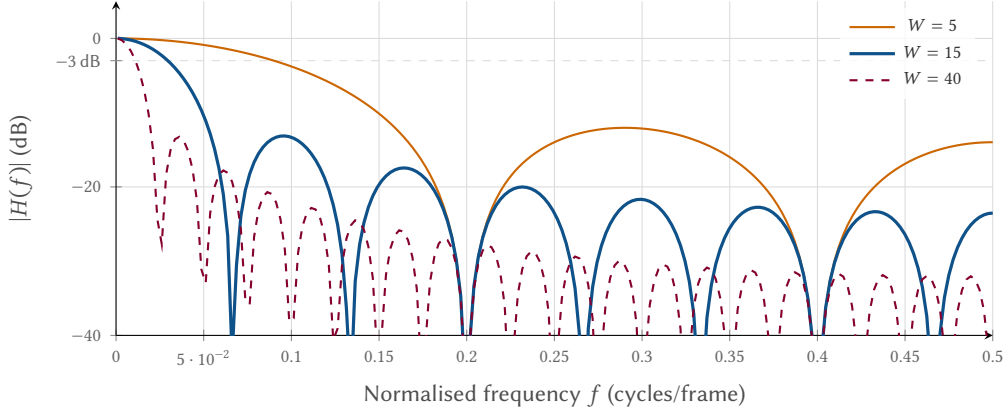


Figure 10. Magnitude response of the boxcar filter at three window sizes. Wider windows push the first null to lower frequencies, providing stronger low-pass suppression of frame-to-frame flicker but also attenuating legitimate scene changes. The -3 dB line (dashed red) marks the half-power cutoff.

Why Boxcar Over Gaussian?

A Gaussian kernel offers a smoother spectral roll-off without sidelobes, but the boxcar’s computational simplicity (a single `ndarray::mean()` call per slice) and its adequacy for the slowly-varying luminance signals encountered in practice make it the pragmatic choice. The sidelobes visible in fig. 10 are at -13 dB or lower for $W \geq 10$, well below the perceptual threshold for flicker.

4 Sigma-Clipping for Robust Estimation

Before the smoothing operator can be applied, we must first obtain a robust estimate of the per-frame mean luminance $Y_{\text{obs}}[n]$. A naïve arithmetic mean over all pixels is sensitive to outliers: specular highlights (sun reflections, car headlights) and dead pixels can bias the luminance estimate by several percent.

4.1 Two-Pass Algorithm

The engine implements a single-iteration sigma-clipping procedure controlled by the `-sigma` parameter (default $\sigma_{\text{clip}} = 2.0$):

Algorithm 2 Sigma-Clipped Mean Luminance

Require: Pixel luminance array $\{y_p\}_{p=1}^P$, threshold σ_{clip}

Ensure: Robust mean $\bar{Y}_{\text{clip}}$

- 1: **Pass 1:** Compute $\bar{Y} \leftarrow \frac{1}{P} \sum_p y_p$, $s \leftarrow \sqrt{\frac{1}{P} \sum_p (y_p - \bar{Y})^2}$
 - 2: **Pass 2:** $A \leftarrow \{p : |y_p - \bar{Y}| \leq \sigma_{\text{clip}} \cdot s\}$
 - 3: $\bar{Y}_{\text{clip}} \leftarrow \frac{1}{|A|} \sum_{p \in A} y_p$
-

The same clipping mask A is applied simultaneously to the R, G, and B channels, ensuring consistent per-channel means for chromatic deflickering.

Sigma-Clipping Rejection Zones

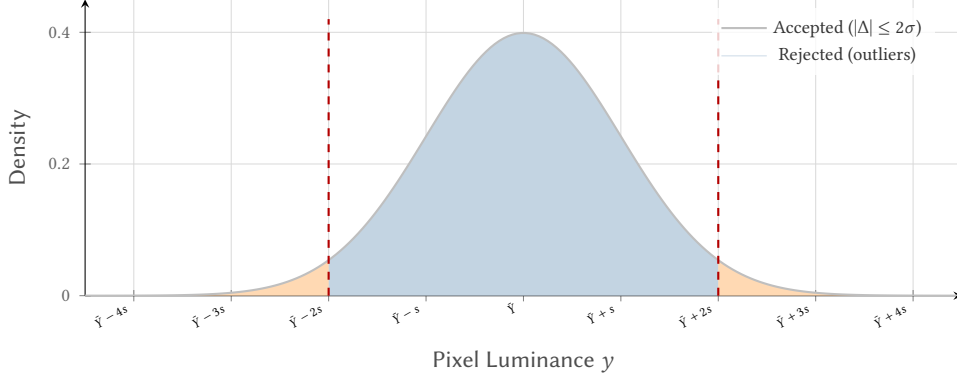


Figure 11. Gaussian pixel distribution with 2σ sigma-clipping. Pixels in the shaded blue region contribute to the robust mean; orange-shaded tails (specular highlights, dead pixels) are rejected.

4.2 Why Single-Iteration Suffices

Iterative sigma-clipping (repeating pass 1 and pass 2 until convergence) is necessary when the contamination fraction is high or when the underlying distribution is unknown. For timelapse frames, the pixel luminance distribution is approximately normal with contamination limited to specular highlights ($< 1\%$ of pixels). A single pass at 2σ rejects approximately 4.6% of the total distribution, which is sufficient to eliminate the tail artefacts without over-trimming the body of the distribution.

4.3 Temporal Confidence Smoothing

The sigma-clipping inlier ratio $\rho[n] = N_{\text{inlier}}[n]/N_{\text{total}}$ serves as a per-frame confidence score. Frames with abnormally low confidence ($\rho < 0.7$) indicate transient contamination—passing clouds, vehicle headlights, or lens flare—that may bias the brightness estimate despite sigma-clipping.

To prevent single-frame “analysis flicker,” the engine applies a sequential exponential smoothing pass after the parallel analysis is complete. For each frame n with $\rho[n] < \rho_{\text{threshold}}$:

$$\hat{Y}[n] = \alpha \cdot Y_{\text{raw}}[n] + (1 - \alpha) \cdot \hat{Y}[n - 1] \quad (12)$$

where $\alpha = 0.3$ weights the current (noisy) estimate lightly and allows the previous frame’s more reliable estimate to dominate. Frames with $\rho \geq 0.7$ pass through unmodified. Both $\rho_{\text{threshold}} = 0.7$ and $\alpha = 0.3$ are compile-time constants in `process::temporal_smooth_confidence`.

Temporal Confidence Smoothing

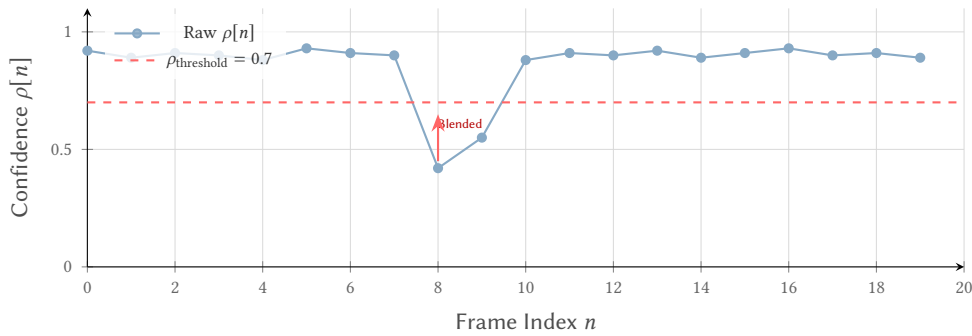


Figure 12. Confidence scores across a sequence. Frames 8–9 fall below the $\rho = 0.7$ threshold (dashed red) and their brightness estimates are blended toward the previous frame using $\alpha = 0.3$.

This sequential pass is computationally trivial ($O(N)$) and runs after the parallel analysis, preserving the Rayon parallelism of the primary analysis stage.

4.4 Spatiotemporal Stability Map

A complementary pre-analysis technique identifies *spatially* unstable regions—pixels that are frequently outliers across the sequence. Passing clouds, moving vehicles, flickering signs, and other transient objects occupy consistent spatial regions and can be pre-filtered before per-frame sigma-clipping begins.

4.4.1 Two-Pass Architecture

Pass 1 (Stability Discovery): The engine subsamples every 10th frame and, for each pixel position, counts how many times the green-channel value deviates more than 50% from the frame mean. The stability weight at pixel p is:

$$w_{\text{stab}}(p) = 1 - \frac{C_{\text{outlier}}(p)}{N_{\text{sampled}}} \quad (13)$$

where $C_{\text{outlier}}(p)$ is the outlier count and N_{sampled} is the number of sampled frames. Pixels with $w_{\text{stab}} < 0.5$ are excluded from luminance analysis in Pass 2.

Pass 2 (Filtered Analysis): The standard sigma-clipping analysis (section 4) proceeds, but pixels flagged as spatially unstable are skipped entirely. This improves the robustness of the brightness estimate for scenes with persistent motion.



Spatially unstable pixels (road, clouds) are excluded from sigma-clipping

Figure 13. Spatiotemporal stability map. Green cells represent stable pixels included in luminance analysis; red cells represent frequently unstable pixels (e.g., moving traffic, passing clouds) that are excluded. The map is computed once as a pre-analysis pass and reused for all frames.

The stability map is enabled by default (`-stability true`) and adds approximately 10% to the total analysis time (due to the subsampled pre-pass), but significantly improves deflicker quality for urban and weather-variable scenes.

4.4.2 Temporal Mask Smoothing

When the set of included pixels changes abruptly between adjacent frames (e.g. a cloud edge drifting across the frame), the brightness estimate can “jump” even though the underlying scene luminance is smooth—a phenomenon termed *secondary flicker*. To prevent this, the stability map applies an exponential smoothing (Alpha filter) across the subsampled temporal axis rather than accumulating raw outlier counts.

The per-pixel stability weight evolves according to:

$$w_{\text{stab}}^{(t)}(p) = \alpha \cdot m^{(t)}(p) + (1 - \alpha) \cdot w_{\text{stab}}^{(t-1)}(p) \quad (14)$$

where $m^{(t)}(p) \in \{0, 1\}$ is the binary mask observation at subsampled frame t (1 if the pixel is within 50% of the frame mean, 0 otherwise), $\alpha = 0.15$ is the default smoothing constant (configurable via `-mask-smooth`), and the prior $w_{\text{stab}}^{(0)} = 1.0$ assumes all pixels are initially stable.

With subsampling every 10th frame, $\alpha = 0.15$ gives approximately 87% weight to the running estimate and 13% to the current observation, smoothing over ~ 70 frames of temporal context. A single transient outlier depresses the weight only slightly, while a persistently unstable pixel converges below the 0.5 exclusion threshold after several consecutive observations.

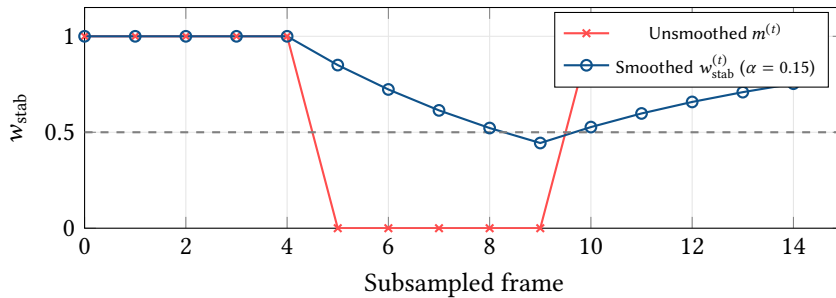


Figure 14. Smoothed vs. unsmoothed stability weight for a pixel that becomes transiently unstable (frames 5–9). The exponential filter prevents the abrupt inclusion/exclusion transitions that cause secondary flicker.

Setting `-mask-smooth 0` disables temporal smoothing and reverts to the original counting approach. The memory footprint is unchanged—a single `Vec<f32>` of $W \times H$ elements.

4.5 Radial Vignette Normalisation

Wide-angle lenses exhibit $\cos^4$ vignetting—a radial luminance falloff from the optical axis. When the sigma-clipping analysis treats all pixels equally, darker corner pixels bias the mean brightness downward, producing a spatially dependent error in the deflicker correction.

The engine applies an optional radial normalisation mask prior to sigma-clipping. For a pixel at position p in an image of width W and height H :

$$M(p) = 1 + r_{\text{norm}}^2 \cdot s \quad (15)$$

where $r_{\text{norm}} = \|p - c\|/r_{\text{max}}$ is the normalised distance from the image centre $c = (W/2, H/2)$ to the corner $r_{\text{max}} = \sqrt{c_x^2 + c_y^2}$, and s is the user-specified vignette strength (`-vignette`). The luminance estimate for pixel p is then $Y(p) \cdot M(p)$, which lifts corner pixels to compensate for the optical falloff before the sigma-clipping pass.

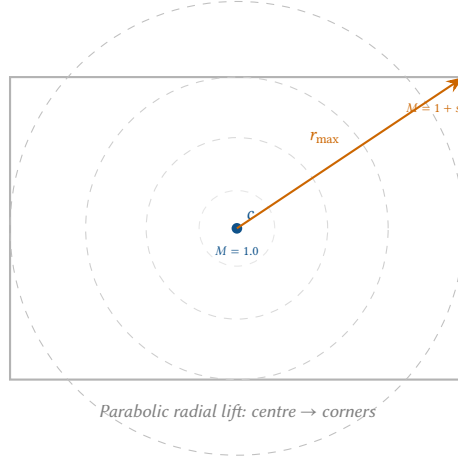


Figure 15. Radial vignette normalisation mask. The multiplicative correction $M(p) = 1 + r_{\text{norm}}^2 \cdot s$ increases monotonically from unity at the optical centre to $1 + s$ at the image corners, compensating for $\cos^4$ lens vignetting before sigma-clipping.

The mask is pre-computed once from the first frame’s dimensions and reused for all subsequent frames. With $s = 0$ (the default), $M(p) = 1$ everywhere and the normalisation is a no-op.

5 Holy Grail Log-Linear Transform

The “Holy Grail” day-to-night transition involves discrete exposure adjustments—typically in $1/3$ EV increments—as the ambient light diminishes. These produce visible “steps” in the luminance time-series that must be flattened into a continuous ramp.

5.1 Cumulative EV Shift

The cumulative Exposure Value shift at frame n is computed from the EXIF metadata:

$$\Delta\text{EV}[n] = \sum_{i=1}^n \left[2 \log_2 \left(\frac{N_{i-1}}{N_i} \right) + \log_2 \left(\frac{t_i}{t_{i-1}} \right) + \log_2 \left(\frac{\text{ISO}_i}{\text{ISO}_{i-1}} \right) \right] \quad (16)$$

where N is the f-number, t is the shutter speed, and ISO is the sensor gain. The factor of 2 on the aperture term arises because exposure is proportional to the *area* of the aperture ($\propto D^2$), and $N = f/D$.

5.2 Inverse Compensation

The engine applies the inverse offset $2^{-\Delta\text{EV}[n]}$ to the 16-bit linear data, effectively flattening the discrete exposure jumps into a continuous ramp. The cumulative offset is tracked with a deadband threshold of $|\Delta\text{EV}| > 0.01$ stops to suppress numerical noise from floating-point EXIF rounding.

: Deadband Threshold

Only exposure changes exceeding 0.01 stops trigger compensation. This prevents amplifying quantised EXIF rounding artefacts (many cameras store shutter speed as a rational number with limited denominator precision).

5.3 Worked Example

Consider a 3-frame transition at sunset where the photographer opens the aperture by one stop and doubles the ISO:

Frame	N	t (s)	ISO	ΔEV_i	Cumulative Ω
$n = 0$	$f/5.6$	$1/125$	200	—	0.00
$n = 1$	$f/4.0$	$1/125$	200	+0.97	-0.97
$n = 2$	$f/4.0$	$1/125$	400	+1.00	-1.97

At frame $n = 2$, the linear multiplier applied to pixel data is $2^{-(-1.97)} = 2^{1.97} \approx 3.92$, scaling the (now brighter due to higher ISO) frame back down to match the original exposure level.

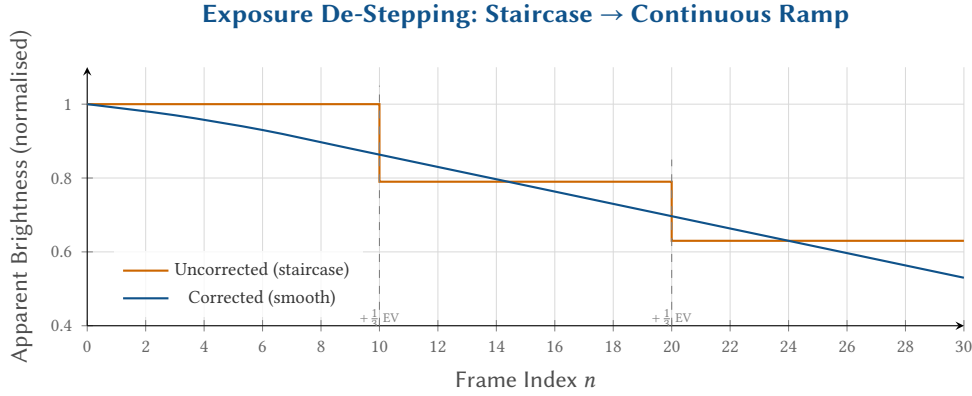


Figure 16. The Holy Grail transform flattens discrete exposure steps (orange staircase) into a continuous luminance ramp (blue). Each vertical jump corresponds to a camera exposure adjustment.

5.4 Deadband Filtering

The deadband threshold ($|\Delta EV| > 0.01$ stops) is critical for suppressing false EV transitions caused by EXIF rounding. Many cameras store shutter speed as a rational number (e.g., $1/128$ rounded from $1/125$), introducing sub-centistop artefacts that would accumulate over thousands of frames without filtering.

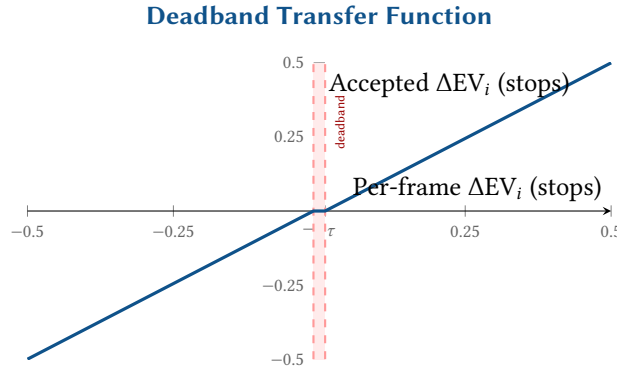


Figure 17. Deadband transfer function with threshold $\tau = 0.01$ stops. Inputs within the shaded dead zone are zeroed, preventing EXIF rounding noise from accumulating into false exposure drift.

6 Monotonic PCHIP Interpolation

Narrative speed ramping requires mapping the acquisition frame index n to a virtual timeline $T(n)$. A simple cubic spline is insufficient because it does not guarantee monotonicity; if the user defines a sharp deceleration, the spline may “dip” backwards, creating a non-physical temporal reversal where the video appears to run in reverse.

6.1 The Fritsch–Carlson Theorem (1980)

Theorem 1 (Fritsch–Carlson Monotonicity Conditions). A piecewise cubic Hermite interpolant $p(x)$ on intervals $[x_i, x_{i+1}]$ with data values y_i and tangents d_i is monotone if and only if:

1. $\text{sign}(d_i) = \text{sign}(\Delta_i)$ for all interior nodes, where $\Delta_i = (y_{i+1} - y_i)/h_i$ is the secant slope and $h_i = x_{i+1} - x_i$.
2. At local extrema ($\Delta_{i-1} \cdot \Delta_i \leq 0$), the tangent $d_i = 0$.
3. The tangent magnitudes satisfy a bound related to the secant slopes (specifically, the point $(d_i/\Delta_i, d_{i+1}/\Delta_i)$ must lie within the monotonicity region).

6.2 Weighted Harmonic Mean Tangents

The engine computes tangents using the Fritsch–Butland weighted harmonic mean, which automatically satisfies the monotonicity conditions:

$$d_i = \begin{cases} \frac{\frac{w_1 + w_2}{\frac{w_1}{\Delta_{i-1}} + \frac{w_2}{\Delta_i}}}{\Delta_{i-1} + \Delta_i} & \text{if } \Delta_{i-1} \Delta_i > 0 \\ 0 & \text{otherwise} \end{cases} \quad (17)$$

where the weights are:

$$w_1 = 2h_i + h_{i-1} \quad (18)$$

$$w_2 = h_i + 2h_{i-1} \quad (19)$$

This weighted harmonic mean biases the tangent toward the smaller of the two adjacent secants, preventing the interpolant from overshooting either data value.

6.3 Boundary Conditions

At the endpoints, the engine uses parabolic (one-sided) tangent estimates:

$$d_0 = \frac{(2h_0 + h_1)\Delta_0 - h_0 \Delta_1}{h_0 + h_1} \quad (20)$$

$$d_{n-1} = \frac{(2h_{n-2} + h_{n-3})\Delta_{n-2} - h_{n-2} \Delta_{n-3}}{h_{n-2} + h_{n-3}} \quad (21)$$

These estimates are derived from fitting a parabola through the first (or last) three data points, providing a C^1 -continuous extension that avoids the artificial flattening of natural boundary conditions.

6.4 Hermite Basis Evaluation

For a query point x in the interval $[x_i, x_{i+1}]$, we compute the normalised parameter $t = (x - x_i)/h_i$ and evaluate using the cubic Hermite basis polynomials:

$$p(x) = h_{00}(t) y_i + h_{10}(t) h_i d_i + h_{01}(t) y_{i+1} + h_{11}(t) h_i d_{i+1} \quad (22)$$

where:

$$h_{00}(t) = 2t^3 - 3t^2 + 1 \quad h_{10}(t) = t^3 - 2t^2 + t \quad (23)$$

$$h_{01}(t) = -2t^3 + 3t^2 \quad h_{11}(t) = t^3 - t^2 \quad (24)$$

Cubic Hermite Basis Polynomials

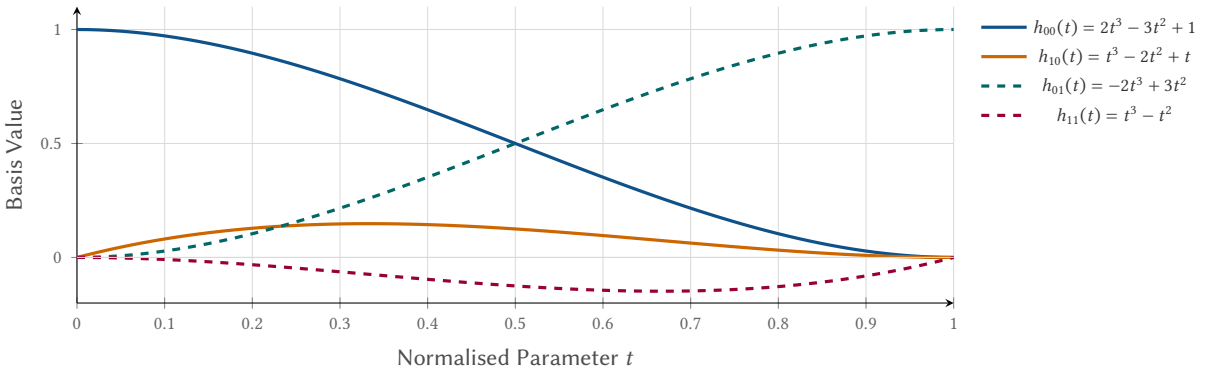


Figure 18. The four cubic Hermite basis polynomials on $[0, 1]$. h_{00} and h_{01} interpolate values at the endpoints ($h_{00}(0) = 1, h_{01}(1) = 1$); h_{10} and h_{11} control the tangent slopes. The PCHIP tangent constraints ensure that these basis functions combine to produce a monotone interpolant.

6.5 Interpolation Comparison

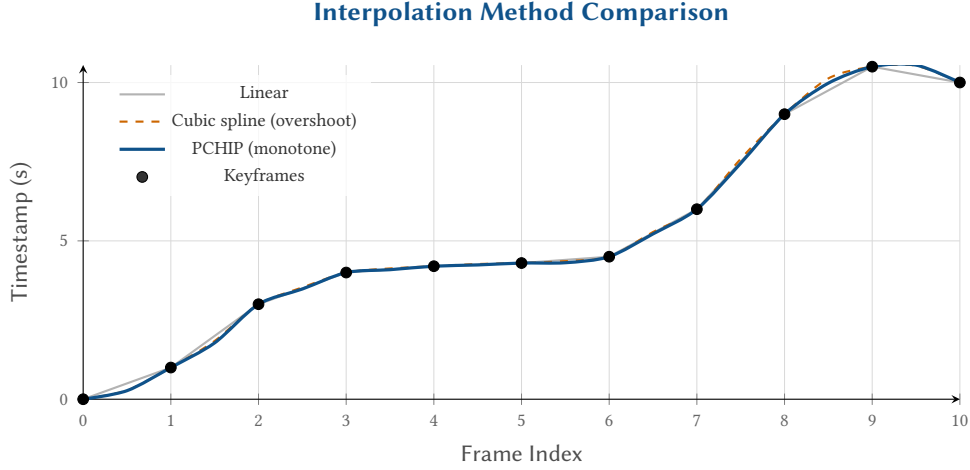


Figure 19. Comparison of linear, cubic spline, and PCHIP interpolation through a set of narrative keyframes. Note how the cubic spline overshoots near the deceleration at frames 9–10, while PCHIP remains monotone between consecutive monotone segments.

6.6 Algorithm Pseudocode

Algorithm 3 PCHIP Interpolation with Fritsch–Carlson Tangents

Require: Keyframes $\{(x_i, y_i)\}_{i=0}^m$, query points $\{q_j\}$

Ensure: Interpolated values $\{T(q_j)\}$

```

1: for  $i \leftarrow 0$  to  $m - 1$  do
2:    $h_i \leftarrow x_{i+1} - x_i$ ;  $\Delta_i \leftarrow (y_{i+1} - y_i)/h_i$ 
3: end for
4: for  $i \leftarrow 1$  to  $m - 1$  do ▷ Interior tangents
5:   if  $\Delta_{i-1} \cdot \Delta_i > 0$  then
6:      $w_1 \leftarrow 2h_i + h_{i-1}$ ;  $w_2 \leftarrow h_i + 2h_{i-1}$ 
7:      $d_i \leftarrow (w_1 + w_2)/(w_1/\Delta_{i-1} + w_2/\Delta_i)$ 
8:   else
9:      $d_i \leftarrow 0$ 
10:  end if
11: end for
12: Compute  $d_0, d_m$  via parabolic boundary conditions (eq. (20))
13: for each query  $q_j$  do
14:   Find interval  $i$  such that  $x_i \leq q_j < x_{i+1}$ 
15:    $t \leftarrow (q_j - x_i)/h_i$ 
16:   Evaluate via eq. (22)
17: end for

```

7 Stochastic Dithering Theory

7.1 The Quantisation Problem

When the 16-bit linear pipeline (0–65535) is downsampled to 8-bit output (0–255) for video encoding, each pixel value is divided by 256 and truncated. The quantisation error $e_q = x - \lfloor x \rfloor$ is deterministic and signal-correlated, producing visible banding artefacts in smooth gradients—particularly in dark regions where the 8-bit step size corresponds to a proportionally larger luminance change.

7.2 RPDF Dithering

To decorrelate the quantisation error from the signal, the engine injects a stochastic offset before truncation:

$$x_{\text{out}} = \left\lfloor \frac{x_{16} + \xi}{256} \right\rfloor, \quad \xi \sim \mathcal{U}(-0.5, 0.5) \quad (25)$$

where ξ is drawn from a Rectangular Probability Density Function (RPDF)—a uniform distribution over $[-0.5, +0.5]$. This replaces the deterministic banding with a spectrally flat noise floor at ± 0.5 LSB, which is perceptually neutral at typical viewing distances.

Insight:
Quantisation noise in dark gradients is not white; it is highly correlated with the signal, leading to visible Mach bands.

7.3 RPDF vs TPDF

Triangular PDF (TPDF) dithering—the sum of two independent RPDF samples, $\xi_T = \xi_1 + \xi_2 \sim \text{Tri}(-1, +1)$ —provides superior noise shaping in audio applications where the noise floor is audible. For 8-bit video, RPDF is preferred because:

1. The ± 0.5 LSB noise floor of RPDF is already below the perceptual threshold for ≥ 8 -bit video.
2. TPDF doubles the noise variance ($\sigma_{\text{RPDF}}^2 = 1/12$ vs $\sigma_{\text{TPDF}}^2 = 1/6$), reducing the effective dynamic range by ~ 3 dB.
3. The computational cost of a second RNG draw per pixel is non-trivial at 4K resolution (~ 25 M pixels per frame).

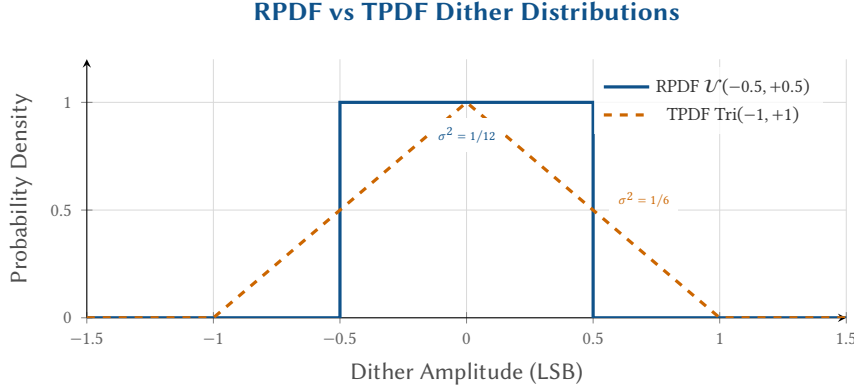


Figure 20. Comparison of RPDF and TPDF dither distributions. RPDF (blue) has half the variance of TPDF (orange dashed), introducing less noise while still fully decorrelating quantisation error from the signal. The engine uses RPDF.

Truncated (banding):



RPDF Dithered:



Figure 21. Visual comparison of 8-bit quantisation strategies. Top: naïve truncation produces visible banding. Bottom: RPDF dithering breaks the banding into perceptually neutral grain.

NARRATIVE CONTROL & OPERATIONS

8 Narrative Warping and Temporal Beats

The engine decouples “acquisition frequency” from “playback velocity.” The user provides a narrative plan in YAML format, defining key moments in the sequence as temporal anchors. The PCHIP interpolator (section 6) generates smooth transitions between these anchors.

8.1 YAML Schema Reference

The narrative plan file contains a single top-level key `keyframes`, which is a list of anchor objects. Each anchor has two required fields:

Field	Type	Description
<code>frame_index</code>	<code>usize</code>	Source frame number (0-indexed)
<code>timestamp_seconds</code>	<code>f64</code>	Desired playback time (PTS) in seconds

Keyframes are automatically sorted by `frame_index` before interpolation. The ratio of frame indices to timestamp deltas between consecutive keyframes determines the effective playback speed in that segment:

$$v_{\text{eff}} = \frac{\Delta \text{frame_index}}{\Delta \text{timestamp_seconds}} \quad (\text{frames per second}) \quad (26)$$

Listing 1. Narrative Plan for a Holy Grail Sequence

```
keyframes:
  - frame_index: 0
```

```

timestamp_seconds: 0.0 # Start of sequence
- frame_index: 300
timestamp_seconds: 10.0 # Constant 30fps: daylight
- frame_index: 600
timestamp_seconds: 15.0 # Accelerated: 60fps effective for sunset
- frame_index: 900
timestamp_seconds: 40.0 # Decelerated: 12fps effective for starlight

```

8.2 Frame Index to PTS Mapping

Narrative PTS Mapping: Acceleration and Deceleration

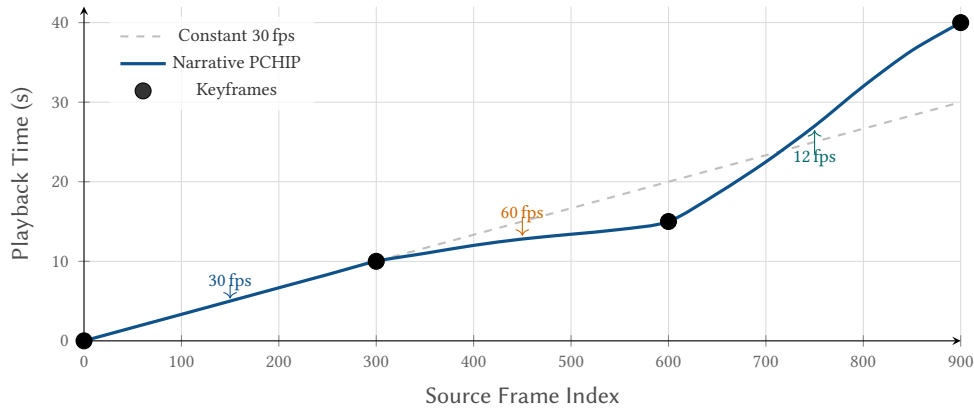


Figure 22. Frame index to PTS mapping. The slope of the curve represents the effective playback speed: steeper segments are faster (acceleration during sunset), shallower segments are slower (deceleration during starlight).

8.3 Instantaneous Speed Profile

The derivative of the PTS mapping curve yields the instantaneous playback speed—a direct measure of how the viewer experiences time. This “speed profile” is the primary creative control surface for the timelapse editor.

Instantaneous Playback Speed Profile

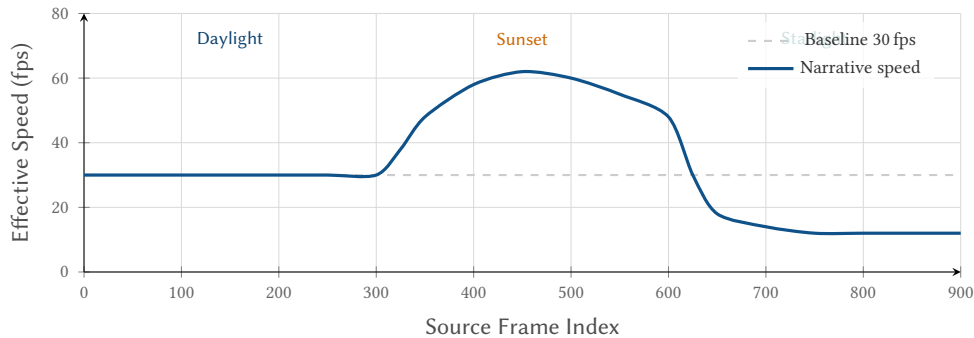


Figure 23. Instantaneous playback speed derived from the PTS mapping of fig. 22. The speed ramps up during the sunset transition (orange region) and settles at a slow 12 fps for the starlight hero segment (teal region). The smooth transitions are a direct consequence of the C^1 -continuous PCHIP interpolation.

: The Law of Temporal Pacing

Transitions (twilight, sunrise) should be *accelerated* to emphasise the shift in ambient light, while core narrative events (the Milky Way crossing the meridian) should be *decelerated* to provide the viewer time to absorb the visual detail.

: Speed Ratio Guidelines

- **Daylight / static scenes:** 24–30 fps effective (natural motion).
- **Transitions (sunset/sunrise):** 48–120 fps effective (2–4× acceleration).
- **Hero moments (astro, weather):** 8–16 fps effective (0.5–0.3× deceleration).
- **Maximum safe ratio:** 6× between adjacent segments to avoid jarring speed shifts.

9 Synthetic Motion Blur via Temporal Convolution

In segments where narrative pacing is slowed, the lack of native motion blur results in a “staccato” motion signature. Each source frame was captured with a shutter speed appropriate for the acquisition interval, not the (longer) playback interval. To mitigate this, the engine implements temporal frame blending.

9.1 Temporal Convolution Kernel

For an output frame at index i with blur window B , the blended pixel values are computed as a uniform (box) temporal average:

$$F'(i) = \frac{1}{|W_i|} \sum_{j \in W_i} F(j), \quad W_i = \{j : |j - i| \leq B, 0 \leq j < N\} \quad (27)$$

The window W_i contains up to $2B + 1$ frames, with asymmetric truncation at sequence boundaries (identical to the luminance smoothing boundary handling in section 3).

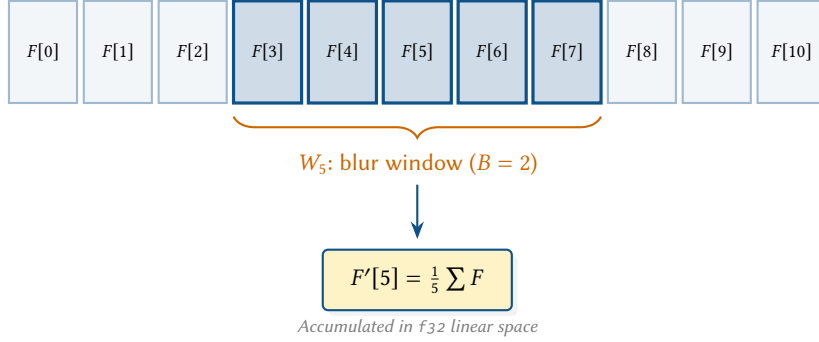


Figure 24. Temporal accumulation buffer for synthetic motion blur with $B = 2$. Five frames ($2B + 1 = 5$) contribute equally to the blended output frame $F'[5]$.

9.2 Kernel Selection: Gaussian vs Linear

The engine supports two temporal convolution kernels, selectable via `-blur-weight`:

gaussian (default)

A normalised Gaussian kernel emphasises the centre frame while smoothly tapering contributions from temporal neighbours:

$$G[k] = \frac{1}{Z} \exp\left(-\frac{k^2}{2\sigma^2}\right), \quad \sigma = \frac{B}{2}, \quad Z = \sum_{k=-B}^B G[k] \quad (28)$$

This produces a “physical shutter” simulation where the primary frame dominates the exposure.

linear

A uniform (boxcar) kernel assigns equal weight to all frames in the window:

$$L[k] = \frac{1}{2B + 1}, \quad k \in [-B, B] \quad (29)$$

This maximises temporal averaging and produces a stronger motion blur effect at the cost of reduced temporal sharpness.

Blur Kernel Comparison ($B = 3$)

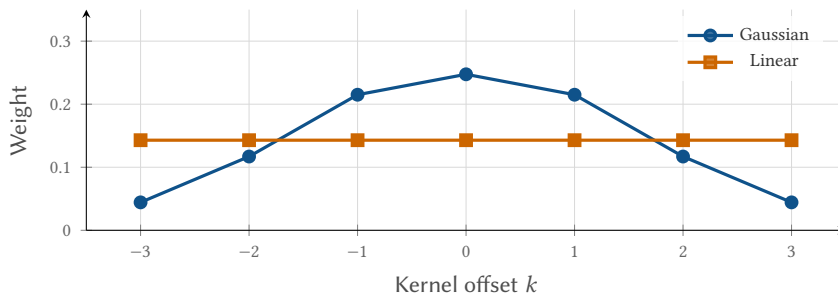


Figure 25. Comparison of Gaussian and linear (uniform) blur kernels for $B = 3$. The Gaussian kernel ($\sigma = 1.5$) concentrates $\approx 25\%$ of the weight on the centre frame, producing a sharper result. The linear kernel distributes weight uniformly ($\approx 14.3\%$ each), producing stronger averaging.

Both kernels sum to 1.0 by construction, ensuring that the blended output preserves the overall luminance level. The kernel is pre-computed once before the encoding loop and stored as a `Vec<f32>`.

9.3 Linear-Space Blending

Why Not sRGB?

Frame blending *must* be performed in linear light space. Averaging gamma-encoded values produces perceptually incorrect results: a 50/50 blend of black (0.0) and white (1.0) in sRGB-encoded space yields $0.5_{\text{sRGB}} \approx 0.214_{\text{linear}}$ —too dark. In linear space, the blend correctly yields 0.5_{linear} , which corresponds to the physical midpoint of photon flux.

The engine performs all blending in the 16-bit linear domain. The accumulator is promoted to f32 to prevent integer overflow when summing up to $2B + 1$ frames of u16 data (maximum sum: $(2B + 1) \times 65535$, which exceeds 2^{16} for $B \geq 1$).

9.4 Memory Implications

Each 16-bit 4K frame occupies approximately:

$$3840 \times 2160 \times 3 \times 2 = 49,766,400 \text{ bytes} \approx 47.5 \text{ MB}$$

A blur window of $B = 5$ requires $2 \times 5 + 2 = 12$ frames in the LRU cache, consuming ≈ 570 MB of resident memory. See section 16 for the cache capacity analysis.

10 Optical-Flow Guided Blending

The standard temporal convolution (section 9) produces “ghosting” artefacts in scenes with significant inter-frame motion—the blended output contains translucent echoes of moving objects. Optical-flow guided blending addresses this by warping each neighbouring frame toward the centre frame before accumulation, aligning moving objects across the temporal window.

10.1 Farneback Dense Optical Flow

The engine computes dense optical flow between each neighbour frame and the centre frame using the Farneback algorithm [1]. For each pixel (x, y) , the flow field $\mathbf{f}(x, y) = (f_x, f_y)$ estimates the displacement vector that maps the pixel in the neighbour frame to its corresponding position in the centre frame.

The Farneback parameters are tuned for timelapse motion characteristics:

Parameter	Value	Rationale
pyr_scale	0.5	Standard pyramid downscale factor
levels	3	Sufficient for typical timelapse inter-frame motion
win_size	15	Captures large displacements (clouds, shadows)
iterations	3	Balance between accuracy and speed
poly_n	5	Polynomial expansion neighbourhood
poly_sigma	1.2	Gaussian smoothing for polynomial expansion

10.2 Bidirectional Warp and Bilinear Interpolation

Each neighbour frame $F[n + k]$ is remapped toward the centre frame $F[n]$ using the computed flow field. For each output pixel (x, y) in the warped frame:

$$F_{\text{warp}}[n + k](x, y) = F[n + k](x + f_x(x, y), y + f_y(x, y)) \quad (30)$$

Since the source coordinates are generally non-integer, bilinear interpolation is applied:

$$F(x', y') = (1 - \alpha)(1 - \beta)F_{\lfloor x' \rfloor, \lfloor y' \rfloor} + \alpha(1 - \beta)F_{\lfloor x' \rfloor, \lceil y' \rceil} + (1 - \alpha)\beta F_{\lceil x' \rceil, \lfloor y' \rfloor} + \alpha\beta F_{\lceil x' \rceil, \lceil y' \rceil} \quad (31)$$

where $\alpha = x' - \lfloor x' \rfloor$ and $\beta = y' - \lfloor y' \rfloor$.

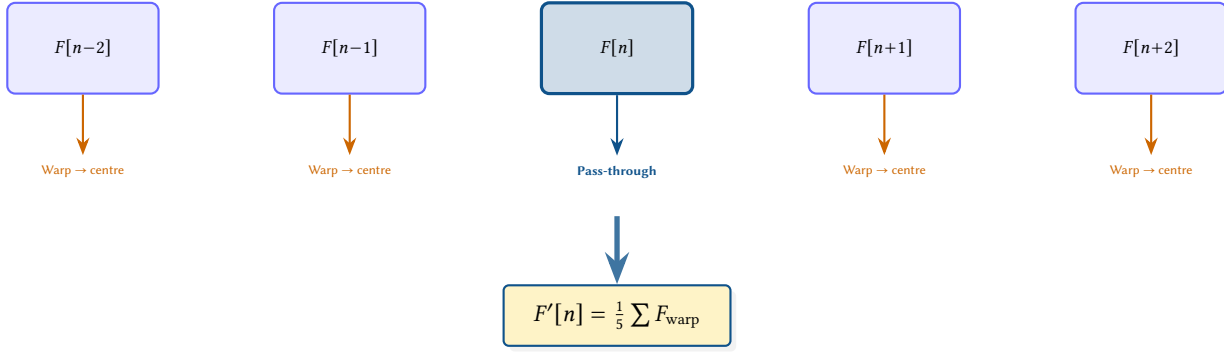


Figure 26. Optical-flow guided blending pipeline. Each neighbour frame is warped toward the centre frame using dense flow vectors before averaging, eliminating ghosting artefacts from inter-frame motion.

10.3 Feature Gating and Graceful Fallback

Optical flow requires OpenCV, a substantial system dependency. The implementation is gated behind a Cargo feature flag:

Listing 2. Feature-gated OpenCV dependency

```
[features]
flow = ["opencv"]

[dependencies]
opencv = { version = "0.92", optional = true }
```

The user enables flow blending with `-blur-method flow`. If flow computation fails for any frame pair (e.g., due to degenerate motion), the engine logs a warning and falls back to the standard temporal average for that frame, ensuring robustness.

11 Narrative Audio Time-Stretching

Professional timelapse productions frequently pair the visual sequence with an audio track. When narrative ramping (section 8) compresses or expands temporal segments, the audio must be correspondingly time-stretched to maintain synchronisation without introducing pitch artefacts.

11.1 Audio Pipeline Architecture

The audio pipeline consists of four stages: decode, stretch, write, and mux.

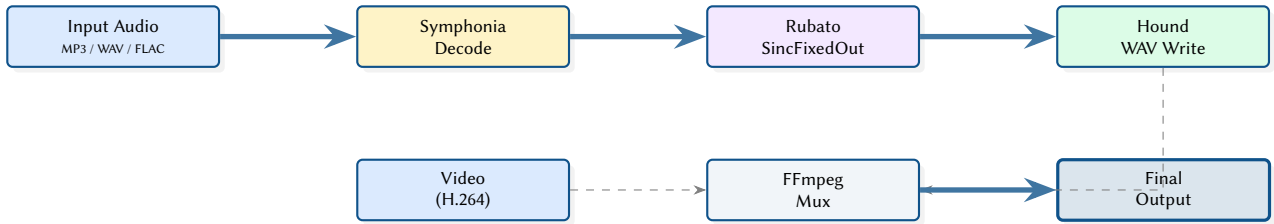


Figure 27. Audio time-stretching pipeline. The input audio is decoded via symphonia, time-stretched using rubato’s sinc interpolation resampler, written as a temporary WAV file via hound, and finally muxed with the video track using FFmpeg CLI.

11.2 Stretch Ratio Calculation

The global stretch ratio R is derived from the ratio of video duration (PCHIP output) to real-world capture duration (EXIF timestamps):

$$R = \frac{T_{\text{video}}}{T_{\text{capture}}} = \frac{\text{PTS}[N-1] - \text{PTS}[0]}{t_{\text{capture}}[N-1] - t_{\text{capture}}[0]} \quad (32)$$

where $\text{PTS}[n]$ is the PCHIP-interpolated presentation timestamp and $t_{\text{capture}}[n]$ is the EXIF capture time. For a typical timelapse that compresses 2 hours of real time into 30 seconds of video, $R \approx 0.004$.

11.3 Pitch-Preserved Resampling

The engine uses rubato’s `SincFixedOut` resampler with windowed sinc interpolation. Key parameters:

Parameter	Value	Effect
Sinc length	256	High-quality interpolation kernel
Cutoff	0.95	Preserves 95% of the Nyquist bandwidth
Interpolation	Linear	Fast per-sample interpolation
Window	Blackman-Harris	Excellent sidelobe suppression (−92 dB)

The resampler operates on fixed-size chunks (1024 samples), processing each audio channel independently. This preserves the original pitch while stretching or compressing the temporal axis, producing a natural-sounding result even at extreme ratios.

12 Professional Ecosystem Integration

The engine provides deep integration with professional photography and NLE tools through four export channels: XMP sidecars, XMEML timeline manifests, Resolve CSV edit lists, and HDR bracket manifests.

12.1 XMP Sidecar Anatomy

Each source RAW file receives a companion .xmp file containing the calculated exposure offset and optional HDR stacking metadata. The following shows a sidecar with bracket stacking:

Listing 3. Generated XMP Sidecar with HDR Stack Metadata

```
<x:xmpmeta xmlns:x="adobe:ns:meta/">
  <rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#">
    <rdf:Description
      xmlns:crs="http://ns.adobe.com/camera-raw-settings/1.0/"
      xmlns:lr="http://ns.adobe.com/lightroom/1.0/"
      crs:ProcessVersion="15.4"
      crs:Exposure2012="-0.97"
      crs:HasSettings="True"
      lr:stacked="True"
      lr:stackId="a3f2e1d4-..."
      lr:stackPosition="0"
      lr:topOfStack="True">
    </rdf:Description>
  </rdf:RDF>
</x:xmpmeta>
```

The `lr:` namespace attributes enable Lightroom to automatically group bracketed exposures into stacks on import, with the reference frame displayed as the stack thumbnail.

12.2 Workflow Integration

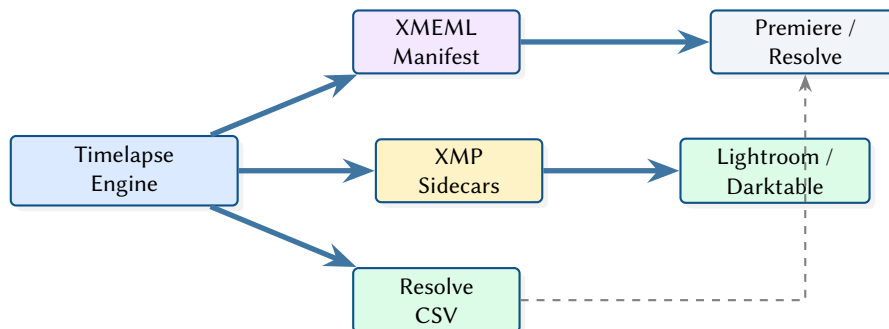


Figure 28. Multi-format export pipeline. The engine simultaneously generates XMP sidecars (for Lightroom), XMEML manifests (for FCP/Premiere), and Resolve CSV edit lists—ensuring the PCHIP narrative timing is preserved across all professional tools.

12.3 Field Mapping

Internal Field	XMP Tag	Notes
ev_offset	crs:Exposure2012	Formatted to 2 decimal places
—	crs:ProcessVersion	Hardcoded 15.4
—	crs:HasSettings	Always True
bracket.stack_id	lr:stackId	UUID for bracket group
bracket.position	lr:stackPosition	Position within stack
bracket.is_reference	lr:topOfStack	Reference frame flag

12.4 HDR Bracket Detection

The engine automatically identifies exposure-bracketed frame sets by analysing EXIF metadata patterns. The detection algorithm operates in two passes:

Pass 1 (Temporal Clustering): Consecutive frames are clustered when their EXIF capture timestamps differ by less than a configurable threshold (default 2.0 s). Rapid bursts of captures indicate potential bracket sets.

Pass 2 (EV Spread Analysis): Within each temporal cluster, the relative exposure value is computed from EXIF:

$$EV_{rel}[n] = \log_2(t_n) - 2 \log_2(N_n) + \log_2\left(\frac{ISO_n}{100}\right) \quad (33)$$

If the cluster contains ≥ 2 distinct EV levels (separated by ≥ 0.5 stops), it is tagged as a bracket group. The reference frame is selected as the frame closest to the median EV.

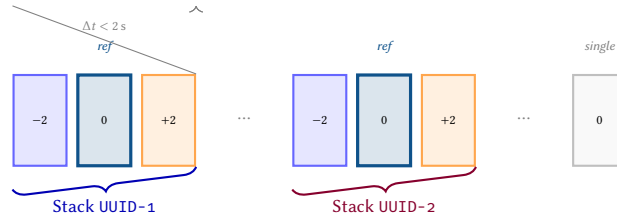


Figure 29. HDR bracket detection. Temporally clustered frames ($\Delta t < 2$ s) with distinct EV levels are grouped into bracket stacks identified by UUID. The reference frame (closest to median EV) is marked with `lr:topOfStack = True`.

Detected bracket groups are exported as a `BRACKET_MANIFEST.json` file containing stack IDs, member paths, and relative EV offsets for consumption by external HDR merging tools.

12.5 XMEML Timeline Export

The engine generates Final Cut Pro 7 XML (XMEML v4) manifests that encode the PCHIP narrative timing as native NLE timeline metadata. Each source frame becomes a `<clipitem>` with `<start>` and `<end>` computed from the PCHIP presentation timestamps:

$$\text{start}[n] = \lfloor \text{PTS}[n] \cdot \text{fps} \rfloor, \quad \text{end}[n] = \lfloor \text{PTS}[n + 1] \cdot \text{fps} \rfloor \quad (34)$$

This allows Premiere Pro, DaVinci Resolve, and Final Cut Pro to import the sequence with variable frame durations that exactly reproduce the narrative speed ramps.

12.6 DaVinci Resolve CSV Export

For Resolve-specific workflows, the engine also exports a CSV edit list with SMPTE timecodes. The timecode conversion follows the standard HH:MM:SS:FF format:

$$\text{SMPTE}(t, \text{fps}) = \left\lfloor \frac{F}{3600 \cdot \text{fps}} \right\rfloor : \left\lfloor \frac{F \bmod (3600 \cdot \text{fps})}{60 \cdot \text{fps}} \right\rfloor : \left\lfloor \frac{F \bmod (60 \cdot \text{fps})}{\text{fps}} \right\rfloor : (F \bmod \text{fps}) \quad (35)$$

where $F = \lfloor t \cdot \text{fps} \rfloor$ is the total frame count.

Column	Description
Clip Name	Source RAW filename
Source In	Source timecode (always 00:00:00:00 for stills)
Source Out	Source timecode (clip duration)
Record In	Timeline record-in timecode
Record Out	Timeline record-out timecode

Future Extension

Chromatic correction factors (`color_correction`) could be mapped to `crs:Temperature` and `crs:Tint` for white balance round-tripping, though this would require converting the per-channel multipliers to the Lightroom temperature/tint colour space.

13 CLI Reference

The engine is invoked via a command-line interface built with `clap` v4.5. All parameters have sensible defaults; only `-input` is required.

13.1 Flag Reference

Short	Long	Type	Default	Description
-i	-input	PathBuf	<i>required</i>	Input directory containing RAW files
-o	-output	PathBuf	output.mp4	Output video file path
-f	-fps	usize	24	Output video frame rate
-w	-window	usize	10	Deflicker smoothing window size
—	-sigma	f64	2.0	Sigma threshold for luminance outlier clipping
-b	-blur	usize	0	Temporal blur window (0 = disabled)
—	-blur-weight	String	gaussian	Blur kernel: gaussian or linear
—	-blur-method	String	average	Blur method: average or flow
-n	-narrative	PathBuf	<i>none</i>	Path to narrative plan YAML file
-c	-codec	String	h264	Output codec: h264, hevc, prores
—	-crf	u8	18	Constant Rate Factor (0–51)
—	-bitrate	usize	<i>none</i>	Bitrate in bps (overrides CRF)
—	-gamut	String	srgb	Colour gamut: srgb, rec2020, acescg
—	-vignette	f64	0.0	Radial vignette normalisation strength
—	-max-ev	f64	8.0	Maximum Holy Grail EV correction (stops)
—	-no-cache	bool	false	Force full re-analysis
—	-dry-run	bool	false	Stop after analysis (skip encoding)
—	-preview	usize	<i>none</i>	Render every <i>N</i> th frame
—	-audio	PathBuf	<i>none</i>	Audio file to time-stretch and mux
—	-hdr-group	String	auto	HDR bracket detection: auto or disable
—	-hdr-time	f64	2.0	Bracket clustering time threshold (s)
—	-stability	bool	true	Enable spatiotemporal stability map
—	-project	PathBuf	<i>none</i>	Project file for full state persistence
—	-re-render	String	all	Re-render scope: all or changed

13.2 Common Workflows

Listing 4. Basic deflicker-only workflow

```
timelapse -i ./raw_photos -o sunset.mp4 -f 30 -w 15
```

Listing 5. Holy Grail with narrative pacing and motion blur

```
timelapse -i ./holy_grail -o epic.mp4 -f 24 \  
-w 20 --sigma 1.8 -b 3 -n narrative.yaml
```

Listing 6. ProRes with wide-gamut and audio

```
timelapse -i ./raw_photos -o master.mov -c prores \  
--gamut rec2020 --vignette 0.3 --audio ambient.mp3
```

Listing 7. HDR bracket detection with project persistence

```
timelapse -i ./brackets -o hdr_tl.mp4 \  
--hdr-group auto --project timelapse.json --dry-run
```

IMPLEMENTATION ARCHITECTURE

14 System Architecture

The engine is implemented in Rust, leveraging `rsraw` (LibRaw bindings) for RAW ingestion, `ndarray` for array processing, `rayon` for data parallelism, and `ffmpeg-next` (FFmpeg bindings) for H.264 encoding. The pipeline consists of 18 discrete stages executed in a fixed order.

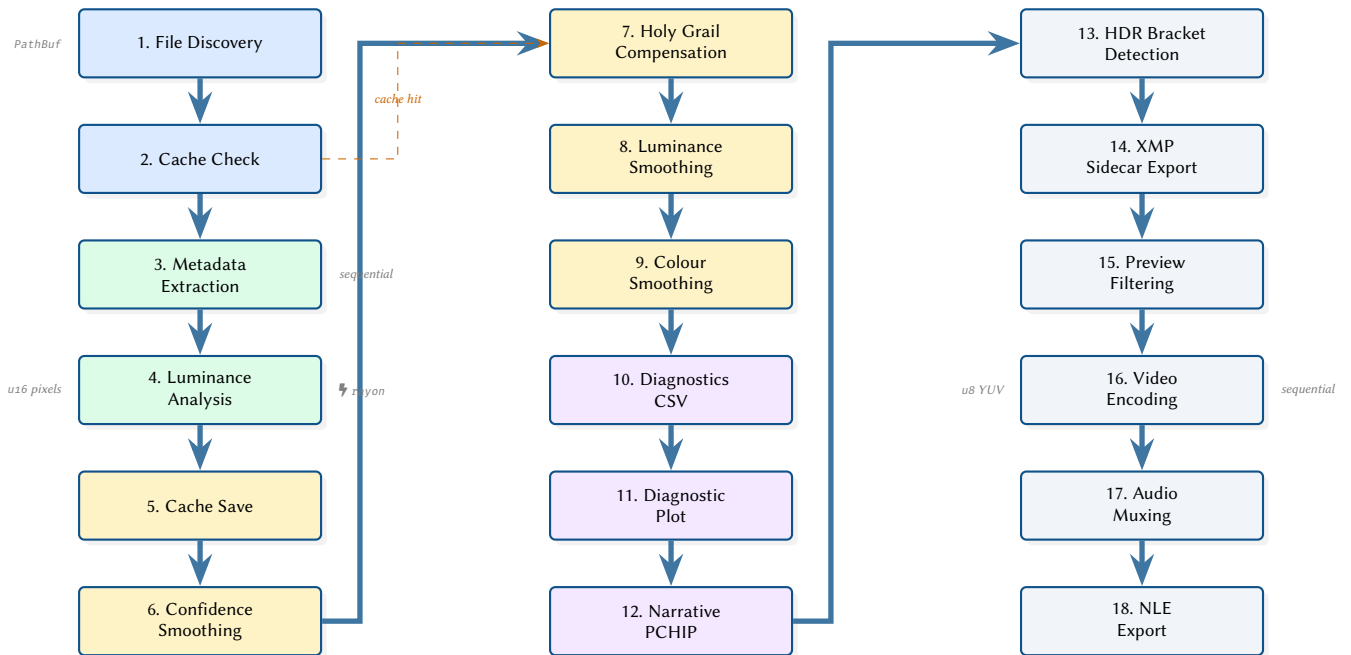


Figure 30. Complete 18-stage pipeline architecture. Blue stages handle discovery and analysis; yellow stages perform correction; purple stages produce output artefacts. The dashed orange line shows the cache-hit fast path that bypasses stages 3–6.

14.1 Crate Dependency Map

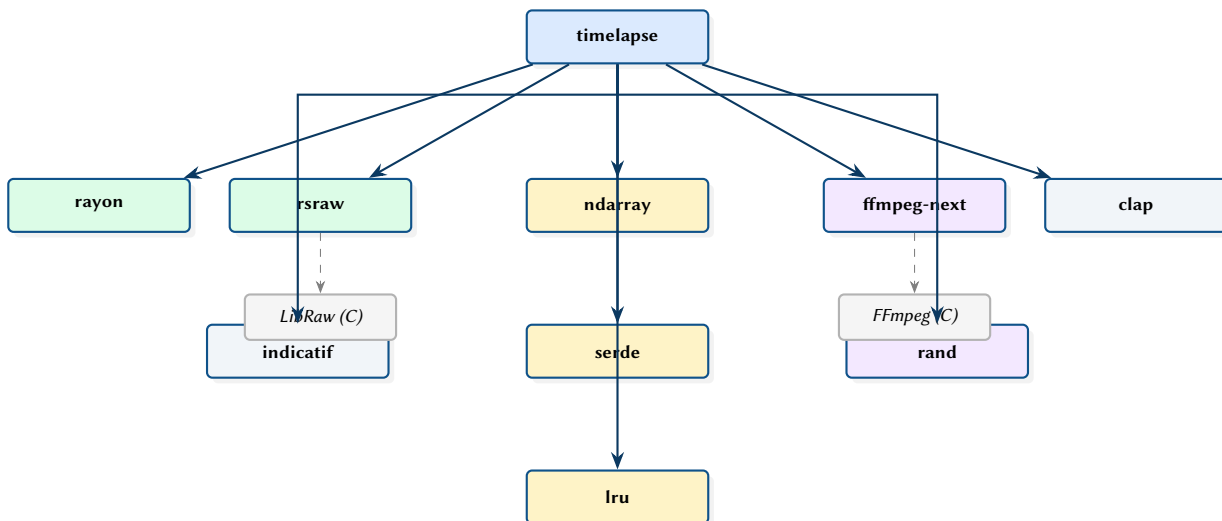


Figure 31. Crate dependency map. Solid arrows show Rust crate dependencies; dashed arrows show FFI bindings to C libraries. The engine’s core processing relies on ndarray for array operations and rsraw/ffmpeg-next as bindings to LibRaw and FFmpeg respectively.

14.2 Stage Descriptions

The 18 stages execute in strict sequential order, with one exception: stage 4 (luminance analysis) is parallelised via rayon. The stages group into three logical phases:

Analysis Phase (Stages 1–6)

Stage 1: File Discovery scans the input directory for supported RAW extensions (arw, cr2, nef, dng, raw, tiff, jpg, jpeg), sorts alphabetically, and creates an initial `Vec<ImageFrame>` with path and index populated. **Stage 2: Cache Check** looks for `.timelapse_cache.json` in the input directory; if the cached frame count matches the discovered file count, stages 3–6 are skipped. **Stage 3: Metadata Extraction** reads EXIF data (shutter speed, aperture, ISO, focal length) via `rsraw` sequentially. **Stage 4: Luminance Analysis** decodes each RAW file at 16-bit depth using `rayon`’s `par_iter_mut` for data parallelism, computes CIE 1931 luminance per pixel, and applies sigma-clipping to produce the frame’s brightness and `rgb_means`. **Stage 5: Cache Save** persists the frame vector as pretty-printed JSON. **Stage 6: Temporal Confidence Smoothing** applies sequential exponential blending to frames with

low sigma-clipping confidence ($\rho < 0.7$), stabilising brightness estimates before downstream correction (section 4.3).

Correction Phase (Stages 7–12)

Stage 7: Holy Grail Compensation computes cumulative EV offsets from EXIF metadata deltas (section 5). **Stage 8: Luminance Smoothing** applies the temporal boxcar filter and computes per-frame correction factors. **Stage 9: Colour Smoothing** independently smooths R, G, B channels for chromatic deflickering. **Stage 10: Diagnostics CSV** writes the audit trail to `flicker_diagnostics.csv`. **Stage 11: Diagnostic Plot** generates a 4-pane PNG visualisation (`flicker_report.png`) via `plotters` showing brightness trend, EV compensation, correction factors, and confidence scores (section 22). **Stage 12: Narrative PCHIP** interpolates user-provided keyframes to assign PTS timestamps to each frame.

Output Phase (Stages 13–18)

Stage 13: HDR Bracket Detection groups frames into exposure bracket sets based on EXIF capture-time proximity and exports a bracket manifest (section 12.4). **Stage 14: XMP Sidecar Export** writes an Adobe-compliant `.xmp` file alongside each RAW source with the calculated exposure offset and bracket metadata. **Stage 15: Preview Filtering** optionally subsamples every N th frame for rapid preview rendering. **Stage 16: Video Encoding** decodes RAW frames through the LRU cache, applies combined corrections in 16-bit space, optionally performs temporal blur (average or flow-guided), dithers to 8-bit, and encodes H.264/HEVC/ProRes via `ffmpeg-next`. **Stage 17: Audio Muxing** time-stretches an optional audio track to match the output duration and muxes it into the final video. **Stage 18: NLE Export** writes XMEML for FCP/Premiere and a Resolve-compatible CSV with SMPTE timecodes.

14.3 Parallelism Architecture

Rayon parallelism is used in exactly one stage: luminance analysis (stage 4). This stage is the most compute-intensive because it requires a full RAW demosaic per frame. The `par_iter_mut` distributes frames across all available cores, with each thread independently decoding and analysing a single frame. Thread-safe progress reporting is provided by `indicatif` with the `rayon` feature flag.

The remaining stages are sequential for the following reasons:

- **Metadata extraction** (stage 3): sequential file I/O dominates; parallelism yields marginal benefit.
- **Smoothing** (stages 8–9): the rolling mean is an inherently sequential scan.
- **Video encoding** (stage 18): the `ffmpeg-next` encoder requires frames in strict order, and the LRU cache is not thread-safe.

15 Data Model

15.1 ImageFrame Structure

The central data structure is `ImageFrame`, which accumulates metadata and computed corrections as it passes through the pipeline stages.

Field	Type	Semantics
<code>path</code>	<code>PathBuf</code>	Absolute filesystem path to the source RAW file
<code>index</code>	<code>usize</code>	Sequential frame number (0-indexed)
<code>timestamp</code>	<code>f64</code>	Initially 0.0; overwritten by PCHIP to become PTS
<code>shutter_speed</code>	<code>f64</code>	Exposure time in seconds (from EXIF)
<code>aperture</code>	<code>f64</code>	F-number (from EXIF)
<code>iso</code>	<code>f64</code>	ISO sensitivity (from EXIF)
<code>focal_length</code>	<code>f64</code>	Focal length in mm (extracted, reserved)
<code>brightness</code>	<code>Option<f64></code>	Sigma-clipped mean luminance (16-bit linear)
<code>rgb_means</code>	<code>Option<[f64; 3]></code>	Per-channel sigma-clipped means (16-bit linear)
<code>ev_offset</code>	<code>f64</code>	Cumulative EV compensation (Holy Grail)
<code>correction_factor</code>	<code>f64</code>	Deflicker multiplier (smoothed / actual brightness)
<code>color_correction</code>	<code>[f64; 3]</code>	Per-channel R/G/B chromatic correction multipliers

Field	Type	Semantics
target_fps	f64	Reserved field (default: 24.0)
gamut	ColorGamut	Colour space used during analysis
state_hash	String	BLAKE3 hash for CAS caching
confidence	Option<f64>	Sigma-clipping inlier ratio (0.0–1.0)
capture_time	f64	EXIF capture time (Unix epoch seconds)
bracket	Option<BracketInfo>	HDR bracket grouping metadata

15.2 NarrativePlan and NarrativeKeyframe

Struct	Field	Type
NarrativePlan	keyframes	Vec<NarrativeKeyframe>
NarrativeKeyframe	frame_index	usize
	timestamp_seconds	f64

15.3 Frame Lifecycle

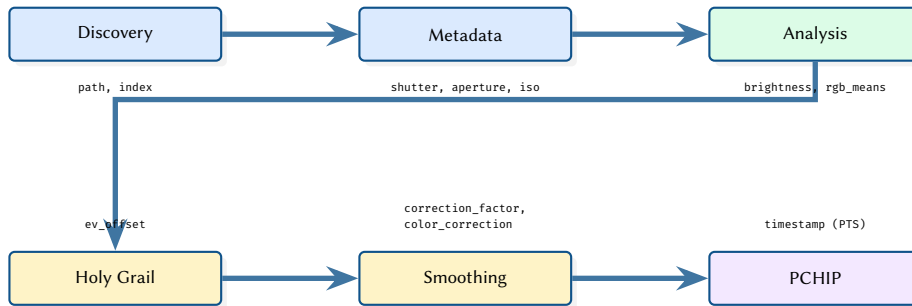


Figure 32. Field population through pipeline stages. Each stage writes specific fields to the ImageFrame struct; downstream stages consume earlier fields.

16 LRU Frame Caching

Motion blur requires random access to a sliding window of decoded, corrected 16-bit frames. With 4K frames at ≈ 47.5 MB each, re-decoding from RAW for every access would be prohibitively slow. The engine maintains an in-memory Least Recently Used (LRU) cache.

16.1 Cache Sequence

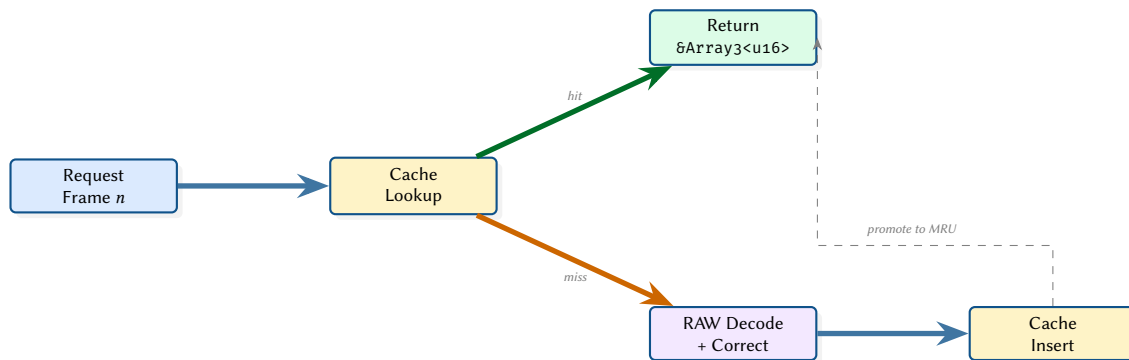


Figure 33. LRU cache lookup sequence. On a cache miss, the frame is decoded from RAW, corrected in 16-bit linear space, and inserted into the cache. On a hit, the existing `Array3<u16>` is returned by reference.

16.2 Capacity Formula

The cache capacity is set to:

$$C = \max(2B + 2, 1) \quad (36)$$

where B is the blur window size (`-blur`).

Hit-Rate Guarantee

The blur loop iterates linearly through frames, and for each output frame i it accesses frames in $[\max(0, i-B), \min(N-1, i+B)]$. The maximum number of simultaneously required frames is $2B+1$ (the full window). A cache of size $2B+2$ guarantees that when the loop advances from frame i to $i+1$:

1. The new frame $i+B+1$ (one miss) is decoded and inserted.
2. The oldest frame $i-B$ (now outside the window) is evicted.
3. All other $2B$ frames remain cached (hits).

This achieves a **100% hit rate** for all frames within the blur window, with exactly **one cache miss per output frame** (the leading edge of the window).

16.3 Sliding Window Advancement

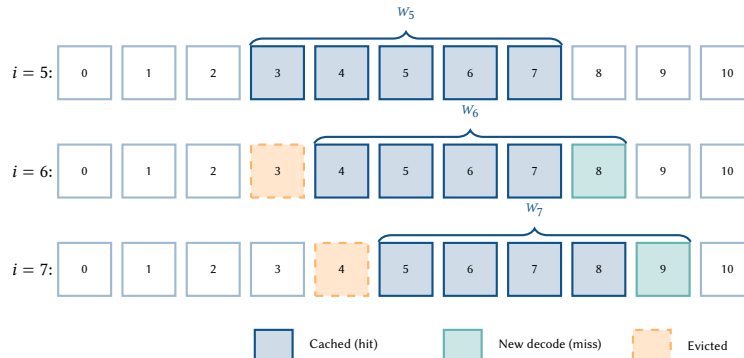


Figure 34. LRU cache sliding window advancement with $B = 2$. As the output frame advances from $i = 5$ to $i = 7$, the window slides right: one frame is decoded (teal) and the trailing frame is evicted (dashed orange). Interior frames remain cached.

16.4 Memory Budget

For 4K frames ($3840 \times 2160 \times 3 \times 2 = 47.5$ MB per frame):

Blur (B)	Capacity (C)	Memory	Frames in Window
0	2	95 MB	1
1	4	190 MB	3
3	8	380 MB	7
5	12	570 MB	11
10	22	1.05 GB	21

17 Persistent Metadata Caching

RAW metadata extraction and sigma-clipped luminance analysis are the most expensive per-frame operations (each frame requires a full RAW demosaic at 16-bit depth). The engine persists these results to avoid redundant recomputation.

17.1 Cache Format

The cache is stored as `.timelapse_cache.json` in the input directory, containing the full `Vec<ImageFrame>` serialised via `serde_json` with pretty-printing. Fields cached include: `path`, `index`, `shutter_speed`, `aperture`, `iso`, `focal_length`, `brightness`, and `rgb_means`.

Listing 8. Excerpt from `.timelapse_cache.json`

```
[
  {
    "path": "/photos/DSC_0001.ARW",
    "index": 0,
    "shutter_speed": 0.008,
    "aperture": 5.6,
    "iso": 200.0,
    "focal_length": 24.0,
    "brightness": 31205.4,
    "rgb_means": [28100.2, 33500.8, 29800.1],
    "ev_offset": 0.0,
    "correction_factor": 1.0,
  }
]
```

```

    "color_correction": [1.0, 1.0, 1.0],
    "target_fps": 24.0,
    "timestamp": 0.0
  }
]

```

17.2 Cache Invalidation

The invalidation strategy is deliberately simple: the cached frame count is compared against the discovered file count. If they match, the cache is considered valid.

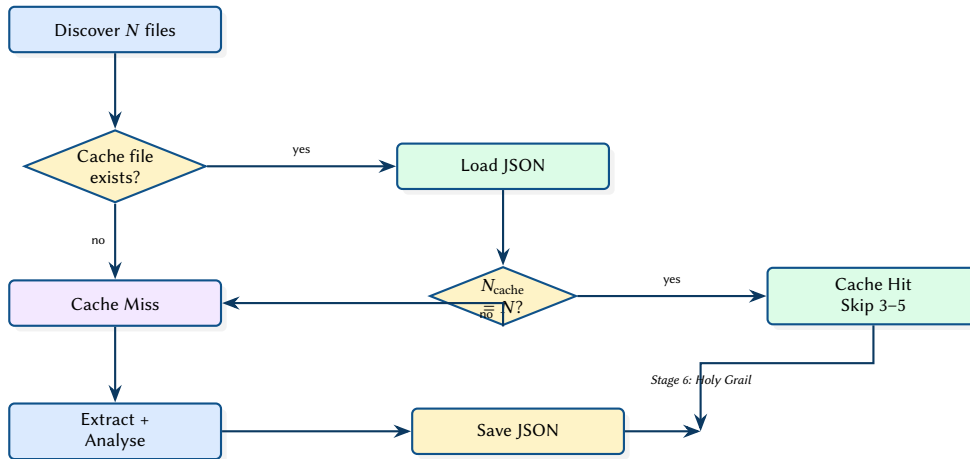


Figure 35. Cache invalidation decision tree. The only validation criterion is frame count equality—a deliberate trade-off favouring simplicity and speed over strict correctness.

This handles the primary use case (re-running with different narrative or smoothing parameters on the same image set) while avoiding the complexity of per-file checksums.

Scenario	Cache Behaviour
Same directory, same files	Hit — stages 3–4 skipped
Files added or removed	Miss — full re-analysis
Files renamed (same count)	Hit (stale) — user must delete cache
Different <code>-window</code> or <code>-sigma</code>	Hit — smoothing always re-runs

17.3 Performance Impact

On a typical 1000-frame sequence with Sony ARW files (~ 25 MB each):

Operation	Approx. Time
First run (extraction + analysis)	Several minutes
Cached re-run (smoothing + encode only)	Seconds (metadata)

The cache converts an $O(N \times \text{demosaic})$ operation into an $O(1)$ JSON deserialisation, enabling rapid iteration on narrative parameters and smoothing windows without re-processing RAW data.

18 Full Project Persistence

While the metadata cache (section 17) stores analysis results for incremental re-processing, the *project file* provides complete state persistence across sessions. This extends the cache into a self-contained project archive.

18.1 ProjectState Structure

Field	Type	Semantics
version_hash	String	Cache invalidation key (pkg version + algorithm version)
settings	ProjectSettings	Complete snapshot of all CLI parameters
frames	Vec<ImageFrame>	Full frame data including analysis, corrections, brackets
narrative	Option<NarrativePlan>	Narrative keyframes (if any)

ProjectSettings captures all user-facing parameters: sigma, gamut, vignette, window, max_ev, codec, crf, bitrate, blur, blur_weight, and fps. This enables the engine to detect when settings have changed and trigger selective re-processing.

18.2 Load/Save Lifecycle

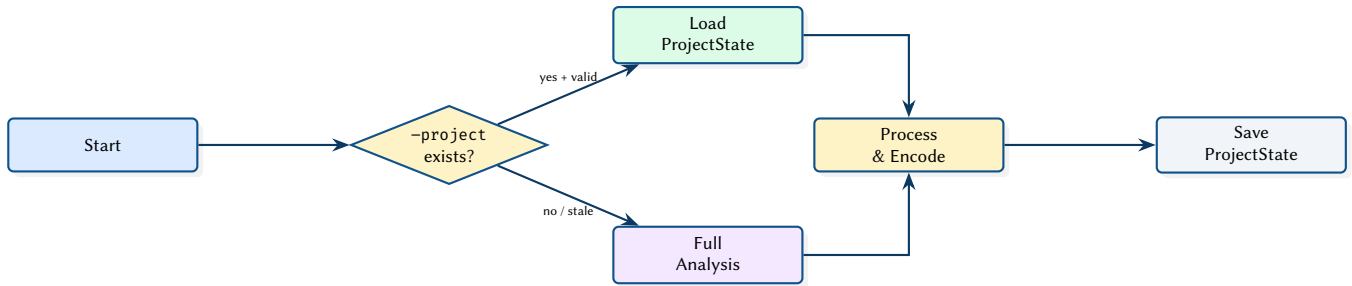


Figure 36. Project persistence lifecycle. If a valid project file exists with a matching version hash, the engine skips analysis and loads the full state directly. After processing, the updated state is saved back.

18.3 Partial Re-Rendering

The `-re-render` changed flag leverages the per-frame state_hash (BLAKE3, section 17) to identify frames whose source file or processing parameters have changed since the last render. Only these frames are re-decoded and re-encoded, enabling near-instantaneous updates when adjusting narrative timing or colour grading on a large sequence.

19 16-Bit Linear Pipeline

19.1 RAW Sensor Data and Demosaicing

Digital camera sensors capture light through a Colour Filter Array (CFA), most commonly the Bayer pattern. Each photosite records only one colour channel; full-colour pixels are reconstructed via demosaicing interpolation.

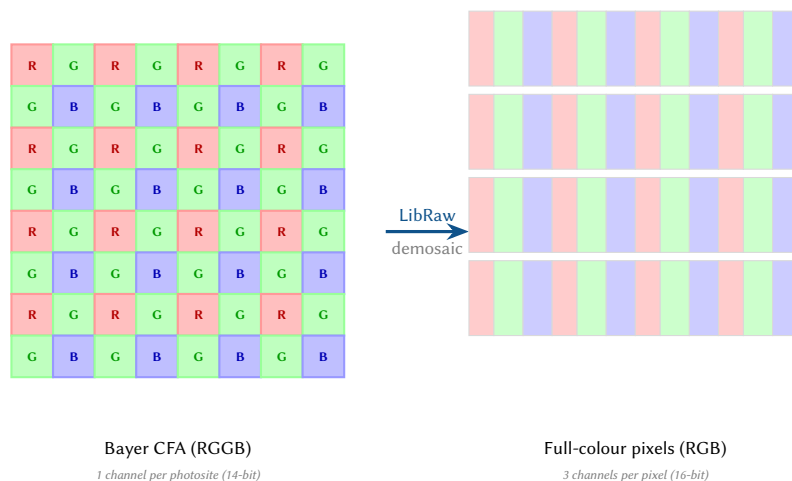


Figure 37. Bayer RGGB colour filter array to full-colour pixel reconstruction. The sensor captures one colour per photosite; rsraw (LibRaw) performs the demosaicing interpolation, promoting to 16-bit depth. Note the 2:1 green-to-red/blue ratio, matching the human eye's peak sensitivity at 555 nm.

The engine requests 16-bit output from LibRaw (`process::<BIT_DEPTH_16>()`), ensuring the full 14-bit sensor range is preserved with headroom for correction multipliers that may push values above the original maximum.

19.2 Precision Through the Pipeline

The engine maintains maximum precision by operating in 16-bit linear space throughout the processing stages. The bit depth is only reduced at the final encoding step.

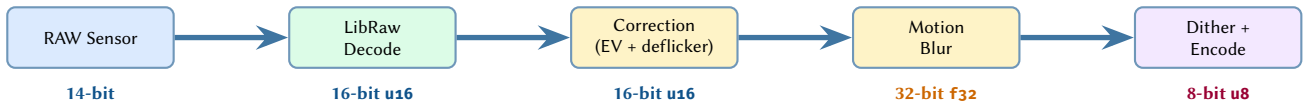


Figure 38. Bit-depth precision through pipeline stages. The 14-bit sensor data is promoted to 16-bit during decode, maintained through correction, temporarily promoted to 32-bit float during blur accumulation, and finally dithered down to 8-bit for H.264 encoding.

19.3 Quantisation Error Analysis

The quantisation step size q determines the minimum resolvable luminance difference at each bit depth:

Bit Depth	Levels	Step Size q	Shadow Impact
8-bit	256	$1/256 \approx 0.39\%$	Visible banding below $\sim 10\%$
14-bit	16,384	$1/16384 \approx 0.006\%$	Imperceptible
16-bit	65,536	$1/65536 \approx 0.0015\%$	Well below noise floor

In the shadow region (bottom 10% of the dynamic range), an 8-bit pipeline has only ~ 25 usable levels, while the 16-bit pipeline maintains $\sim 6,500$ levels—a $260\times$ improvement in tonal resolution. This is critical for timelapse sequences that transition into nighttime scenes where the majority of pixel values fall in the bottom few percent of the dynamic range.

Relative Quantisation Error: 8-bit vs 16-bit

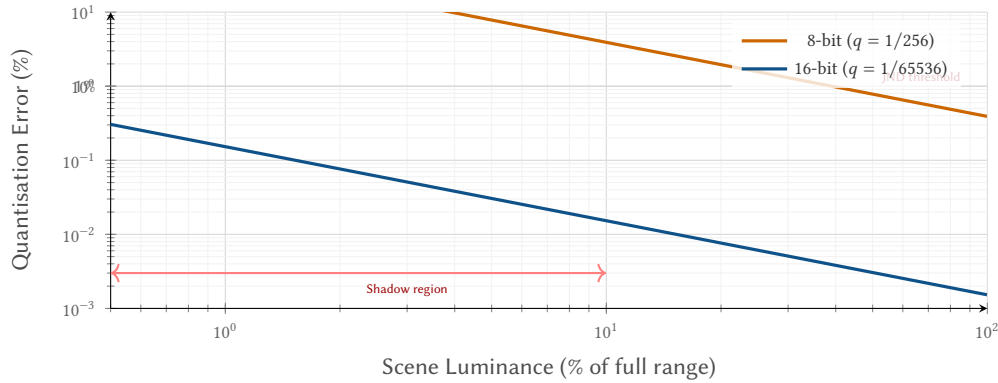


Figure 39. Relative quantisation error as a function of scene luminance on log-log scale. The 8-bit pipeline (orange) exceeds the $\sim 1\%$ Just Noticeable Difference (JND) threshold below $\sim 40\%$ scene luminance, producing visible banding. The 16-bit pipeline (blue) remains well below the JND threshold across the entire dynamic range.

19.4 Array Memory Layout

The engine uses ndarray with shape (height, width, 3) in row-major (C-contiguous) order. For a 4K frame:

$$\text{Array3<u16> : } 2160 \times 3840 \times 3 \times 2 \text{ bytes} = 49,766,400 \text{ bytes}$$

Scanline (row-major, channel-last): stride = 2 bytes per u16

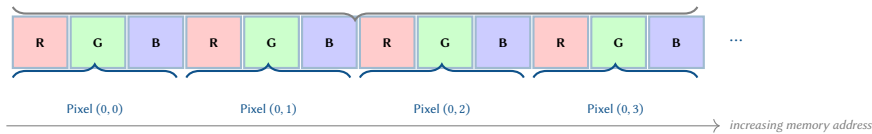


Figure 40. ndarray channel-last memory layout. Each pixel's R, G, B values are contiguous in memory, enabling efficient per-pixel correction (all three channels accessed with spatial locality). Scanlines are laid out consecutively in row-major order.

The channel-last layout enables efficient per-pixel operations (all three channels are contiguous in memory), though it is less cache-friendly for per-channel operations. The correction loop iterates over pixels in scanline order, applying the combined multiplier to all three channels:

$$\text{pixel}[c] = \text{clamp}(\text{pixel}[c] \times C[n] \times 2^{\Omega[n]} \times \text{color}[c], 0, 65535) \quad (37)$$

where $C[n]$ is the deflicker correction, $\Omega[n]$ is the EV offset, and $\text{color}[c]$ is the per-channel chromatic correction.

20 Encoding and Dithering Implementation

20.1 H.264 YUV 4:2:0 Setup

The final encoding stage uses `ffmpeg-next` (direct FFmpeg bindings) to configure the encoder programmatically. The encoder accepts `ndarray::Array3<u8>` frames in RGB format; the RGB → YUV 4:2:0 conversion is handled by FFmpeg's `libswscale` via a `scaling::Context`.

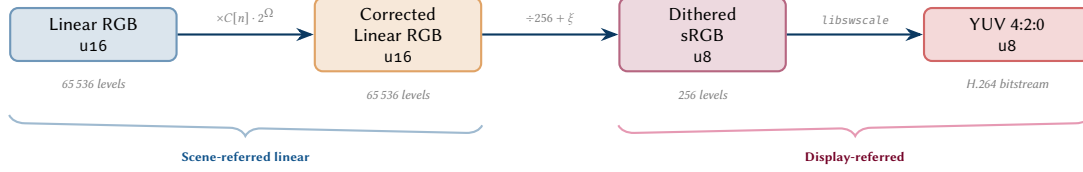


Figure 41. Colour space conversion chain. All corrections are applied in the 16-bit scene-referred linear domain. The transition to display-referred space occurs only at the final dither/quantise step; the subsequent RGB→YUV conversion is handled by FFmpeg.

20.2 Per-Pixel Correction Application

Before dithering, each 16-bit pixel receives the combined correction. The total linear multiplier is computed once per frame and applied to every pixel:

$$M_{\text{total}} = C[n] \times 2^{\Omega[n]} \quad (38)$$

The per-channel application promotes each u16 value to f64, multiplies by $M_{\text{total}} \times \text{color_correction}[c]$, clamps to $[0, 65535]$, and truncates back to u16. This arithmetic is exact to within the precision of IEEE 754 double-precision (~ 15 significant digits), introducing no measurable quantisation error.

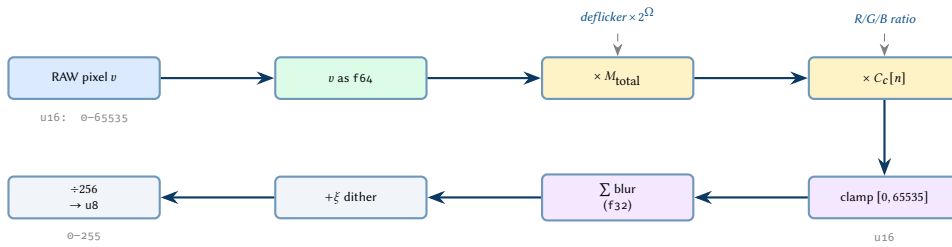


Figure 42. Per-pixel correction pipeline. Each pixel is promoted to f64 for correction, clamped back to u16, optionally accumulated with blur neighbours in f32, then dithered and quantised to u8 for encoding.

20.3 Dithering Code Path

The engine implements two code paths depending on whether motion blur is enabled:

20.3.1 Without Motion Blur ($B = 0$)

Each u16 pixel is directly dithered to u8:

$$\text{out} = \text{clamp}\left(\left\lfloor \frac{v + \xi}{256} \right\rfloor, 0, 255\right), \quad \xi \sim \mathcal{U}(-0.5, +0.5) \quad (39)$$

20.3.2 With Motion Blur ($B > 0$)

The blur accumulator is an `Array3<f32>` containing the sum of $|W_i|$ frames. Dithering and normalisation are combined in a single step:

$$\text{out} = \text{clamp}\left(\left\lfloor \frac{v + \xi}{|W_i| \times 256} \right\rfloor, 0, 255\right) \quad (40)$$

where v is the accumulated f32 value and $|W_i|$ is the number of frames in the blur window.

The RNG is `rand::thread_rng()` (`ThreadRng`), providing cryptographically strong randomness. While a faster PRNG (e.g., `SmallRng`) would suffice for dithering, the overhead of `ThreadRng` is negligible compared to the RAW decode and array operations.

DIAGNOSTICS & VALIDATION

21 Diagnostic CSV Export

The engine writes a `flicker_diagnostics.csv` file to the input directory after every run. This audit trail enables scientific validation of the noise rejection profile and exposure compensation accuracy.

21.1 Column Definitions

Column	Type	Description
frame_index	usize	Sequential frame number (0-indexed)
raw_brightness	f64	Sigma-clipped mean luminance before correction (16-bit scale)
smoothed_brightness	f64	Rolling mean luminance after temporal smoothing
ev_offset	f64	Cumulative EV compensation from Holy Grail de-stepping
correction_factor	f64	Final deflicker multiplier ($\hat{Y}[n]/Y_{\text{obs}}[n]$)

21.2 Luminance Curve Visualisation

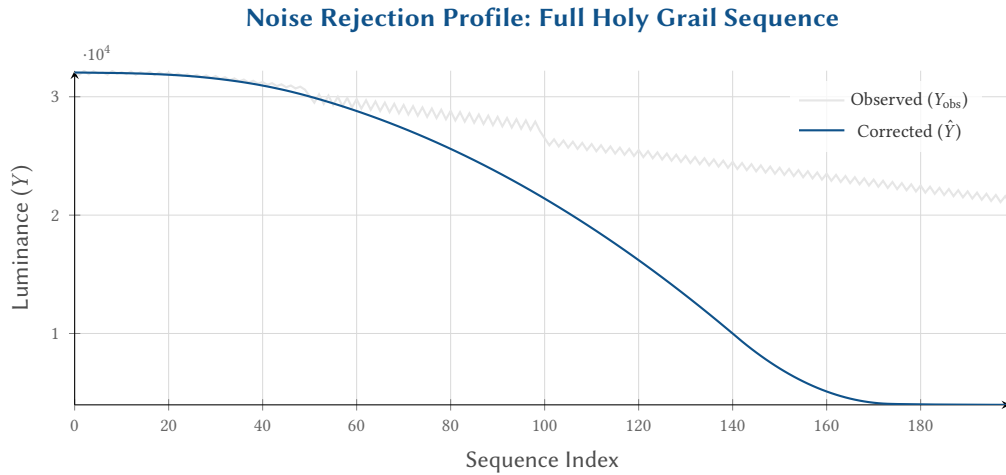


Figure 43. Full Holy Grail diagnostic output: 200 frames transitioning from daylight (high luminance) through twilight to night. The raw signal (grey) shows stochastic flicker and discrete exposure steps; the smoothed trend (blue) captures the continuous luminance descent. Discrete EV jumps at frames ~ 50 , ~ 100 , and ~ 141 are clearly visible in the raw signal and successfully flattened by the compensation algorithm.

22 Visual Diagnostic Report

In addition to the CSV audit trail, the engine generates a four-pane PNG report (`flicker_report.png`) via the `plotters` crate, providing an at-a-glance visual summary of the photometric pipeline.

22.1 Pane Layout

The report is rendered at 1280×1440 pixels (four 1280×360 panes stacked vertically):

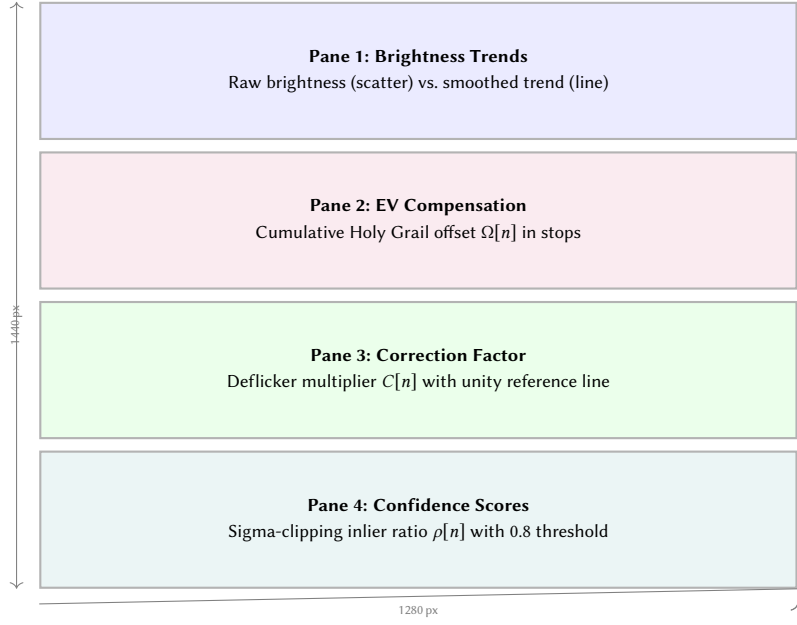


Figure 44. Layout of the four-pane diagnostic PNG (`flicker_report.png`). Each pane plots a different photometric quantity against the frame index, enabling rapid visual inspection of the entire pipeline’s behaviour.

The plot is generated at stage 11 of the pipeline, immediately after the CSV export. It is particularly useful for identifying sequences where the Holy Grail compensation saturates the EV clamp (visible as plateaux in pane 2) or where transient objects cause confidence dips (visible as outliers in pane 4).

23 Validation Methodology

23.1 Deflicker Quality Verification

The deflicker quality can be assessed by computing the residual flicker metric—the standard deviation of the correction factors:

$$\sigma_{\text{residual}} = \text{std} \left(\left\{ \frac{Y_{\text{obs}}[n]}{\hat{Y}[n]} \right\}_{n=0}^{N-1} \right) \quad (41)$$

: Deflicker Quality Thresholds

- $\sigma_{\text{residual}} < 0.005$: Excellent — no visible flicker.
- $0.005 \leq \sigma_{\text{residual}} < 0.02$: Good — perceptible only in extreme slow-motion.
- $\sigma_{\text{residual}} \geq 0.02$: Poor — increase window size or reduce sigma threshold.

23.2 PCHIP Monotonicity Validation

Monotonicity of the narrative timeline can be verified by checking the finite differences of the generated timestamps:

$$\Delta T[n] = T[n+1] - T[n] \geq 0 \quad \forall n \in [0, N-2] \quad (42)$$

Any negative ΔT indicates a temporal reversal—a bug in the interpolation logic. In practice, the Fritsch–Carlson theorem provides a mathematical guarantee of monotonicity (not merely an empirical one), so this validation is primarily a regression test.

Monotonicity Check: Finite Differences of PTS

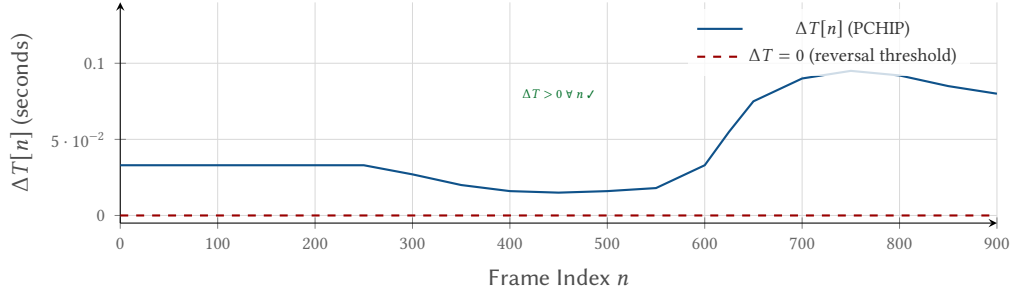


Figure 45. Finite differences of the PCHIP-generated timeline. All values are strictly positive, confirming monotonicity. Smaller values correspond to faster playback (more source frames per second of output); larger values correspond to slower, decelerated segments.

23.3 EV Compensation Accuracy

The accuracy of Holy Grail compensation can be audited by comparing the engine's computed ΔEV values against the known EXIF metadata. For each transition where the camera adjusts exposure:

1. Extract N , t , ISO from consecutive EXIF records.
2. Compute the expected ΔEV analytically via eq. (16).
3. Compare with `ev_offset` in the diagnostic CSV.
4. Verify that $|EV_{\text{expected}} - EV_{\text{actual}}| < 0.01$ stops (the deadband threshold).

EV Offset Accumulation: Holy Grail Sequence

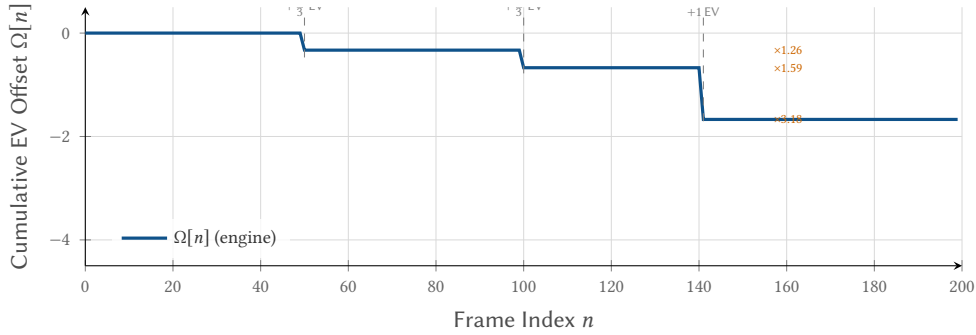


Figure 46. Cumulative EV offset for the mock Holy Grail sequence from fig. 43. Each step corresponds to a camera exposure change detected via EXIF metadata. The annotations on the right show the linear multiplier $2^{-\Omega}$ applied to pixel data at each plateau.

23.4 Before/After Correction Quality

The combined effect of all correction stages can be summarised by measuring the frame-to-frame luminance variance before and after processing. For a well-deflickered sequence, the coefficient of variation $CV = \sigma/\mu$ of consecutive frame luminances should approach the natural scene gradient.

Flicker Reduction Through Pipeline Stages

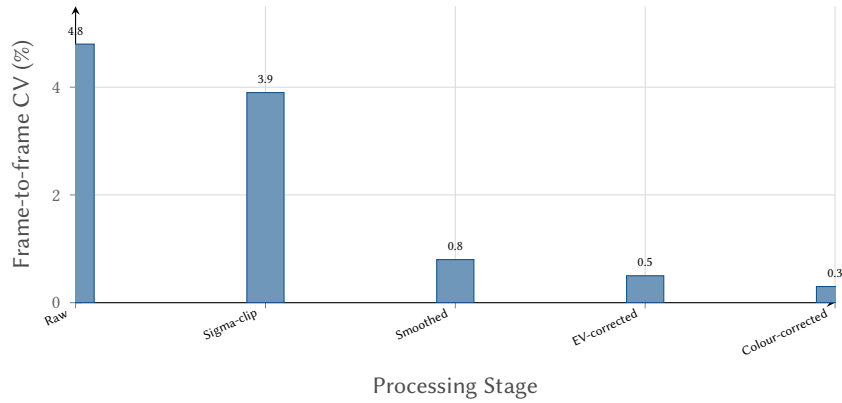


Figure 47. Progressive flicker reduction through pipeline stages (illustrative). Raw frame-to-frame luminance variance is dominated by mechanical noise (~5% CV); sigma-clipping removes outlier pixels; smoothing suppresses the stochastic component; EV correction flattens discrete steps; colour correction eliminates residual per-channel drift.

24 Future Directions

While the current engine covers the core timelapse pipeline, eleven major extension domains would transform it from a single-sequence processor into a comprehensive timelapse production platform. This section presents the architectural vision for each domain, followed by the original performance and quality extensions.

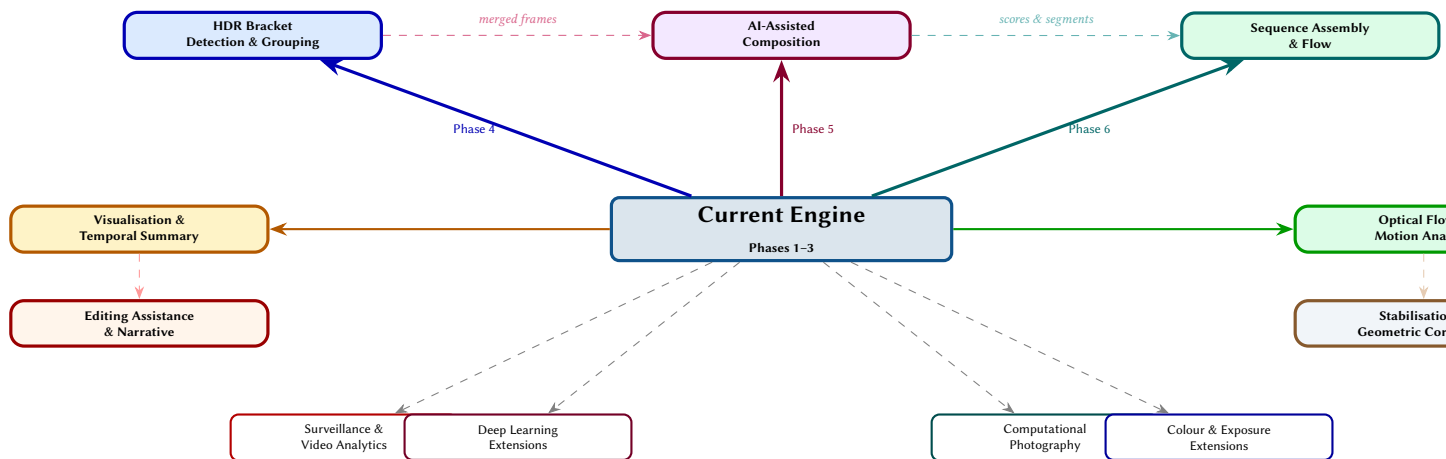


Figure 48. Extension roadmap. The original three feature domains (top, Phases 4–6) are joined by eight new extension domains spanning visualisation, editing, motion analysis, stabilisation, surveillance, deep learning, computational photography, and colour management. Performance extensions integrate independently.

24.1 AI-Assisted Composition Analysis

The second extension domain introduces machine learning to the analysis stage, transforming the engine from a purely signal-processing tool into an intelligent creative assistant.

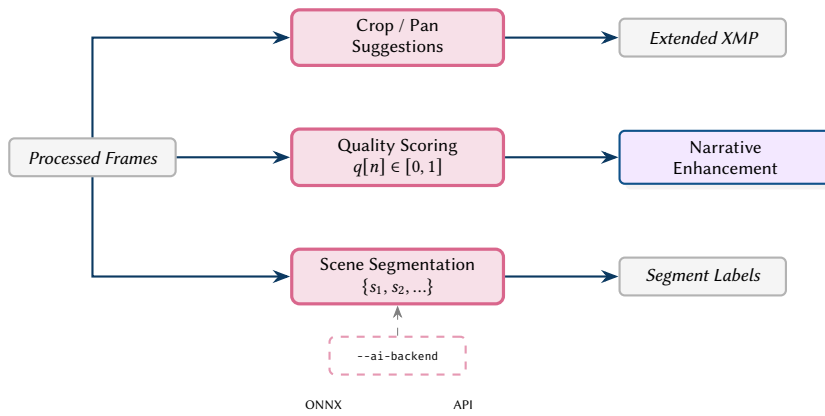


Figure 49. AI analysis pipeline. Three parallel analysis passes feed into narrative enhancement, extended XMP metadata, and scene labelling. The inference backend is configurable between local ONNX and external API.

24.1.1 Standout Frame Detection

A scoring model assigns each frame a quality score $q[n] \in [0, 1]$ based on light quality, colour harmony, contrast, and compositional interest. These scores feed back into the narrative planner: high-scoring frames can automatically receive longer hold durations or slower playback in the PCHIP timeline, emphasising the strongest moments without manual keyframe placement.

: AI Score Integration

Quality scores are *suggestions*, not overrides. The narrative YAML remains the authoritative timeline definition. Scores are surfaced as weighted recommendations that the user can accept, modify, or ignore. The engine never alters the user's explicit keyframes.

24.1.2 Crop & Pan Suggestions

For source material shot at higher resolution than the output (e.g., 8K source for 4K output), the engine can detect subjects, horizon lines, and regions of interest, then generate smooth crop keyframe paths—effectively a virtual Ken Burns pan/zoom. Suggestions are stored as non-destructive metadata:

```

{
  "crop_keyframes": [
    {"frame": 0, "x": 0.1, "y": 0.05, "w": 0.8, "h": 0.8},
    {"frame": 120, "x": 0.3, "y": 0.10, "w": 0.6, "h": 0.6},
    {"frame": 240, "x": 0.1, "y": 0.05, "w": 0.8, "h": 0.8}
  ]
}

```

The crop path is interpolated using the same PCHIP machinery (section 6) to guarantee smooth, monotonic camera movement without overshoot.

24.1.3 Exposure & Development Suggestions

Histogram analysis detects frames with clipped highlights, crushed shadows, or colour casts, and generates per-frame Lightroom-compatible development suggestions written as extended XMP sidecar fields:

```

{
  "frame": "IMG_0042.ARW",
  "suggestions": {
    "crs:Exposure2012": +0.3,
    "crs:Highlights2012": -40,
    "crs:Shadows2012": +25,
    "crs:Temperature": 5200
  },
  "reason": "highlight_clipping_detected"
}

```

These complement the existing Holy Grail EV compensation (section 5) by addressing per-frame tonal issues that temporal smoothing cannot correct.

24.1.4 Scene Segmentation

The engine partitions the timelapse into semantically meaningful segments by detecting inflection points in the luminance trend, colour temperature shift, and content change rate. Each segment receives a label (e.g.,

golden_hour, blue_hour, night, cloud_movement) that can serve as narrative chapter markers or inform the sequence assembly stage (section 24.2).

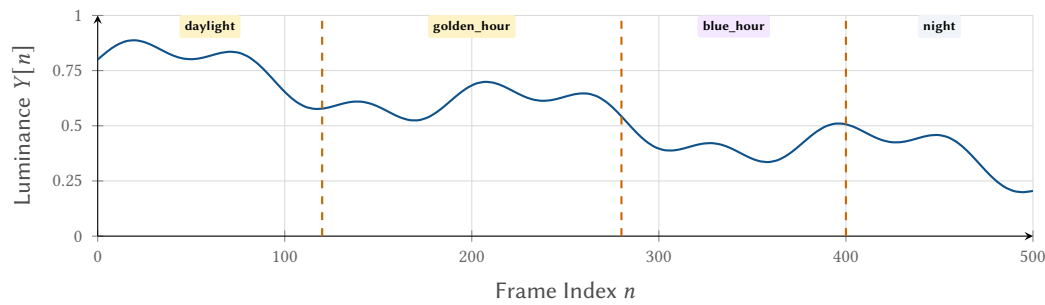


Figure 50. Scene segmentation via luminance gradient analysis. Inflection points in the smoothed luminance curve define segment boundaries, each assigned a semantic label for narrative structuring.

24.2 Sequence Assembly & Flow

The third extension domain addresses a fundamental workflow limitation: the current engine processes a single directory of RAW files as one linear sequence. Real-world timelapse projects often comprise multiple shooting sessions—different locations, different days, different camera angles—that need to be assembled into a cohesive narrative.

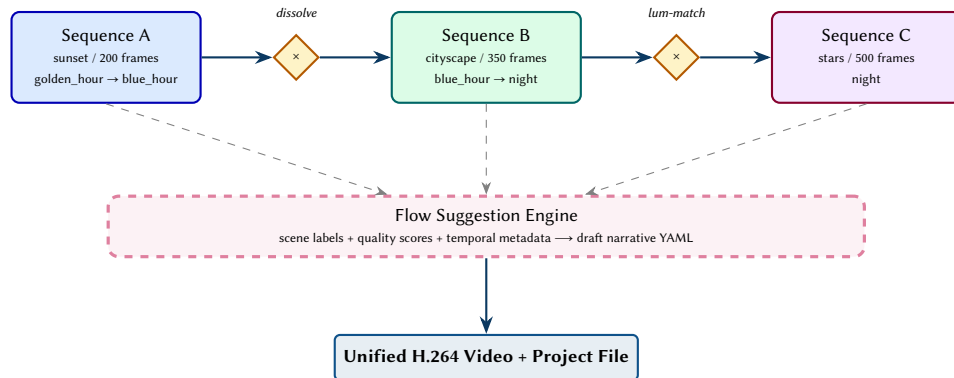


Figure 51. Multi-sequence assembly. Independent shooting sessions are combined via the Flow Suggestion Engine, which uses scene labels, quality scores, and temporal metadata to propose an ordering with appropriate transitions. The output is a unified video and a persistent project file.

24.2.1 Multi-Sequence Ingestion

The engine accepts multiple input directories, each as a named sequence:

```
timelapse \
  --input sunset:/path/to/sunset \
  --input city:/path/to/cityscape \
  --input stars:/path/to/stars \
  --project timelapse.json \
  --output final.mp4
```

Each sequence maintains independent metadata, analysis caches, and bracket groups, but shares a unified output timeline and project file.

24.2.2 Intelligent Flow Suggestions

The Flow Suggestion Engine consumes scene segmentation labels (section 24.1), quality scores, and temporal metadata (time-of-day from EXIF, sequence duration) to propose:

1. **Sequence ordering** based on temporal continuity (sunset → blue hour → night) or thematic contrast.
2. **Transition points** at segment boundaries where luminance and colour temperature align, minimising visible jumps.
3. **Hold durations** on standout frames identified by the quality scorer, weighted by their position in the narrative arc.

The output is a draft narrative YAML that the user can refine before rendering.

24.2.3 Transition Generation

At sequence boundaries, the engine generates smooth transitions in 16-bit linear space:

$$F_{\text{out}}[n] = (1 - \alpha[n]) \cdot F_A[n_A] + \alpha[n] \cdot F_B[n_B], \quad \alpha \in [0, 1] \quad (43)$$

where $\alpha[n]$ is a PCHIP-interpolated blend curve ensuring C^1 continuity. Before blending, the engine applies luminance and white balance matching between the tail of sequence A and the head of sequence B , preventing the “exposure pop” artefact common in naive cross-dissolves.

24.2.4 Project State Persistence

A JSON project file persists the full state across sessions:

```
{
  "version": "1.0",
  "sequences": [
    { "name": "sunset", "path": "/path/to/sunset", "frames": 200 },
    { "name": "city", "path": "/path/to/cityscape", "frames": 350 }
  ],
  "bracket_groups": [...],
  "ai_analysis": {
    "quality_scores": "cache/scores.json",
    "segments": "cache/segments.json",
    "crop_suggestions": "cache/crops.json"
  },
  "narrative": "narrative.yaml",
  "user_overrides": {
    "hold_frames": [42, 187, 312],
    "excluded_frames": [99]
  }
}
```

This enables iterative refinement: users can re-run AI analysis, adjust suggestions, re-merge HDRs, and re-render without losing prior work. The cache invalidation strategy (section 17) extends to track per-sequence and per-analysis-stage freshness.

24.3 Visualisation & Temporal Summary

Timelapse sequences encode rich temporal information that is difficult to assess from individual frames or even from playback at speed. This domain introduces a family of visualisation tools that compress temporal dynamics into static or interactive representations, enabling rapid quality assessment, creative exploration, and metadata-driven storytelling.

24.3.1 Temporal Summary Strip

The simplest whole-sequence overview is the *temporal summary strip*: a single-row image where each column corresponds to one frame. Each column is filled with the spatially averaged colour (or a luminance-weighted gradient from sky to ground) of that frame, producing a compact ribbon analogous to the waveform display in audio editing.



Figure 52. Temporal summary strip. Each vertical column represents one frame’s spatially averaged colour, producing a compact visual signature of the entire sequence. Scene transitions—daylight to golden hour to blue hour—are immediately apparent.

The strip is generated by computing the spatial mean in CIE Lab space for each frame:

$$\bar{C}[n] = \frac{1}{W \cdot H} \sum_{x,y} \text{Lab}(I[n, x, y]) \quad (44)$$

and rendering each column as a vertical gradient from the upper-third mean (sky region) to the lower-third mean (ground region). The result can be exported as a PNG or embedded in an HTML report.

24.3.2 Slit-Scan Imaging

Slit-scan imaging extracts a single scanline—typically a vertical or horizontal slice at a fixed pixel coordinate—from each frame and stacks these slices sequentially, producing an image where one spatial axis encodes time.

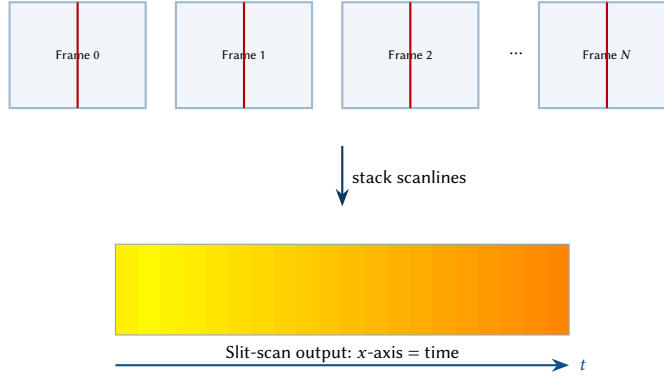


Figure 53. Slit-scan sampling geometry. A fixed vertical scanline (red) is extracted from each frame and concatenated horizontally, producing an image where the horizontal axis represents time rather than spatial position.

Formally, for a vertical scanline at column x_0 , the slit-scan image S is:

$$S[n, y] = I[n, x_0, y], \quad n \in \{0, \dots, N-1\}, \quad y \in \{0, \dots, H-1\} \quad (45)$$

where the frame index n maps to the horizontal axis of the output and y is preserved as the vertical axis. Diagonal scanlines and radial sweeps provide further creative variations.

24.3.3 Radial Time Spiral

For sequences spanning multiple days, a polar representation maps frames around a circle where one revolution corresponds to one diurnal cycle. The radius increases with each successive day, producing a spiral that reveals repeating patterns (dawn/dusk symmetry, weather cycles) and anomalies.

Each frame n with capture timestamp t_n is placed at angle θ_n and radius r_n :

$$\theta_n = 2\pi \cdot \frac{t_n \bmod T_{\text{day}}}{T_{\text{day}}}, \quad r_n = r_0 + \Delta r \cdot \left\lfloor \frac{t_n}{T_{\text{day}}} \right\rfloor \quad (46)$$

where $T_{\text{day}} = 86400$ s, r_0 is the inner radius, and Δr is the radial increment per day.

24.3.4 Temporal Statistical Composites

Statistical operations along the time axis produce composites that reveal structure invisible in any single frame. Given N registered frames $I[0], \dots, I[N-1]$, the following per-pixel statistics are computed:

$$\text{Median}[x, y] = \text{median}_n(I[n, x, y]) \quad (47)$$

$$\text{MaxChange}[x, y] = \max_n |I[n, x, y] - I[n-1, x, y]| \quad (48)$$

$$\sigma_t[x, y] = \sqrt{\frac{1}{N} \sum_{n=0}^{N-1} (I[n, x, y] - \bar{I}[x, y])^2} \quad (49)$$

The *temporal median* (eq. (47)) suppresses transient objects (pedestrians, vehicles), producing a “clean plate” of the static scene. The *max-change* map (eq. (48)) highlights regions of maximum inter-frame difference, useful for identifying areas of rapid motion. The *temporal standard deviation* (eq. (49)) quantifies per-pixel variability, producing a heatmap of temporal activity.

: Statistical Composites as Quality Diagnostics

Beyond creative output, temporal composites serve as diagnostic tools. The standard deviation map reveals sensor hot pixels (high σ_t on isolated pixels), unstable regions caused by flicker, and areas of genuine scene motion—all information that feeds back into the deflicker and stabilisation pipelines.

24.3.5 Time-Slice Composite

A time-slice composite divides the output image into K vertical strips, each sourced from a different frame uniformly spaced across the sequence:

$$C[x, y] = I\left[\left\lfloor \frac{x \cdot N}{W} \right\rfloor, x, y\right] \quad (50)$$

where W is the image width and N is the total frame count. The result compresses the entire temporal span into a single image, with smooth spatial transitions revealing how the scene evolves from left to right across time.

24.3.6 Histogram Evolution Strip

Each frame's RGB histogram is rendered as a narrow vertical bar (R, G, B channels as colour-coded intensity profiles) and stacked horizontally, producing a strip where colour distribution changes are visible at a glance. This is particularly useful for detecting white balance drift, exposure ramping artefacts, and colour cast introduced by ambient light changes.

24.3.7 Key Moment Extraction & Chapter Poster Generation

Using the quality scores from the AI analysis pipeline (section 24.1) and the scene segmentation boundaries, the engine automatically selects one representative frame per scene segment as a *chapter poster*. Selection criteria combine quality score $q[n]$, temporal centrality within the segment, and visual distinctiveness (maximum colour distance from adjacent posters):

$$n_k^* = \arg \max_{n \in S_k} [\alpha \cdot q[n] + \beta \cdot \text{centrality}(n, S_k) + \gamma \cdot \text{distinctiveness}(n)] \quad (51)$$

where S_k is the set of frame indices in segment k and α, β, γ are user-tunable weights.

24.3.8 Activity Sparkline Grid

The frame is divided into an $M \times N$ spatial grid, and for each cell a sparkline of motion magnitude over time is computed. The result is a grid of miniature time-series plots, each showing local activity patterns. Regions with periodic motion (e.g., traffic signals, tidal flows) produce characteristic waveforms, while static regions show flat lines.

```
{
  "sparkline_grid": {
    "grid_size": [8, 6],
    "metric": "motion_magnitude",
    "output": "sparklines.svg",
    "time_range": [0, 500],
    "cells": [
      {"row": 0, "col": 3, "peak_frame": 142, "mean_activity": 0.73},
      {"row": 4, "col": 1, "peak_frame": 288, "mean_activity": 0.12}
    ]
  }
}
```

24.3.9 Focus Sharpness Tracking

Per-frame sharpness is estimated via the variance of the Laplacian operator:

$$\text{Sharp}[n] = \text{Var}(\nabla^2 I[n]) = \text{Var}(I[n] * L) \quad (52)$$

where L is the discrete Laplacian kernel. Plotting $\text{Sharp}[n]$ over time reveals focus drift, vibration-induced blur, and thermal defocus events. Frames below a configurable threshold can be flagged for exclusion or replacement.

24.3.10 Exposure Event Markers & Sky-Ground Ratio Timeline

The engine detects exposure events—sudden changes in mean luminance exceeding a threshold τ_{EV} —and marks them on the timeline. Simultaneously, a sky-ground luminance ratio $R_{sg}[n] = \bar{Y}_{sky}[n] / \bar{Y}_{ground}[n]$ is tracked, providing a proxy for atmospheric conditions (clear sky $\Rightarrow$ high ratio; overcast $\Rightarrow$ ratio ≈ 1).

24.4 Editing Assistance & Narrative Intelligence

The editing assistance domain bridges the gap between raw timelapse footage and a polished narrative, automating the mechanical aspects of edit decision-making while keeping creative control with the user.

24.4.1 Change Magnitude Timeline & Auto-Keyframe Suggestion

The *change magnitude* $\Delta[n]$ quantifies the visual difference between successive frames:

$$\Delta[n] = \frac{1}{W \cdot H} \sum_{x,y} \|I[n, x, y] - I[n-1, x, y]\|_2 \quad (53)$$

Peaks in $\Delta[n]$ correspond to moments of rapid scene change—cloud shadows sweeping across a landscape, headlights appearing at dusk, or sudden weather transitions. The engine identifies local maxima exceeding a threshold τ_Δ as candidate keyframes.

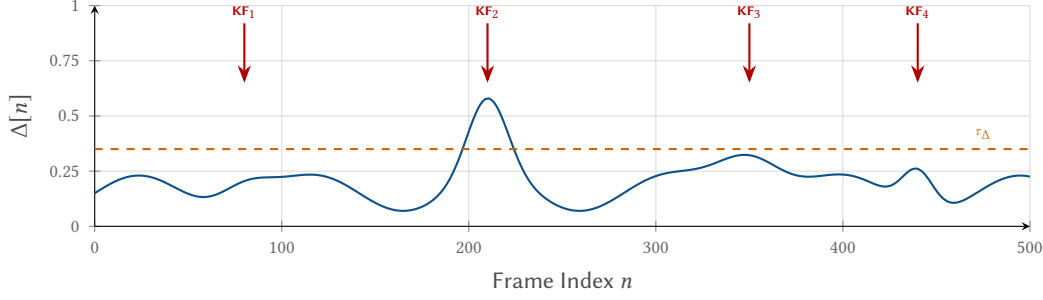


Figure 54. Change magnitude timeline with auto-suggested keyframe positions. Peaks in $\Delta[n]$ exceeding the threshold τ_Δ are proposed as narrative keyframes (red arrows). The user can accept, shift, or dismiss each suggestion.

24.4.2 Storyboard Auto-Generation

The keyframe extractor produces a contact sheet—a grid of thumbnail images at suggested keyframe positions plus segment boundaries—that serves as a visual storyboard. Each thumbnail is annotated with frame index, timestamp, quality score, and scene label:

```
{
  "storyboard": {
    "frames": [
      {"index": 0, "time": "17:30:00", "label": "daylight", "score": 0.72},
      {"index": 80, "time": "18:15:22", "label": "golden_hour", "score": 0.91},
      {"index": 210, "time": "19:42:10", "label": "blue_hour", "score": 0.88},
      {"index": 350, "time": "20:58:45", "label": "night", "score": 0.65}
    ],
    "output": "storyboard.png",
    "columns": 4
  }
}
```

24.4.3 Interactive Scrub Preview

For web-based review workflows, the engine exports an HTML file containing a lightweight JavaScript scrubber. The scrubber displays a low-resolution proxy of each frame, synchronised with the temporal summary strip (section 24.3) and change magnitude timeline. Reviewers can scrub through the sequence, click to mark favourite frames, and export their selections back to the project file.

24.4.4 Auto Speed Ramping from Motion Intensity

Motion intensity—derived from the change magnitude $\Delta[n]$ or optical flow magnitude (section 24.5)—drives automatic speed ramping. The playback rate $v[n]$ is inversely related to motion intensity, slowing down during dramatic moments and accelerating through static intervals:

$$v[n] = v_{\min} + (v_{\max} - v_{\min}) \cdot \left(1 - \frac{\Delta[n] - \Delta_{\min}}{\Delta_{\max} - \Delta_{\min}}\right) \quad (54)$$

where $v_{\min}$ and $v_{\max}$ are user-specified bounds. The resulting variable-rate timeline is smoothed with a PCHIP interpolant to avoid abrupt speed changes.

24.5 Optical Flow & Motion Analysis

Optical flow—the per-pixel displacement field between consecutive frames—is the foundation for a family of motion-aware processing techniques. This domain introduces dense flow estimation and its applications to timelapse-specific problems.

The optical flow field $\mathbf{u}[n](x, y) = (u_x, u_y)$ between frames n and $n + 1$ satisfies the brightness constancy assumption:

$$I[n + 1](x + u_x, y + u_y) \approx I[n](x, y) \quad (55)$$

In practice, dense flow is computed using variational methods (Farnebäck, 2003) or deep learning (RAFT; Teed & Deng, 2020).

24.5.1 Motion Heatmap Overlay

The motion heatmap accumulates flow magnitude across the sequence to reveal regions of persistent activity:

$$H(x, y) = \frac{1}{N - 1} \sum_{n=0}^{N-2} \|\mathbf{u}[n](x, y)\|_2 \quad (56)$$

The heatmap H is normalised and rendered as a colour overlay (blue $\rightarrow$ red) on the temporal median composite (eq. (47)), highlighting traffic corridors, pedestrian paths, cloud movement patterns, and water flow.

24.5.2 Long-Exposure Simulation via Flow Accumulation

Traditional long exposures blur moving objects along their trajectories. The engine simulates this effect by warping and accumulating K adjacent frames along their flow vectors:

$$L[n](x, y) = \frac{1}{K} \sum_{k=-\lfloor K/2 \rfloor}^{\lfloor K/2 \rfloor} I[n + k] \left(x + \sum_{j=n}^{n+k-1} u_x^{(j)}, y + \sum_{j=n}^{n+k-1} u_y^{(j)} \right) \quad (57)$$

Unlike naive temporal averaging, flow-compensated accumulation produces physically plausible motion trails while keeping static elements sharp.

24.5.3 Cinemagraph Extraction

A cinemagraph isolates a moving region within an otherwise static image. The engine segments the frame into static and dynamic regions using the temporal standard deviation map (eq. (49)), masks the dynamic region, and loops a short sub-sequence while holding the static background fixed.

24.5.4 Motion-Compensated Temporal Blending

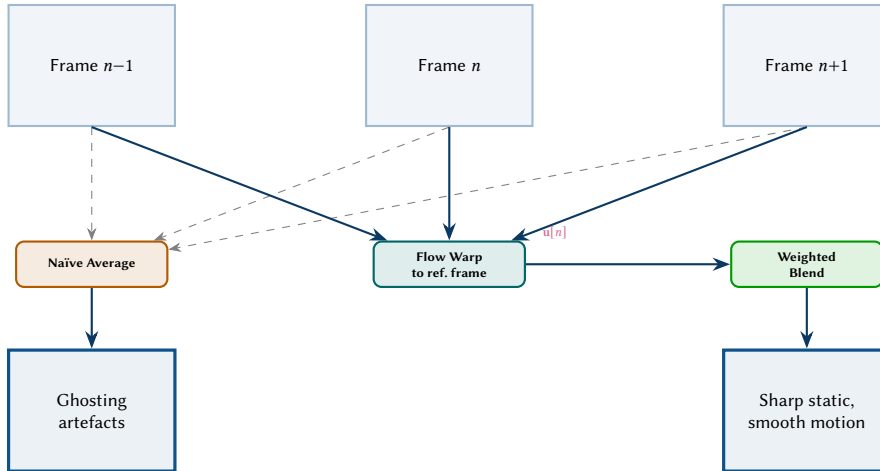


Figure 55. Motion-compensated blending (right path) vs. naïve temporal averaging (left path). Flow warping aligns neighbouring frames to the reference before blending, eliminating ghosting artefacts on moving objects while preserving noise reduction on static regions.

The flow-compensated average for frame n is:

$$\hat{I}[n](x, y) = \sum_{k=-K}^K w_k \cdot I[n + k] \left(x + U_x^{(n \rightarrow n+k)}, y + U_y^{(n \rightarrow n+k)} \right) \quad (58)$$

where $U^{(n \rightarrow n+k)}$ is the accumulated flow from frame n to frame $n + k$, and w_k are normalised Gaussian weights with $\sum_k w_k = 1$. This produces temporally smooth results without the ghosting artefacts of naïve averaging.

24.5.5 Temporal Contrast Enhancement

Local temporal contrast can be boosted by computing the deviation of each frame from the running mean and amplifying it:

$$I'[n] = \bar{I}_{\text{local}}[n] + \lambda \cdot (I[n] - \bar{I}_{\text{local}}[n]) \quad (59)$$

where $\bar{I}_{\text{local}}[n]$ is the temporally windowed mean and $\lambda > 1$ is the enhancement factor. This is particularly effective for revealing subtle cloud dynamics or slow colour temperature shifts.

24.5.6 Subject Tracking & Virtual Pan

Given a user-specified subject region in the first frame, the engine tracks it across the sequence using flow accumulation and generates a virtual camera pan that keeps the subject centred. The crop window trajectory is smoothed via PCHIP interpolation (section 6) to produce cinematically smooth virtual camera movement.

24.5.7 Scene Boundary Detection from Flow Patterns

Abrupt changes in global flow statistics—mean magnitude, dominant direction, or spatial coherence—indicate scene boundaries (e.g., camera repositioning, lighting rig changes). The engine computes a scene change score:

$$\text{SCS}[n] = \|\bar{\mathbf{u}}[n] - \bar{\mathbf{u}}[n-1]\|_2 + \lambda_\sigma |\sigma_u[n] - \sigma_u[n-1]| \quad (60)$$

where $\bar{\mathbf{u}}[n]$ is the spatial mean flow and $\sigma_u[n]$ is the spatial standard deviation of flow magnitude. Frames where $\text{SCS}[n] > \tau_{\text{scene}}$ are flagged as candidate scene boundaries.

24.6 Stabilisation & Geometric Correction

Even tripod-mounted timelapse sequences suffer from geometric disturbances: wind gusts, accidental bumps, thermal expansion of the tripod, and ground settlement over multi-day shoots. This domain provides a hierarchy of geometric correction techniques, from simple homography alignment to depth-aware parallax compensation.

24.6.1 Homography-Based Realignment

The fundamental geometric correction aligns each frame to a reference using a 3×3 homography matrix $\mathbf{H}[n]$ estimated from matched feature correspondences:

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} \sim \mathbf{H}[n] \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} = \begin{pmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} \quad (61)$$

where $\sim$ denotes equality up to scale. The eight degrees of freedom of the homography capture translation, rotation, scale, and perspective distortion. Feature matching uses ORB or SIFT descriptors with RANSAC outlier rejection.

24.6.2 Feature-Locked Stabilisation

Rather than aligning to a single reference frame (which may drift over long sequences), feature-locked stabilisation identifies persistent features—building corners, lamp posts, horizon edges—that remain visible across hundreds or thousands of frames, and locks the alignment to these anchor points. The approach maintains a persistent feature map $\mathcal{F}$ and updates it incrementally:

$$\mathcal{F}_{n+1} = \mathcal{F}_n \cup \{f \in \text{detect}(I[n+1]) : \text{track}(f, \mathcal{F}_n) = \emptyset\} \setminus \{f \in \mathcal{F}_n : \text{age}(f) > \tau_{\text{age}}\} \quad (62)$$

24.6.3 Sub-Pixel Thermal Drift Correction

Temperature changes cause slow, sub-pixel drift as the tripod and lens expand or contract. The engine detects this by fitting a low-order polynomial to the frame-to-frame displacement time series:

$$d_x[n] = a_0 + a_1 n + a_2 n^2, \quad d_y[n] = b_0 + b_1 n + b_2 n^2 \quad (63)$$

and subtracting the fitted drift, preserving genuine camera motion (e.g., intentional pans) while removing the thermal component.

24.6.4 Displacement Event Detection

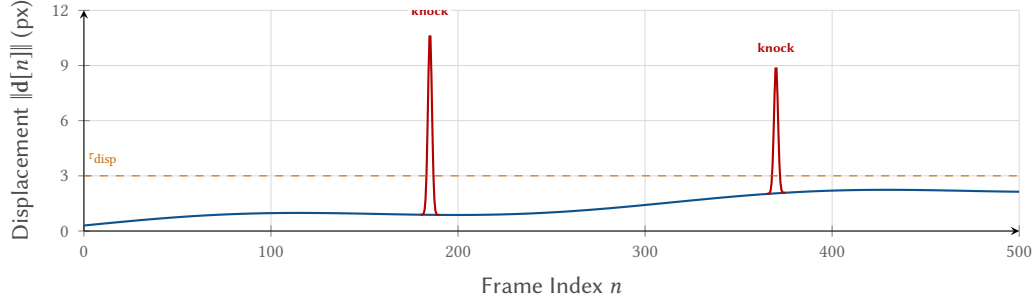


Figure 56. Displacement event detection. The global displacement $\|d[n]\|$ is tracked over time; sudden spikes exceeding τ_{disp} indicate physical disturbances (tripod knocks, wind gusts). The engine can automatically correct these by re-aligning post-event frames to the pre-event reference.

A displacement event is detected when the inter-frame displacement exceeds the threshold τ_{disp} by a significant margin. The engine distinguishes between *transient* events (single-frame spikes from vibration, corrected by interpolating the affected frame’s geometry from its neighbours) and *persistent* events (tripod repositioning, requiring a new reference frame and alignment reset).

24.6.5 Crop-Aware Realignment with Content-Aware Fill

Geometric correction introduces black borders where the warped frame does not cover the output canvas. Rather than cropping to the largest inscribed rectangle (which discards potentially valuable border content), the engine can apply content-aware fill (Criminisi, Pérez & Toyama, 2004) to synthesise plausible border pixels from the temporal neighbourhood.

24.6.6 Parallax-Aware Depth-Dependent Correction

For scenes with significant depth variation, a single homography cannot correct all planes simultaneously. The engine optionally estimates a depth map (via monocular depth estimation; section 24.8) and applies depth-dependent warps:

$$\mathbf{p}'(x, y) = \mathbf{H}_0 \cdot \mathbf{p}(x, y) + \frac{\mathbf{t}}{Z(x, y)} \quad (64)$$

where $\mathbf{H}_0$ is the background homography, $\mathbf{t}$ is the estimated camera translation, and $Z(x, y)$ is the estimated depth.

24.6.7 Rolling Correction Blending

Rather than applying a hard correction at the event frame (which can produce a visible “snap”), the engine spreads the correction over $2M + 1$ frames using a smooth blending function:

$$\mathbf{H}_{\text{corr}}[n] = (1 - \alpha_M[n]) \cdot \mathbf{H}_{\text{pre}} + \alpha_M[n] \cdot \mathbf{H}_{\text{post}} \quad (65)$$

where $\alpha_M[n]$ is a sigmoid blending weight centred on the event frame with transition width M :

$$\alpha_M[n] = \frac{1}{1 + e^{-\kappa(n - n_{\text{event}})/M}} \quad (66)$$

This produces a visually imperceptible correction that distributes the geometric adjustment over several seconds of playback.

24.7 Surveillance & Video Analytics Techniques

Although timelapse photography is primarily a creative medium, the long-duration observation it entails has natural overlap with video surveillance and analytics. This domain adapts established surveillance techniques to timelapse-specific use cases.

24.7.1 Background Subtraction via Gaussian Mixture Model

The Gaussian Mixture Model (GMM) approach (Stauffer & Grimson, 1999) models each pixel’s intensity distribution as a mixture of K Gaussians:

$$P(I[n, x, y]) = \sum_{k=1}^K w_k \cdot \mathcal{N}(I[n, x, y]; \mu_k, \sigma_k^2) \quad (67)$$

where the weights w_k , means μ_k , and variances σ_k^2 are updated online with each new frame. Pixels that do not match any component with sufficient weight are classified as foreground. In timelapse context, the GMM naturally handles gradual illumination changes (day/night cycles) while detecting transient objects.

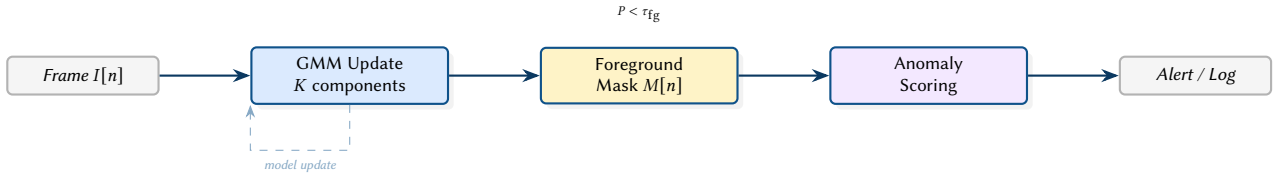


Figure 57. Anomaly detection pipeline. Each frame updates the per-pixel GMM; pixels deviating from the learned background model are classified as foreground. Persistent or large foreground regions trigger anomaly scoring.

24.7.2 Anomaly Detection

Building on the GMM foreground mask, the engine computes an anomaly score by evaluating the foreground region’s persistence, size, and location against a temporal model of “normal” activity. A frame is flagged as anomalous when its foreground statistics deviate significantly from the learned distribution:

$$A[n] = \frac{|\{(x, y) : M[n, x, y] = 1\}|}{W \cdot H} - E[A_{\text{normal}}] \quad (68)$$

where $M[n]$ is the binary foreground mask and $E[A_{\text{normal}}]$ is the expected foreground fraction from the historical model.

24.7.3 Object Counting Over Time

Integration with a pre-trained object detector (YOLO; Redmon et al. 2016) enables per-frame object counting. The engine runs detection on each frame (or on a decimated subset for efficiency), accumulates counts by class (vehicles, pedestrians, birds), and produces time-series plots of activity levels. This is particularly valuable for urban timelapse projects documenting traffic patterns or construction site activity.

24.7.4 Dwell & Occupancy Heatmaps

For scenes with recurring objects (e.g., parked vehicles, market stalls), the engine accumulates foreground mask time to produce a *dwell heatmap*: regions where foreground pixels persist longer receive higher values. The dwell map D is:

$$D(x, y) = \sum_{n=0}^{N-1} M[n, x, y] \cdot \Delta t[n] \quad (69)$$

where $\Delta t[n]$ is the inter-frame time interval. The result reveals spatial patterns of occupancy, congestion, and usage.

24.8 Deep Learning Extensions

Deep learning models offer capabilities beyond traditional signal processing, from perceptual quality assessment to semantic scene understanding. This domain describes how pre-trained models integrate into the timelapse pipeline as optional, modular analysis stages.

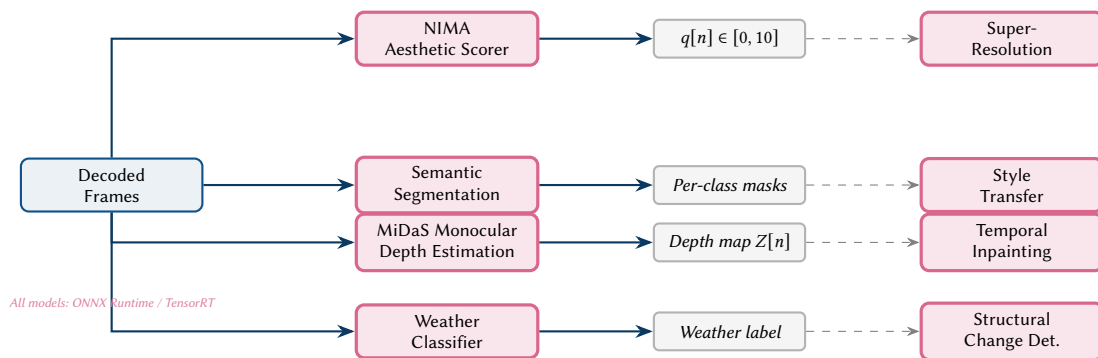


Figure 58. Deep learning integration pipeline. First-column models (purple) produce per-frame metadata that feeds both the narrative engine and second-column models for advanced processing. All models run via ONNX Runtime for cross-platform deployment.

24.8.1 Aesthetic Quality Scoring (NIMA)

The Neural Image Assessment model (NIMA; Talebi & Milanfar, 2018) predicts a distribution over human aesthetic ratings for each frame, producing a mean opinion score $q[n] \in [1, 10]$. Unlike the heuristic quality scoring in section 24.1, NIMA captures perceptual qualities—composition balance, colour harmony, lighting drama—that are difficult to express as hand-crafted features.

24.8.2 Semantic Segmentation for Per-Zone Processing

A semantic segmentation model (e.g., DeepLabV3+; Chen et al. 2018) partitions each frame into semantic regions: sky, buildings, vegetation, water, road. These masks enable zone-specific processing—different deflicker parameters for sky vs. ground, selective colour grading, or masked stabilisation that locks to buildings while allowing cloud motion.

24.8.3 Weather Classification

A lightweight CNN classifier assigns each frame a weather label (clear, partly cloudy, overcast, rain, fog, snow). The weather timeline complements the scene segmentation (section 24.1) and enables weather-conditional processing: for example, applying stronger deflicker during overcast segments where luminance variation is dominated by sensor noise rather than natural light changes.

24.8.4 Monocular Depth Estimation

MiDaS (Ranftl et al. 2020) produces per-pixel relative depth maps from single images. In the timelapse context, depth maps enable parallax-aware stabilisation (section 24.6), layered compositing (separate foreground/background processing), and depth-dependent fog simulation.

24.8.5 Style Transfer with Temporal Consistency

Neural style transfer (Gatys, Ecker & Bethge, 2016) applied naïvely to individual frames produces temporally incoherent results—the stylised appearance flickers between frames. The engine enforces temporal consistency by incorporating an optical flow-based temporal loss:

$$\mathcal{L}_{\text{temporal}} = \sum_{x,y} \|O[n](x, y) - W(O[n-1]; \mathbf{u}[n-1])(x, y)\|_2^2 \quad (70)$$

where $O[n]$ is the stylised output and $W(\cdot; \mathbf{u})$ denotes warping by optical flow $\mathbf{u}$. This term penalises stylistic changes that are not explained by actual motion.

24.8.6 Super-Resolution

For sequences shot at lower resolution (e.g., older cameras, cropped sensors), a super-resolution model (ESRGAN; Wang et al. 2018) upscales each frame. Temporal consistency is maintained by feeding the previous frame’s upscaled result as a conditioning signal, preventing the “boiling” artefact where hallucinated high-frequency details change between frames.

24.8.7 Inpainting for Obstruction Removal

Persistent obstructions—raindrops on the lens housing, spider webs, bird droppings—can be masked and inpainted using temporal information from adjacent frames (where the obstruction may be absent or differently positioned) combined with a deep inpainting model. The temporal median (eq. (47)) provides a strong prior for static scene content behind the obstruction.

24.8.8 Structural Change Detection for Construction Progress

For long-duration construction timelapses, a change detection model identifies structural changes (new floors, crane movements, façade completion) by comparing semantically segmented frames across time. The engine can automatically annotate the timeline with construction milestones:

```
{
  "construction_events": [
    {"frame": 1200, "type": "floor_added", "description": "Level 3 slab poured"},
    {"frame": 3400, "type": "crane_moved", "description": "Tower crane repositioned"},
    {"frame": 5600, "type": "facade_start", "description": "Cladding begun on north face"}
  ]
}
```

24.9 Computational Photography

Computational photography techniques exploit the temporal redundancy inherent in timelapse sequences to achieve effects that would require specialised hardware in single-shot photography.

24.9.1 Multi-Exposure Fusion from Single RAW

Each 14-bit RAW frame contains sufficient dynamic range to synthesise multiple virtual exposures by applying different tone curves to the linear data. The engine generates K virtual exposures at EV offsets $\{-2, -1, 0, +1, +2\}$ and fuses them using exposure fusion (Mertens, Kautz & Van Reeth, 2007):

$$F(x, y) = \sum_{k=1}^K \hat{w}_k(x, y) \cdot I_k(x, y), \quad \hat{w}_k = \frac{w_k}{\sum_j w_j} \quad (71)$$

where $w_k(x, y) = C_k(x, y)^\alpha \cdot S_k(x, y)^\beta \cdot E_k(x, y)^\gamma$ combines contrast C , saturation S , and well-exposedness E quality measures.

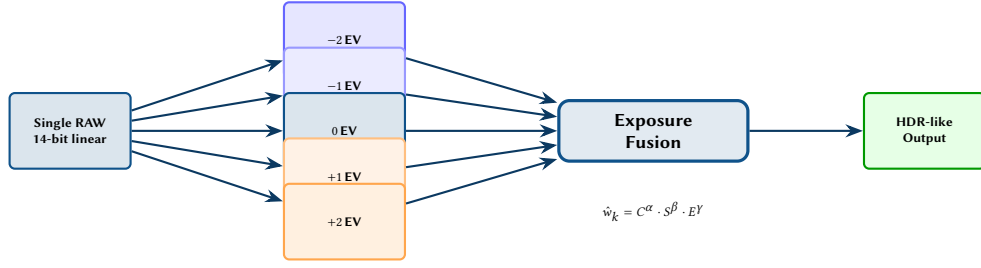


Figure 59. Multi-exposure fusion from a single RAW file. Virtual exposures are generated by applying different gains to the 14-bit linear data, then fused using contrast, saturation, and well-exposedness weights. No bracketing hardware is required.

24.9.2 Reflection Removal

Glass-mounted cameras (window timelapses, aquarium observations) frequently capture reflections that vary with interior lighting. By exploiting the temporal variation—reflections change with indoor lighting while the outdoor scene follows a different temporal pattern—the engine can separate the transmitted and reflected layers using independent component analysis (ICA) or a learned decomposition network.

24.9.3 Atmospheric Turbulence Mitigation

Long-focal-length sequences shot over heated surfaces (rooftops, deserts) suffer from atmospheric turbulence—spatially varying, time-dependent refractive distortions. The engine mitigates this by registering frames to a “lucky region” reference (selecting the sharpest local patch from a temporal window) and averaging the registered results:

$$I_{\text{restored}}(x, y) = \frac{1}{|\mathcal{W}|} \sum_{n \in \mathcal{W}} I[n](x + \delta_x^{(n)}(x, y), y + \delta_y^{(n)}(x, y)) \quad (72)$$

where $\delta^{(n)}$ is the local registration displacement for frame n and $\mathcal{W}$ is the temporal window.

24.9.4 Micro-Motion Amplification

Eulerian Video Magnification (Wu et al. 2012) amplifies subtle temporal variations invisible to the naked eye—structural vibrations, breathing of buildings in wind, slow plant growth. The technique decomposes each frame into spatial frequency bands via a Laplacian pyramid, applies temporal bandpass filtering to each band, and amplifies the filtered signal:

$$I'[n](x, y) = I[n](x, y) + \alpha \cdot \text{BPF}_{\omega_1, \omega_2} \{ L_\ell[n](x, y) \} \quad (73)$$

where L_ℓ is the ℓ -th level of the Laplacian pyramid, $\text{BPF}_{\omega_1, \omega_2}$ is a temporal bandpass filter selecting frequencies in $[\omega_1, \omega_2]$, and α is the amplification factor. For timelapse, the bandpass is tuned to the frame rate and phenomenon of interest (e.g., 0.5–2 Hz for structural resonance at 24 fps capture).

24.9.5 Shadow Path Tracking

The engine tracks shadow boundaries across the sequence by detecting high-gradient edges in the luminance channel that move predictably with solar geometry. The shadow path—the trajectory of each shadow edge over time—can be rendered as a composite overlay, visualising the sun’s arc across the sky projected onto the scene geometry.

24.10 Colour & Exposure Extensions

The current pipeline performs luminance-domain deflicker (section 3), but colour-domain corrections are equally important for professional results. This domain extends the engine with chrominance-aware temporal processing.

24.10.1 White Balance Smoothing

Camera auto white balance (AWB) introduces frame-to-frame colour temperature jitter. The engine smooths this by operating in correlated colour temperature (CCT) and tint space:

$$T_{\text{smooth}}[n] = \sum_{k=-W}^W g[k] \cdot T_{\text{raw}}[n+k], \quad \text{Tint}_{\text{smooth}}[n] = \sum_{k=-W}^W g[k] \cdot \text{Tint}_{\text{raw}}[n+k] \quad (74)$$

where $T_{\text{raw}}[n]$ is the per-frame colour temperature extracted from EXIF (or estimated from a grey-world assumption), $\text{Tint}_{\text{raw}}[n]$ is the green–magenta tint value, and $g[k]$ is a Gaussian kernel with width σ_{WB} .

The correction is applied by converting each frame to a device-independent colour space (CIE XYZ), adjusting the illuminant from $(T_{\text{raw}}, T_{\text{int}_{\text{raw}}})$ to $(T_{\text{smooth}}, T_{\text{int}_{\text{smooth}}})$ via a chromatic adaptation transform (Bradford), and converting back:

$$I_{\text{corr}}[n] = \mathbf{M}_{\text{Bradford}}^{-1} \cdot \text{diag}\left(\frac{\mathbf{w}_{\text{smooth}}[n]}{\mathbf{w}_{\text{raw}}[n]}\right) \cdot \mathbf{M}_{\text{Bradford}} \cdot I[n] \quad (75)$$

where $\mathbf{w}$ denotes the white point in cone response space.

: Chrominance Smoothing Independence

White balance smoothing operates independently of the luminance deflicker pipeline. The two corrections are applied in sequence (luminance first, then chrominance) to avoid cross-domain interference. This separation preserves the validated luminance correction while adding colour stability.

24.10.2 Shadow/Highlight Recovery with Temporal Consistency

Per-frame shadow and highlight recovery (lifting crushed blacks, pulling back clipped whites) can introduce temporal instability if applied independently. The engine enforces temporal consistency by smoothing the recovery parameters—shadow lift amount, highlight roll-off knee point—using the same Gaussian temporal filter as the luminance deflicker:

$$\text{Lift}_{\text{smooth}}[n] = \sum_k g[k] \cdot \text{Lift}_{\text{auto}}[n + k] \quad (76)$$

This ensures that shadow and highlight adjustments evolve smoothly across the sequence, even when individual frames have dramatically different exposure characteristics (e.g., during a day-to-night holy grail transition).

24.10.3 Vignette & Gradient Correction (Flat-Field)

Lens vignetting and sensor non-uniformity produce spatial intensity gradients that, while constant per-frame, interact with the temporal deflicker to create spatially varying correction artefacts. Flat-field correction divides each frame by a normalised flat-field image $F(x, y)$:

$$I_{\text{corr}}[n](x, y) = \frac{I[n](x, y)}{F(x, y) / \max(F)} \quad (77)$$

The flat field can be estimated from a dedicated calibration frame (lens cap + uniform light source) or synthesised from the temporal median of the sequence itself, exploiting the fact that scene content averages out while the fixed vignette pattern persists.

24.10.4 Bilateral Temporal Filtering

The standard temporal Gaussian filter treats all frames within the window equally regardless of their content similarity. Bilateral temporal filtering adds a range weight that reduces the contribution of frames with dissimilar content:

$$\hat{I}[n](x, y) = \frac{\sum_k g[k] \cdot r(I[n](x, y), I[n+k](x, y)) \cdot I[n+k](x, y)}{\sum_k g[k] \cdot r(I[n](x, y), I[n+k](x, y))} \quad (78)$$

where $r(a, b) = \exp(-\|a - b\|^2 / 2\sigma_r^2)$ is the range kernel. This preserves sharp temporal edges (sudden lighting changes, flash events) while still suppressing gradual flicker in stable regions.

24.11 Sonification & Audio Synthesis

Timelapse sequences contain temporal dynamics—luminance ramps, motion bursts, colour temperature shifts—that map naturally onto auditory parameters. This domain explores bidirectional coupling between visual and audio signals: synthesising soundscapes from visual features, synchronising playback speed to musical structure, and driving visual effects from audio input.

24.11.1 Visual-to-Audio Sonification

The core sonification pipeline maps three visual feature streams to three perceptual audio dimensions. For each frame n , the normalised mean luminance $\bar{Y}_{\text{norm}}[n] \in [0, 1]$ maps to pitch, the inter-frame motion magnitude $\|\Delta I[n]\|$ maps to volume, and the correlated colour temperature $T_{\text{CCT}}[n]$ maps to timbre (spectral centroid).

The pitch mapping uses a logarithmic frequency scale spanning a configurable range:

$$f[n] = f_{\min} \cdot \left(\frac{f_{\max}}{f_{\min}}\right)^{\bar{Y}_{\text{norm}}[n]} \quad (79)$$

where $f_{\min}$ and $f_{\max}$ define the musical range (e.g., 110 Hz to 880 Hz for a two-octave span from A2 to A5). Volume is mapped linearly from motion magnitude to amplitude in decibels, and timbre is controlled by interpolating between a warm (low spectral centroid) and cool (high spectral centroid) wavetable based on the normalised colour temperature.

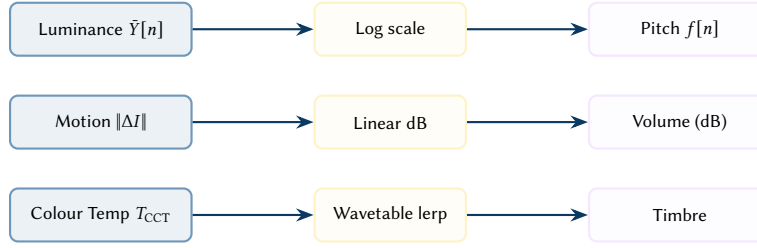


Figure 60. Visual-to-audio sonification pipeline. Three visual feature streams (luminance, motion magnitude, colour temperature) are mapped through independent transfer functions to audio parameters (pitch, volume, timbre), producing a perceptually meaningful soundtrack from the visual dynamics of the sequence.

24.11.2 BPM-Synced Speed Ramping

Professional timelapse edits frequently synchronise playback speed changes to musical beats. The engine imports an audio track, detects beat positions via onset detection (Bello et al. 2005), and aligns narrative keyframes (section 8) to beat markers. Given beat times $\{b_0, b_1, \dots, b_M\}$ and keyframe positions $\{k_0, k_1, \dots, k_M\}$, the playback speed between consecutive beats is:

$$v_j = \frac{k_{j+1} - k_j}{(b_{j+1} - b_j) \cdot \text{fps}} \quad (80)$$

where v_j is the speed multiplier for the j -th inter-beat interval and fps is the output frame rate. The engine interpolates between speed values using the same PCHIP monotonic spline used for narrative pacing, ensuring smooth acceleration and deceleration.

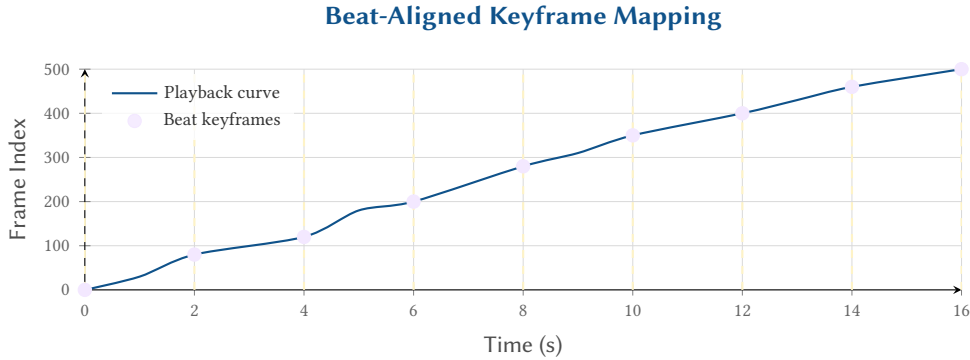


Figure 61. BPM-synced speed ramping. Vertical dashed lines mark detected beat positions in the audio track. Keyframe dots are placed at beat boundaries, and the PCHIP playback curve interpolates between them, producing speed variations that align with musical rhythm. Steeper segments correspond to faster playback.

24.11.3 Audio-Reactive Visual Effects

In the reverse direction, audio features drive visual parameters. The engine computes a short-time Fourier transform (STFT) of the audio signal and extracts per-frame features—RMS energy, spectral centroid, low/mid/high band power—which modulate visual effects such as vignette intensity, colour saturation boost, or zoom level. This enables “music video” style timelapse output where the visual treatment responds to the soundtrack in real time.

24.12 360° & Spatial Video

Omnidirectional and spatial video formats present unique challenges for timelapse processing. The non-uniform pixel density of equirectangular projections, the need for stereoscopic consistency, and the geometric distortions of alternative projections all require extensions to the standard pipeline.

24.12.1 Equirectangular Deflickering

In equirectangular projection, pixel density varies with latitude: polar regions are oversampled while equatorial regions are undersampled. The standard luminance computation (section 1) must be corrected by a latitude-dependent area weight to avoid polar bias:

$$\bar{Y}_{\text{sph}}[n] = \frac{\int_0^\pi \int_0^{2\pi} I[n](\theta, \phi) \sin \theta \, d\phi \, d\theta}{\int_0^\pi \int_0^{2\pi} \sin \theta \, d\phi \, d\theta} \quad (81)$$

In the discrete case, each pixel at row r (corresponding to latitude $\theta_r = \pi r/H$) receives a weight $w_r = \sin(\theta_r)$. The Jacobian correction factor for the equirectangular-to-spherical mapping is:

$$J(\theta) = \frac{\partial(\phi, \theta)}{\partial(u, v)} = \frac{2\pi}{W} \cdot \frac{\pi}{H} \quad (82)$$

where (u, v) are pixel coordinates and (ϕ, θ) are spherical coordinates. The area element in the image plane corresponding to a unit solid angle is thus $dA = J(\theta) \sin \theta$.

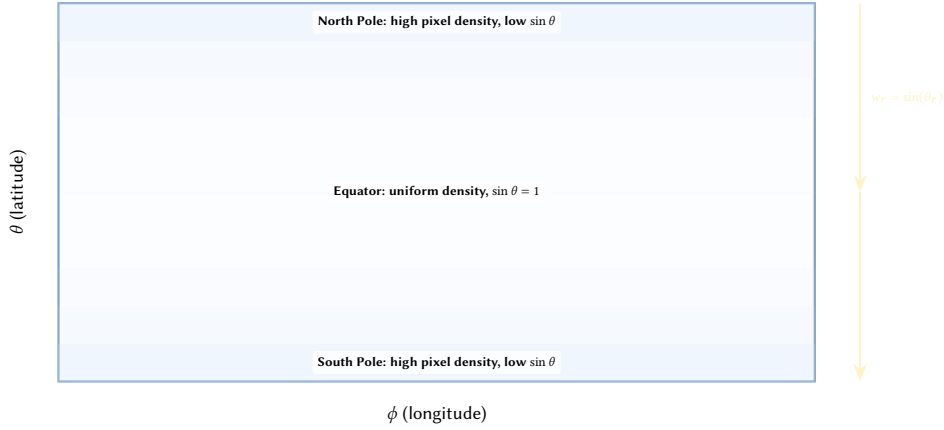


Figure 62. Equirectangular projection with latitude-dependent pixel density. Shaded bands indicate the oversampling at the poles (high opacity) versus the equator (low opacity). The area weight $w_r = \sin(\theta_r)$ corrects the luminance average to account for the non-uniform solid angle per pixel.

24.12.2 Stereoscopic 3D from Monocular Depth

Monocular depth estimation networks (Ranftl et al. 2021, MiDaS) produce per-pixel relative depth maps from single images. The engine synthesises a stereo pair by warping the original image according to the estimated disparity:

$$I_{L/R}(x, y) = I(x \pm d(x, y)/2, y), \quad d(x, y) = \frac{B \cdot f}{Z(x, y)} \quad (83)$$

where B is the virtual baseline, f is the focal length, and $Z(x, y)$ is the estimated depth. Temporal consistency is enforced by smoothing the depth maps with the same bilateral temporal filter (eq. (78)) used in the deflicker pipeline.

24.12.3 Little Planet & Stereographic Projection

The stereographic projection maps the full sphere (or a hemisphere) onto a flat plane, producing the characteristic “little planet” or “tunnel” effect. The projection from spherical to Cartesian coordinates is:

$$x' = \frac{2R \sin \theta \cos \phi}{1 + \cos \theta}, \quad y' = \frac{2R \sin \theta \sin \phi}{1 + \cos \theta} \quad (84)$$

where R is the projection sphere radius and (θ, ϕ) are the zenith and azimuth angles. The engine applies this projection per-frame with optional animated rotation, producing smoothly rotating little planet timelapse sequences.

24.13 Generative AI Extensions

Generative models—diffusion models (Ho, Jain & Abbeel, 2020), video prediction networks (Oprea et al. 2020), and conditional image synthesis architectures—offer capabilities that extend beyond analysis and correction into content creation. This domain describes how generative AI can augment timelapse sequences while maintaining temporal coherence and provenance transparency.

24.13.1 Sequence Extrapolation

Given N observed frames, the engine predicts continuation frames $\hat{I}[N], \hat{I}[N+1], \dots, \hat{I}[N+K]$ by conditioning a video diffusion model on the tail of the observed sequence. The prediction is constrained to match the luminance trend $\hat{Y}[n]$ from the existing deflicker model:

$$\hat{I}[N+k] = G_{\theta}(I[N-L:N], \hat{Y}[N+k], z_k), \quad k = 1, \dots, K \quad (85)$$

where G_{θ} is the generative model conditioned on the last L frames and the extrapolated luminance target, and z_k is a latent noise sample. The luminance target ensures photometric consistency with the observed sequence.

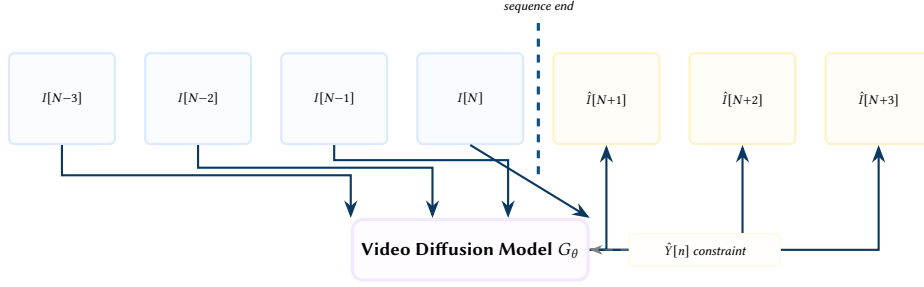


Figure 63. Sequence extrapolation via conditioned video diffusion. The observed tail frames are fed into a generative model along with the extrapolated luminance target $\hat{Y}[n]$ to predict plausible continuation frames beyond the captured sequence.

24.13.2 Weather Replacement with Temporal Consistency

Weather replacement substitutes the atmospheric conditions of a sequence (e.g., overcast to clear sky) while preserving temporal coherence. The engine segments the sky region per-frame using a semantic segmentation model, replaces the sky texture with a synthetic or reference sky sequence, and blends the boundary using Poisson image editing (Pérez, Gangnet & Blake, 2003). Temporal consistency is maintained by applying the replacement to the smoothed luminance envelope rather than individual frame luminance values.

24.13.3 Seasonal Transfer

Seasonal transfer re-renders a sequence as if captured in a different season—transforming summer foliage to autumn colours, adding snow to surfaces, or replacing bare trees with blossoms. This is formulated as a conditional image-to-image translation problem. The engine applies a pre-trained seasonal transfer model (e.g., CycleGAN; Zhu et al. 2017) per-frame with a temporal consistency loss that penalises inter-frame jitter in the translated output.

24.13.4 Ultra-Slow-Motion Frame Hallucination

Between consecutive captured frames separated by large intervals (e.g., 10 s), the engine generates intermediate frames using a video interpolation model (Niklaus & Liu, 2020) combined with a diffusion-based hallucination module for plausible content in regions of large displacement. The interpolated sequence provides smooth slow-motion playback of events that occur between capture intervals.

: Synthetic Content Provenance

All frames generated by generative AI extensions **must** be tagged with a `synthetic=true` flag in their metadata. The XMP sidecar includes a `timelapse:GeneratedBy` field identifying the model and parameters used. This ensures downstream consumers can distinguish captured from generated content, maintaining provenance transparency.

24.14 Scientific & Environmental Measurement

Timelapse cameras deployed in fixed positions over extended periods function as low-cost environmental sensors. By calibrating the imaging pipeline and extracting quantitative measurements from the visual data, the engine transforms creative timelapse output into a scientific instrument.

24.14.1 Photometric Calibration via Reference Targets

Absolute photometric calibration requires a known reference in the scene. The engine detects standard calibration targets—X-Rite ColorChecker cards, 18% grey cards—using template matching or a trained detector, and computes the per-channel transfer function mapping pixel values to scene radiance:

$$L_{\text{scene}}(\lambda) = \frac{I_{\text{raw}}(\lambda)}{R_{\text{target}}(\lambda)} \cdot L_{\text{ref}}(\lambda) \quad (86)$$

where $R_{\text{target}}(\lambda)$ is the known spectral reflectance of the calibration patch, $I_{\text{raw}}(\lambda)$ is the raw pixel value, and $L_{\text{ref}}(\lambda)$ is the reference illuminance.

24.14.2 Cloud Cover Fraction

The engine estimates cloud cover fraction per-frame by segmenting the sky region and classifying pixels as clear or cloudy based on a blue-channel ratio threshold (Long, Sabburg & Calbó, 2006):

$$\text{CF}[n] = \frac{|\{(x, y) \in \mathcal{S}_{\text{sky}} : R_{\text{BR}}(x, y) > \tau_{\text{cloud}}\}|}{|\mathcal{S}_{\text{sky}}|} \quad (87)$$

where $R_{\text{BR}} = B/R$ is the blue-to-red ratio and τ_{cloud} is a calibrated threshold. The resulting time-series of cloud cover fractions enables correlation with other environmental variables.

24.14.3 Shadow Length & Solar Elevation

For scenes containing objects of known height h , the engine tracks shadow tip positions across the sequence and computes solar elevation angle α_{sun} from the shadow length ℓ_s :

$$\alpha_{\text{sun}}[n] = \arctan\left(\frac{h}{\ell_s[n]}\right) \quad (88)$$

This provides a purely visual estimate of solar position that can be cross-validated against astronomical ephemeris data for the capture location and time.

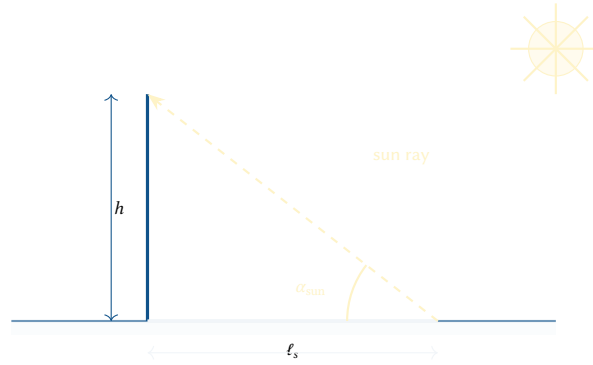


Figure 64. Shadow-to-solar-elevation geometry. A vertical object of known height h casts a shadow of length ℓ_s on the ground. The solar elevation angle α_{sun} is recovered as $\arctan(h/\ell_s)$.

24.14.4 Vegetation Index from Visible Light

In the absence of near-infrared bands, the engine computes a visible-light vegetation index from the green and red channels (Excess Green Index; Woebbecke et al. 1995):

$$\text{ExG}[n](x, y) = 2g(x, y) - r(x, y) - b(x, y) \quad (89)$$

where r, g, b are the normalised chromatic coordinates. Tracking ExG over time reveals phenological cycles—green-up in spring, senescence in autumn—providing quantitative vegetation monitoring from standard RGB cameras.

24.14.5 Temperature Correlation with Visual Changes

When external temperature logs are available (e.g., from a co-located weather station or embedded sensor), the engine aligns temperature readings with the visual timeline and computes cross-correlation between temperature and visual features (mean luminance, colour temperature, vegetation index). This multi-modal analysis enables causal investigation of temperature-driven visual phenomena.

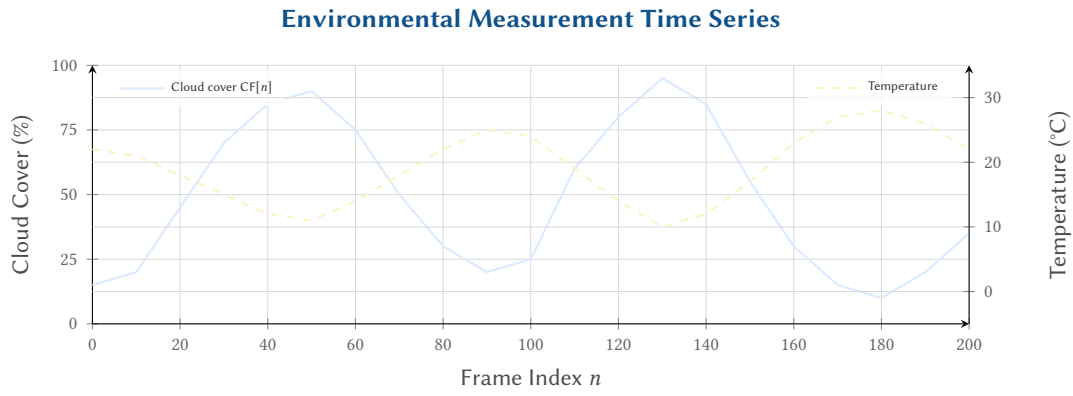


Figure 65. Dual-axis time series showing cloud cover fraction (left axis, solid) and co-located temperature readings (right axis, dashed). The inverse correlation—increasing cloud cover coinciding with falling temperature—is visually apparent and quantifiable via cross-correlation analysis.

24.15 Social Media & Delivery Optimisation

Professional timelapse output increasingly targets social media platforms with diverse aspect ratio, duration, and encoding requirements. This domain automates the adaptation of timelapse output for platform-specific delivery.

24.15.1 Auto Vertical Crop with Subject Tracking

For portrait-orientation platforms (Instagram Reels, TikTok), the engine extracts a 9:16 crop from the landscape source. Rather than a fixed centre crop, the engine tracks the primary subject (detected via saliency or object detection) and positions the crop window to keep the subject centred:

$$x_{\text{crop}}[n] = \text{clamp}(x_{\text{subject}}[n] - W_{\text{crop}}/2, 0, W - W_{\text{crop}}) \quad (90)$$

where $x_{\text{subject}}[n]$ is the horizontal centroid of the detected subject and $W_{\text{crop}} = H \cdot 9/16$ is the crop width for a 9:16 output.

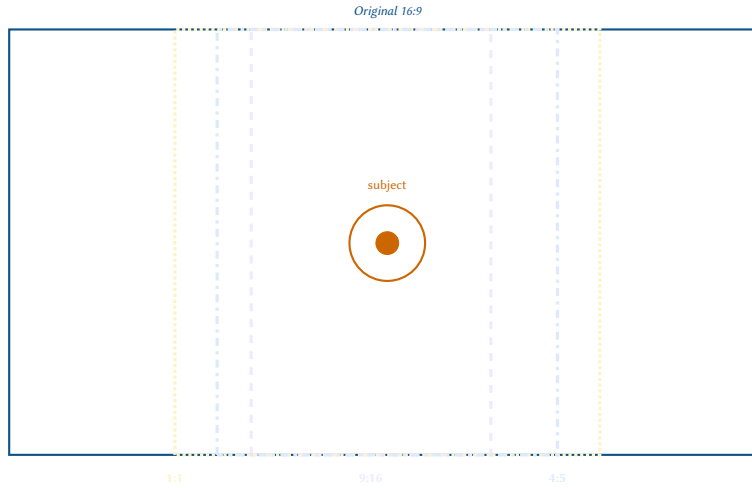


Figure 66. Aspect ratio crop regions overlaid on a landscape frame. The subject tracking centroid (orange) guides the horizontal positioning of each crop window. The 9:16 portrait crop (dashed, purple), 1:1 square crop (dotted, amber), and 4:5 crop (dash-dotted, blue) are all centred on the tracked subject.

24.15.2 Animated Thumbnail GIF

The engine generates an animated thumbnail by sampling K evenly spaced frames from the temporal summary strip (fig. 52) and encoding them as a looping GIF with configurable frame delay. The thumbnail serves as a preview for social media posts, portfolio pages, and video hosting platforms.

24.15.3 Before/After Comparison Widget

For sequences with dramatic transformations (construction, seasons, day-to-night), the engine generates an interactive HTML widget with a draggable slider separating the “before” and “after” frames. The widget is self-contained (no external dependencies) and suitable for embedding in blog posts or social media previews.

24.15.4 Platform-Optimised Encoding Presets

The engine ships with a library of encoding presets tuned to each platform’s recommended specifications:

```

{
  "platform_presets": {
    "instagram_reels": {
      "codec": "h264",
      "resolution": [1080, 1920],
      "aspect_ratio": "9:16",
      "max_duration_s": 90,
      "bitrate_mbps": 8,
      "audio_codec": "aac",
      "audio_bitrate_kbps": 128
    },
    "youtube_4k": {
      "codec": "h265",
      "resolution": [3840, 2160],
      "aspect_ratio": "16:9",
      "max_duration_s": null,
      "bitrate_mbps": 45,
      "audio_codec": "aac",
      "audio_bitrate_kbps": 320
    },
    "tiktok": {
      "codec": "h264",
      "resolution": [1080, 1920],
      "aspect_ratio": "9:16",
      "max_duration_s": 180,
      "bitrate_mbps": 6,
      "audio_codec": "aac",
      "audio_bitrate_kbps": 128
    }
  }
}

```

24.16 Multi-Camera Fusion

Multi-camera timelapse deployments—multiple fixed cameras covering a scene from different angles—enable virtual camera movements, cross-camera consistency, and richer spatial coverage. This domain addresses the geometric and photometric alignment required to fuse multiple viewpoints.

24.16.1 Timestamp-Based Multi-Angle Synchronisation

The engine aligns frames from multiple cameras using EXIF timestamp matching with sub-second interpolation. For cameras i and j with capture times $\{t_n^{(i)}\}$ and $\{t_m^{(j)}\}$, the synchronisation finds the optimal time offset Δt_{ij} by maximising the cross-correlation of their luminance time-series:

$$\Delta t_{ij}^* = \arg \max_{\Delta t} \sum_n \bar{Y}^{(i)}[n] \cdot \bar{Y}^{(j)}[n + \Delta t / \delta t] \quad (91)$$

where δt is the capture interval. This exploits the shared illumination environment: both cameras observe the same ambient light changes, enabling sub-frame temporal alignment.

24.16.2 Virtual Camera Path via Homography Interpolation

Given M fixed camera positions with known or estimated homographies $\{\mathbf{H}_1, \dots, \mathbf{H}_M\}$ relative to a reference plane, the engine synthesises intermediate virtual viewpoints by interpolating between adjacent homographies. The interpolation is performed in the Lie algebra of $\text{SL}(3)$ (Hartley & Zisserman, 2003):

$$\mathbf{H}(\lambda) = \mathbf{H}_i \cdot \exp(\lambda \cdot \log(\mathbf{H}_i^{-1} \mathbf{H}_{i+1})), \quad \lambda \in [0, 1] \quad (92)$$

where $\exp$ and $\log$ are the matrix exponential and logarithm. This produces geometrically smooth virtual camera paths that glide between physical camera positions.

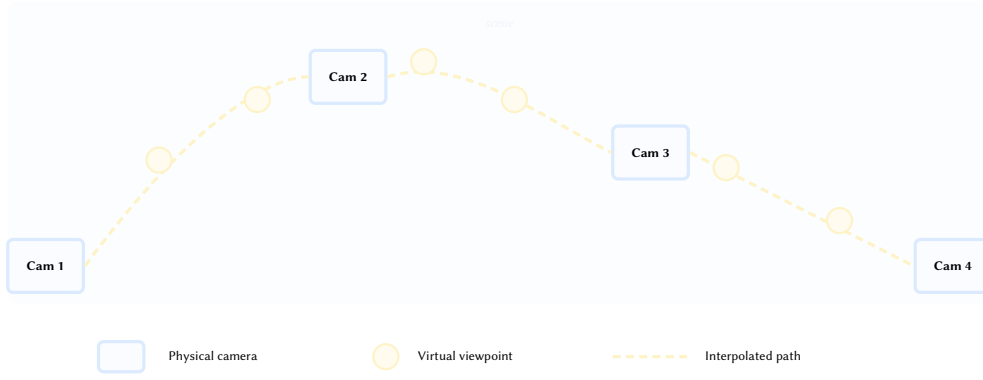


Figure 67. Virtual camera path through fixed viewpoints. Physical cameras (blue rectangles) provide anchor views; intermediate virtual viewpoints (amber circles) are synthesised via homography interpolation in the Lie algebra of $SL(3)$, producing a smooth virtual dolly path through the multi-camera array.

24.16.3 Cross-Camera Exposure & White Balance Matching

Different cameras in a multi-camera array may have different exposure settings, lens characteristics, and colour responses. The engine computes a per-camera affine colour transform to match all cameras to a reference:

$$I_{\text{matched}}^{(i)}[n] = \mathbf{A}_i \cdot I^{(i)}[n] + \mathbf{b}_i \quad (93)$$

where $\mathbf{A}_i \in \mathbb{R}^{3 \times 3}$ and $\mathbf{b}_i \in \mathbb{R}^3$ are estimated by minimising the colour difference in overlapping regions between camera i and the reference camera. The transform is updated per-frame to track changing illumination.

24.17 Incremental & Live Processing

The current pipeline operates in batch mode: all frames must be captured before processing begins. This domain extends the engine to support incremental and live processing, enabling real-time monitoring during active shoots and efficient updates when new frames arrive.

24.17.1 Watch-Folder Live Processing

The engine monitors a designated folder for new image files using filesystem event notifications (inotify on Linux, FSEvents on macOS). When a new frame is detected, the engine:

1. Validates the file (complete write, correct format).
2. Extracts metadata and computes luminance.
3. Updates the running deflicker model with an online Gaussian filter.
4. Extends the temporal summary strip.
5. Pushes updated statistics to the web dashboard.

The online update avoids reprocessing the entire sequence. For a new frame $I[N]$, the smoothed luminance is computed using only the most recent $2W + 1$ frames:

$$\hat{Y}_{\text{online}}[N] = \frac{\sum_{k=-W}^0 g[k] \cdot Y[N+k]}{\sum_{k=-W}^0 g[k]} \quad (94)$$

Note the asymmetric window: only past frames are available in the live setting, introducing a causal constraint that slightly reduces smoothing quality near the sequence head.

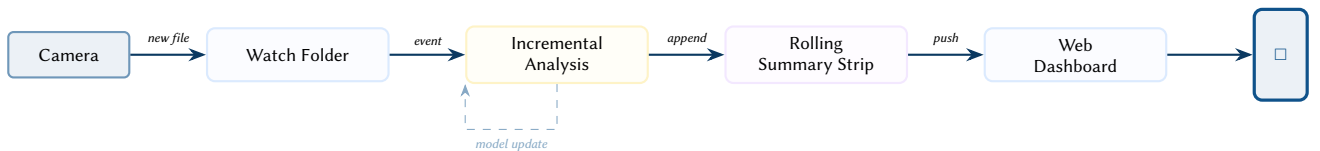


Figure 68. Live processing architecture. The camera writes to a watch folder; filesystem events trigger incremental analysis (luminance extraction, deflicker update); results are appended to the rolling summary strip and pushed to a web dashboard accessible from any device.

24.17.2 Incremental Video Extension

When the sequence grows, re-encoding the entire output video is wasteful. The engine supports incremental video extension by:

1. Encoding new frames as a standalone video segment with matching codec parameters.

2. Concatenating the new segment to the existing video using container-level concatenation (e.g., MP4 fragment appending) without re-encoding previous frames.

The fragment-based approach exploits the ISO BMFF (ISO 14496-12) container format, where each fragment is independently decodable. The engine maintains a running state of the encoder (quantisation parameters, reference frames) to ensure seamless splicing:

$$V_{\text{extended}} = V_{\text{existing}} \oplus \text{Encode}(I[N+1], \dots, I[N+K]; s_N) \quad (95)$$

where $\oplus$ denotes container-level concatenation and s_N is the encoder state at frame N .

24.17.3 Web Dashboard for Remote Monitoring

The web dashboard provides a real-time view of the active shoot, displaying:

- The latest captured frame with overlay annotations (luminance, exposure settings).
- The rolling temporal summary strip, updated with each new frame.
- Time-series plots of luminance, colour temperature, and quality metrics.
- Alert notifications for anomalies (sudden exposure changes, focus loss, camera displacement).

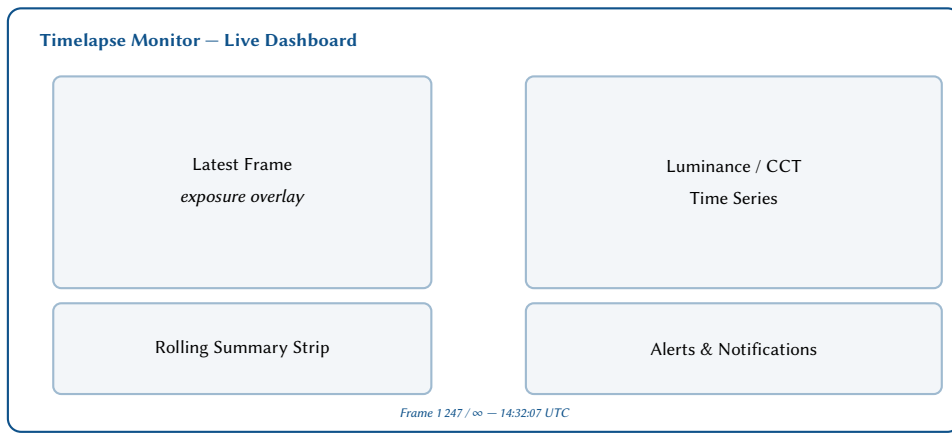


Figure 69. Web dashboard layout for remote shoot monitoring. The interface displays the latest captured frame with exposure overlay (top left), a rolling temporal summary strip (bottom left), luminance and colour temperature time-series plots (top right), and alert notifications (bottom right). The dashboard updates incrementally as new frames arrive.

The dashboard is implemented as a lightweight HTTP server (e.g., using axum or warp) that serves static assets and streams updates via Server-Sent Events (SSE). This architecture requires no client-side installation—any modern web browser suffices—making it suitable for remote monitoring of cameras deployed in the field.

: Causal Filter Latency

In live mode, the asymmetric (causal) temporal filter introduces a systematic bias: the smoothed luminance at the sequence head is computed from fewer samples and thus has higher variance. The dashboard displays a “stabilisation zone” indicator showing the first W frames where deflicker quality is reduced. Once the sequence exceeds $2W + 1$ frames, the engine can optionally re-process the head region with the full symmetric filter during idle periods.

24.18 Timestamp & Clock Correction

Camera clocks drift over time and frequently carry an unknown initial offset relative to true UTC. For timelapse sequences spanning hours or days, uncorrected timestamps propagate errors into every time-dependent analysis stage—sunrise/sunset alignment, astronomical correlation, multi-camera synchronisation. This domain introduces a clock correction pipeline that detects and compensates for both the initial offset and the linear drift rate.

24.18.1 GPS/NTP Cross-Reference Offset Detection

When a subset of frames carries GPS-stamped timestamps (e.g. from an external GPS logger or a camera with integrated GNSS), the engine computes the offset between the EXIF timestamp $t_{\text{exif}}[n]$ and the GPS timestamp

$t_{\text{gps}}[n]$ for each co-located frame. Given M GPS-tagged frames, the clock model parameters are estimated via ordinary least squares on the linear drift model:

$$t_{\text{corrected}}[n] = t_{\text{exif}}[n] + (\alpha + \beta \cdot n) \quad (96)$$

where α is the initial offset (seconds) and β is the drift rate (seconds per frame). The parameters are fitted by minimising:

$$(\hat{\alpha}, \hat{\beta}) = \arg \min_{\alpha, \beta} \sum_{m=1}^M (t_{\text{gps}}[m] - t_{\text{exif}}[m] - \alpha - \beta \cdot m)^2 \quad (97)$$

For cameras without GPS, an NTP-synchronised reference device (smartphone, laptop) can provide ground-truth timestamps for a calibration capture at the start and end of the shoot.

24.18.2 Polynomial Drift Extension

For long deployments where crystal oscillator drift exhibits nonlinear behaviour (temperature dependence), the engine supports a second-order polynomial model:

$$t_{\text{corrected}}[n] = t_{\text{exif}}[n] + \alpha + \beta \cdot n + \gamma \cdot n^2 \quad (98)$$

where γ captures the quadratic drift component. Model selection (linear vs. polynomial) is performed via the Bayesian Information Criterion (BIC) to avoid overfitting.

24.18.3 Sunrise/Sunset Validation

In the absence of GPS reference timestamps, the engine exploits the predictable timing of civil twilight. Given the capture location (from EXIF GPS coordinates or user-supplied latitude/longitude), the engine computes the expected sunrise and sunset times using standard solar position algorithms (Meeus, 1991). It then detects the luminance inflection point—the frame at which the temporal derivative of mean luminance reaches its maximum during the dawn transition—and compares the EXIF timestamp of that frame against the calculated civil twilight time. The discrepancy provides an estimate of the clock offset:

$$\hat{\alpha}_{\text{sun}} = t_{\text{twilight}}^{\text{calc}} - t_{\text{exif}}[n_{\text{inflection}}] \quad (99)$$

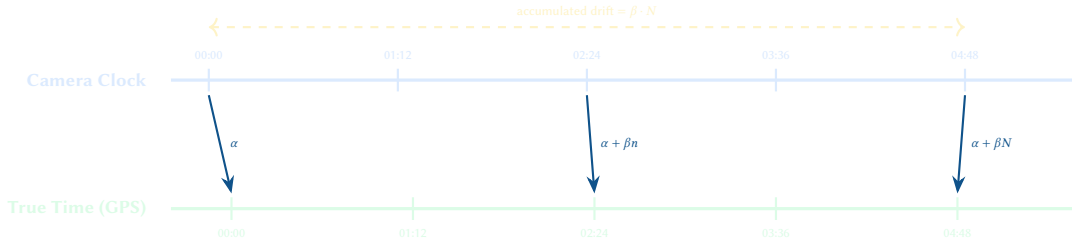


Figure 70. Clock correction model. The camera timeline (top, blue) drifts relative to the true GPS/NTP timeline (bottom, green). The offset at frame n is $\alpha + \beta n$, where α is the initial offset and β is the per-frame drift rate. Arrows show the correction applied to align each camera timestamp to true time.

Sunrise Validation: Luminance Inflection vs Civil Twilight

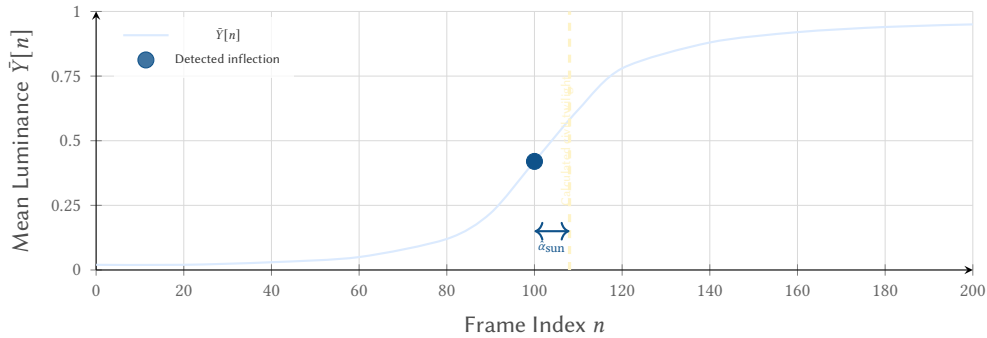


Figure 71. Sunrise validation for clock offset estimation. The luminance curve shows the dawn transition; the detected inflection point (maximum temporal derivative) is compared against the calculated civil twilight time (dashed amber line). The discrepancy $\hat{\alpha}_{\text{sun}}$ provides an offset estimate without GPS hardware.

24.18.4 Multi-Camera Clock Synchronisation

When multiple cameras capture overlapping views of the same scene, the engine synchronises their clocks by detecting shared visual events—flash triggers, sudden lighting changes, moving subjects entering the overlapping field of view. The temporal cross-correlation of luminance signals between camera pairs identifies the inter-camera offset:

$$\hat{\delta}_{ij} = \arg \max_{\delta} \sum_n \bar{Y}_i[n] \cdot \bar{Y}_j[n + \delta] \quad (100)$$

where $\hat{\delta}_{ij}$ is the estimated frame offset between cameras i and j .

24.19 Film & Colour Science

Professional colour grading workflows rely on Look-Up Tables (LUTs), film emulation profiles, and precise colour space management. Integrating these into the timelapse pipeline requires careful attention to the interaction between temporal deflicker processing and creative colour transforms. This domain extends the engine with a full colour science pipeline from camera RAW through to delivery-ready output.

24.19.1 3D LUT Application with Temporal Flicker Isolation

A 3D LUT maps input (R, G, B) triples to output (R', G', B') values using trilinear interpolation within a lattice of control points. For an $N \times N \times N$ cube, the output at an arbitrary input point is:

$$c_{\text{out}} = \sum_{i=0}^1 \sum_{j=0}^1 \sum_{k=0}^1 w_i w_j w_k \cdot L[r_0+i, g_0+j, b_0+k] \quad (101)$$

where (r_0, g_0, b_0) are the lattice indices of the enclosing cube, $w_i = 1 - |r_f - i|$ (and similarly for w_j, w_k) are the trilinear weights from the fractional coordinates (r_f, g_f, b_f) , and L is the LUT lattice.

The engine parses industry-standard .cube files (Resolve, Photoshop) and applies the LUT as a pipeline stage after the temporal deflicker correction.

: LUT After Deflicker

3D LUTs must be applied *after* the luminance deflicker stage. If a LUT is applied before deflickering, the nonlinear colour transform amplifies frame-to-frame luminance variations, producing grade-amplified flicker that the subsequent deflicker stage cannot fully suppress. The correct ordering is: demosaic → deflicker → LUT → encode.

24.19.2 Film Stock Emulation

Film stock emulation reproduces the characteristic response curves, grain structure, and colour rendering of analogue film. The engine models three components: (1) a per-channel tone response curve (Hurter–Driffeld characteristic curve), (2) a wavelength-dependent colour crossover matrix derived from published spectral sensitivity data, and (3) a grain synthesis layer using Perlin noise scaled to the granularity of the target stock (ISO 100 fine grain to ISO 3200 coarse grain).

24.19.3 Colour Space Pipeline

The engine manages colour through a well-defined pipeline of colour space conversions:

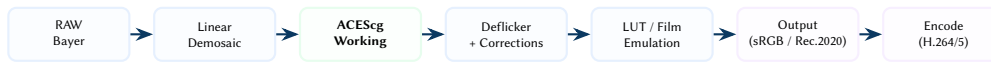


Figure 72. Colour science pipeline. RAW Bayer data is demosaiced to linear RGB, converted to the ACEScg working colour space for all corrections and deflickering, passed through optional LUT or film emulation, transformed to the output colour space (sRGB for web, Rec. 2020 for HDR), and finally encoded. Operating in a scene-referred linear space (ACEScg) preserves maximum dynamic range through the correction stages.

24.19.4 False-Colour Exposure Zebras

The engine generates a false-colour overlay that maps luminance zones to diagnostic colours, analogous to the “zebras” in broadcast cameras. Pixels below 2% luminance are flagged as crushed blacks, pixels above 98% as clipped highlights, and intermediate zones are mapped to a thermal-style colour ramp. This overlay is available as a per-frame diagnostic export or as a real-time preview in the web dashboard (fig. 69).

24.20 Drone & Aerial Processing

Drone-captured timelapse sequences carry rich telemetry—GPS coordinates, altitude, gimbal angles, battery voltage—embedded in each frame’s metadata. This domain exploits that telemetry for spatial processing, altitude-compensated exposure, and cartographic output.

24.20.1 GPS-Tagged Frame Overlay and KML Export

The engine extracts GPS coordinates from each frame's EXIF or XMP metadata and generates:

1. A georeferenced frame overlay: each frame's thumbnail is placed at its capture location on a map tile.
2. A KML/KMZ file for Google Earth, encoding the flight path as a LineString with frame thumbnails as PhotoOverlay elements.

24.20.2 Ortho-Mosaic Generation

For nadir (straight-down) drone captures with sufficient overlap ($\geq 60\%$), the engine generates a 2D ortho-mosaic by stitching adjacent frames using homography estimation and blending in the overlap regions. The mosaic is georeferenced using the per-frame GPS coordinates and exported as a GeoTIFF.

24.20.3 Altitude-Compensated Exposure

Ground illumination from a drone-mounted downward-facing light source (or reflected sunlight) varies with altitude. For a Lambertian ground surface, the irradiance at the sensor follows an inverse-square relationship with altitude:

$$E_{\text{ground}}[n] \propto \frac{1}{h[n]^2} \quad (102)$$

where $h[n]$ is the altitude above ground level at frame n . When the drone changes altitude during a timelapse, the engine compensates by applying a multiplicative correction:

$$I_{\text{corr}}[n] = I[n] \cdot \left(\frac{h[n]}{h_{\text{ref}}} \right)^2 \quad (103)$$

where h_{ref} is a reference altitude (typically the median altitude of the sequence).

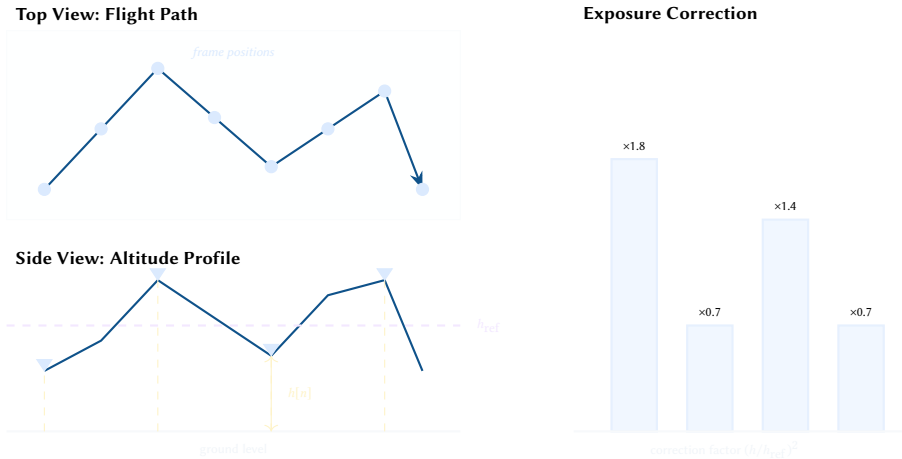


Figure 73. Drone altitude and exposure compensation. Top view (upper left) shows the flight path with frame capture positions. Side view (lower left) shows the altitude profile with a reference altitude h_{ref} (dashed purple). Right panel shows the multiplicative exposure correction factors applied at varying altitudes to normalise ground illumination.

24.20.4 Telemetry HUD Overlay

The engine renders a configurable heads-up display (HUD) overlay on each output frame, showing GPS coordinates, altitude, speed, heading, gimbal pitch, and battery level extracted from the telemetry stream. The HUD uses semi-transparent backgrounds to avoid obscuring the scene content, and its layout is configurable via the YAML configuration (??).

24.21 Extreme Environment Correction

Timelapse cameras deployed in underwater, foggy, or precipitation-heavy environments capture images with severe wavelength-dependent degradation. This domain introduces physics-based correction models that restore image fidelity while maintaining temporal consistency across the sequence.

24.21.1 Underwater Colour Correction via Beer-Lambert Law

Water selectively absorbs light as a function of wavelength and depth. Red light is attenuated most rapidly, followed by green, with blue penetrating deepest. The Beer-Lambert law models this exponential attenuation:

$$I_{\text{observed}}(\lambda) = I_{\text{surface}}(\lambda) \cdot \exp(-\beta(\lambda) \cdot d) \quad (104)$$

where $\beta(\lambda)$ is the wavelength-dependent attenuation coefficient (m^{-1}) and d is the depth (m). Typical values for clear ocean water are $\beta_R \approx 0.44 \text{ m}^{-1}$, $\beta_G \approx 0.07 \text{ m}^{-1}$, $\beta_B \approx 0.04 \text{ m}^{-1}$ (Jerlov, 1976).

Colour restoration inverts the attenuation:

$$I_{\text{restored}}(\lambda) = I_{\text{observed}}(\lambda) \cdot \exp(\beta(\lambda) \cdot d) \quad (105)$$

When depth varies across the sequence (e.g. a descending dive timelapse), the engine reads depth from dive computer telemetry or estimates it from pressure sensor metadata, applying per-frame wavelength-dependent gain correction.

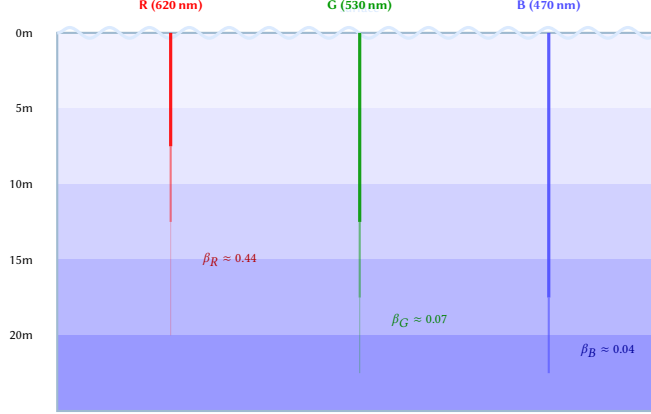


Figure 74. Wavelength-dependent light attenuation in water (Beer–Lambert law). Red light is absorbed most rapidly ($\beta_R \approx 0.44 \text{ m}^{-1}$), followed by green ($\beta_G \approx 0.07 \text{ m}^{-1}$), with blue penetrating deepest ($\beta_B \approx 0.04 \text{ m}^{-1}$). Line thickness and opacity indicate remaining intensity at each depth. Underwater colour restoration inverts this attenuation per-channel.

24.21.2 Dark Channel Prior Dehazing with Temporal Consistency

Fog, haze, and mist reduce contrast and shift colours toward the airlight. The dark channel prior (He, Sun & Tang, 2009) estimates the transmission map $t(x, y)$ from the observation that in haze-free outdoor images, at least one colour channel has very low intensity in most local patches:

$$J^{\text{dark}}(x, y) = \min_{c \in \{R, G, B\}} \left(\min_{(x', y') \in \Omega(x, y)} \frac{I^c(x', y')}{A^c} \right) \quad (106)$$

where $\Omega(x, y)$ is a local patch and A^c is the atmospheric light in channel c . The transmission is estimated as $\hat{t}(x, y) = 1 - \omega \cdot J^{\text{dark}}(x, y)$ with $\omega \approx 0.95$.

For timelapse sequences, independent per-frame dehazing produces temporal flicker in the recovered transmission map. The engine enforces temporal consistency by applying the same Gaussian temporal filter used in luminance deflicker to the transmission map sequence before scene recovery.

24.21.3 Rain and Snow Streak Detection

Rain streaks and snow particles appear as bright, elongated features in individual frames. The engine detects them using a directional derivative filter aligned with the expected streak orientation (approximately vertical for rain, more random for snow). Detected streaks are inpainted using temporal information from adjacent frames where the corresponding pixels are unoccluded.

24.22 Artistic & Glitch Effects

Beyond corrective processing, the engine supports intentionally destructive and experimental effects that exploit the temporal structure of timelapse sequences for creative output. These effects operate at the frame, pixel, and channel levels.

24.22.1 Datamosh

Datamoshing produces characteristic “melting” visual artefacts by manipulating the video codec’s prediction structure. The engine achieves this by selectively dropping I-frames (intra-coded keyframes) from the encoded bitstream, forcing the decoder to apply P-frame (predicted) motion vectors to stale reference frames. The result is that motion from new frames is applied to the texture of old frames, creating surreal morph effects.

The engine provides two datamosh modes:

- **I-frame deletion:** Removes keyframes at user-specified positions, causing P-frame propagation across scene boundaries.
- **P-frame duplication:** Repeats a selected P-frame’s motion vectors across a configurable window, amplifying and abstracting the motion field.

24.22.2 Pixel Sorting with Temporal Evolution

Pixel sorting reorders pixels within each row or column according to a brightness threshold, producing glitch-art bands of sorted colour. The engine extends static pixel sorting with temporal evolution: the sorting threshold $\tau[n]$ varies across the sequence, and the sort direction rotates over time, creating an animated sweep effect.

24.22.3 Echo/Trail with Opacity Decay

The echo effect composites the current frame with geometrically decaying previous frames:

$$O[n] = \sum_{k=0}^K \alpha^k \cdot I[n-k] \quad (107)$$

where $\alpha \in (0, 1)$ is the decay factor and K is the trail length. The geometric series ensures that the contribution of older frames falls off exponentially, creating a ghostly trail behind moving objects. The normalisation factor $\sum_{k=0}^K \alpha^k = (1 - \alpha^{K+1})/(1 - \alpha)$ is applied to maintain consistent brightness.

24.22.4 Chromatic Temporal Aberration

Chromatic temporal aberration offsets the R, G, and B channels by different frame delays, causing colour separation on moving objects while static elements remain unaffected:

$$O[n] = (R[n - \delta_R], G[n - \delta_G], B[n - \delta_B]) \quad (108)$$

where $\delta_R, \delta_G, \delta_B$ are the per-channel frame offsets (typically $\delta_G = 0, \delta_R = -\Delta, \delta_B = +\Delta$ for symmetric separation).

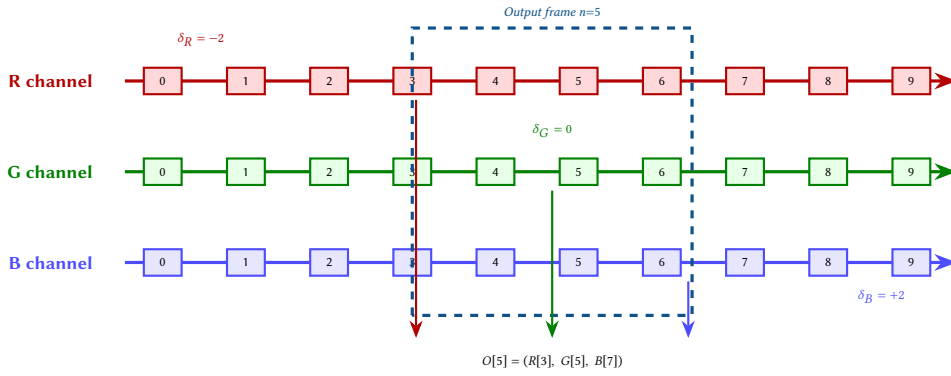


Figure 75. Chromatic temporal aberration. Three timelines show the R, G, and B channels offset by different frame delays ($\delta_R = -2, \delta_G = 0, \delta_B = +2$). The dashed box highlights the frames composited into output frame $n = 5$: red from frame 3, green from frame 5, blue from frame 7. Moving objects exhibit colour fringing while static elements remain aligned.

24.23 Extended Astronomical Analysis

Night-sky timelapse sequences contain a wealth of astronomical information beyond star trails. This domain extends the engine with satellite identification, meteor detection, aurora analysis, and planetary tracking, leveraging the clock-corrected timestamps from section 24.18 for accurate correlation with external ephemeris and space weather data.

24.23.1 Satellite and ISS Detection from TLE Data

Two-Line Element (TLE) sets published by NORAD provide orbital parameters for thousands of satellites. The engine propagates TLE data using the SGP4/SDP4 model to predict satellite positions in the camera's field of view at each clock-corrected frame time. Detected streaks are matched against predicted tracks by minimising the angular residual, enabling automatic identification of satellites including the ISS.

24.23.2 Meteor Detection and Composite

Meteors appear as bright, short-lived streaks spanning 1–3 frames. The engine detects them by:

1. Computing the temporal difference $\Delta I[n] = |I[n] - I[n-1]|$.
2. Applying a directional Hough transform to detect linear features in $\Delta I[n]$.
3. Filtering candidates by angular velocity (meteors traverse $> 1^\circ/\text{frame}$) and photometric profile (rapid brightening then fading).

Detected meteors are composited onto a single reference frame using lighten blending, producing a meteor shower composite image.

24.23.3 Aurora Intensity Tracking and KP Index Correlation

Aurora visibility correlates strongly with geomagnetic activity as measured by the planetary K-index (KP). The engine correlates detected aurora brightness with KP index data retrieved from the NOAA Space Weather Prediction Center API, using clock-corrected timestamps (section 24.18) to ensure accurate temporal alignment:

- 1. Extract per-frame aurora brightness by masking the sky region and computing mean luminance in the green channel (557.7 nm oxygen emission dominates visual aurora).
- 2. Query the NOAA API for 3-hourly KP index values spanning the capture period, using corrected EXIF timestamps.
- 3. Interpolate KP values to per-frame temporal resolution and compute the cross-correlation with the aurora brightness time-series.

Aurora emission colours provide classification of the excitation species:

Table 2. Aurora emission line classification. Each colour corresponds to a specific atmospheric species and excitation altitude.

Colour	Wavelength	Species	Altitude
Green	557.7 nm	Atomic oxygen (O)	100–300 km
Red	630.0 nm	Atomic oxygen (O)	200–400 km
Purple	391–428 nm	Molecular nitrogen (N ₂ ⁺)	80–100 km
Blue	427.8 nm	Molecular nitrogen (N ₂ ⁺)	80–100 km



Figure 76. Aurora–KP index correlation pipeline. EXIF timestamps are first corrected using the clock model (section 24.18), then used to query NOAA’s space weather API for KP index values. The interpolated KP series is cross-correlated with per-frame aurora brightness extracted from the green channel.

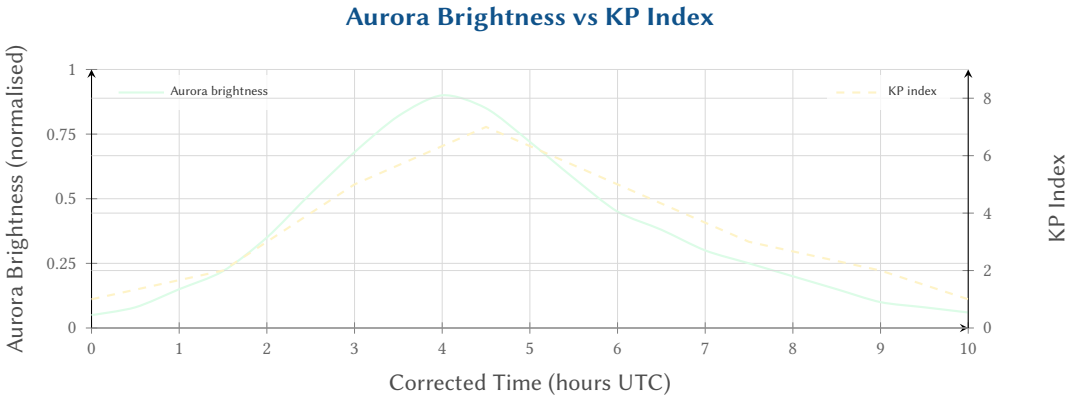


Figure 77. Dual-axis time series of aurora brightness (left axis, solid green) and geomagnetic KP index (right axis, dashed amber). The KP peak at ~4.5 hours precedes the aurora brightness peak, consistent with the propagation delay of magnetospheric disturbances to visible auroral emission. Clock-corrected timestamps are essential for meaningful temporal alignment.

24.23.4 Moon and Planet Tracking via Ephemeris

The engine computes the apparent positions of the Moon, planets, and bright stars using the VSOP87 planetary theory and lunar ephemeris (Meeus, 1991). Given the camera’s GPS coordinates, elevation, and the clock-corrected capture time, the engine predicts the pixel coordinates of each celestial body in the frame and overlays identification labels. Tracking the Moon’s position across a night-long sequence enables automatic lunar phase annotation and transit timing.

24.24 Plugin & Pipeline Extensibility

A fixed pipeline cannot anticipate every domain-specific processing requirement. This domain introduces a WebAssembly (WASM) plugin system that allows third-party developers to inject custom processing stages into the pipeline without recompiling the core engine.

24.24.1 WASM Plugin Architecture

Each plugin is a compiled WASM module that exports a standard interface: `init()`, `process_frame(buf, width, height) -> buf`, and `finalize()`. The engine loads plugins at startup, validates their interface conformance, and inserts them at user-specified positions in the processing chain.

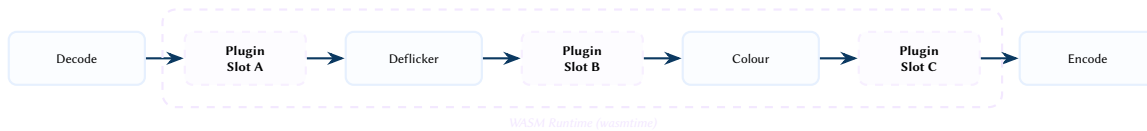


Figure 78. Plugin architecture. Core pipeline stages (solid, blue) are interspersed with WASM plugin slots (dashed, purple). Plugins are loaded at startup from compiled `.wasm` modules and execute within a sandboxed WASM runtime. Each slot is optional—if no plugin is configured, the pipeline bypasses the slot with zero overhead.

24.24.2 Plugin Calling Sequence

The engine manages plugin lifecycle through a three-phase protocol:

1. **Initialisation:** The engine calls `init()` with a JSON configuration blob, allowing the plugin to allocate resources and parse parameters.
2. **Per-frame processing:** For each frame, the engine passes the pixel buffer to `process_frame()`, which returns the modified buffer. The plugin may maintain internal state (e.g. running averages for temporal filtering).
3. **Finalisation:** After the last frame, `finalize()` is called, allowing the plugin to flush buffers, write output files, or report summary statistics.

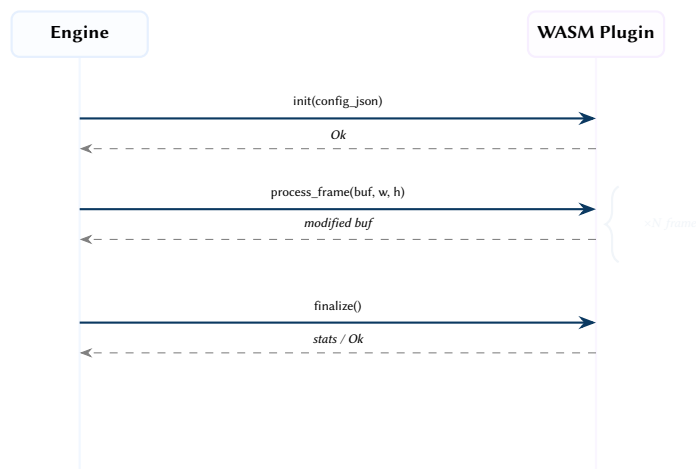


Figure 79. Plugin calling sequence. The engine drives the plugin through three phases: initialisation with a JSON configuration blob, per-frame processing repeated for all N frames, and finalisation for cleanup and statistics reporting.

24.24.3 Plugin Manifest Configuration

Plugins are declared in the YAML pipeline configuration (??) or via a standalone JSON manifest:

```
{
  "plugins": [
    {
      "name": "custom-denoise",
      "wasm_path": "./plugins/denoise.wasm",
      "slot": "after_deflicker",
      "config": {
        "strength": 0.5,
        "temporal_radius": 3
      }
    },
    {
      "name": "watermark",
      "wasm_path": "./plugins/watermark.wasm",

```



```

    "slot": "before_encode",
    "config": {
      "image": "./assets/logo.png",
      "position": "bottom_right",
      "opacity": 0.3
    }
  }
]
}

```

24.24.4 Custom Metric Export

Plugins can register custom metrics via the engine's metrics API. Registered metrics are included in the CSV diagnostic report (section 21) and displayed in the web dashboard (fig. 69). This enables domain-specific measurements—e.g. a vegetation analysis plugin exporting per-frame NDVI values—without modifying the core engine.

24.24.5 Pipeline Stage Reordering

The engine supports declarative pipeline stage reordering via the configuration file. Users specify an ordered list of stage identifiers (including plugin slots), and the engine validates the dependency graph before execution. Invalid orderings—e.g. placing colour correction before demosaicing—are rejected with a diagnostic error message.

24.25 Forensic & Archival

Timelapse sequences used as legal evidence (construction progress, environmental monitoring, surveillance) require verifiable integrity and provenance. This domain introduces cryptographic integrity verification and archival-grade metadata embedding.

24.25.1 Hash Chain Integrity Verification

The engine constructs a hash chain over the processed sequence, where each frame's hash incorporates the previous frame's hash, the pixel data, and the associated metadata:

$$H[n] = \text{SHA256}(H[n-1] \parallel \text{pixels}[n] \parallel \text{metadata}[n]) \quad (109)$$

with the genesis hash $H[0] = \text{SHA256}(\text{pixels}[0] \parallel \text{metadata}[0])$. The final hash $H[N]$ serves as a tamper-evident seal: modifying any single frame or its metadata invalidates the chain from that point forward.

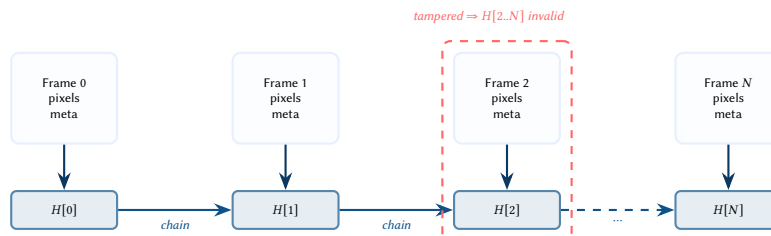


Figure 80. Hash chain integrity verification. Each frame's SHA-256 hash incorporates the previous hash, the pixel data, and the frame metadata. Tampering with any frame (highlighted in red) invalidates all subsequent hashes, providing a tamper-evident seal over the entire sequence.

Verification is performed by recomputing the chain from the stored pixel data and metadata and comparing the final hash against the published seal. The engine supports both full-chain verification (recompute all N hashes) and spot-check verification (verify a random subset of links).

24.25.2 Provenance Metadata Embedding

The engine embeds provenance metadata into each output frame using the C2PA (Coalition for Content Provenance and Authenticity) standard. The metadata records:

- The capture device (camera model, serial number, firmware version).
- The processing pipeline (engine version, configuration hash, plugin list).
- The hash chain position ($H[n]$ and $H[n-1]$).
- GPS coordinates and clock-corrected timestamp (section 24.18).

```

{
  "c2pa_manifest": {
    "claim_generator": "timelapse-engine/0.1.0",
    "assertions": [
      {

```

```

    "label": "c2pa.actions",
    "data": {
      "actions": [
        { "action": "c2pa.captured", "digitalSourceType": "camera" },
        { "action": "c2pa.edited", "description": "deflicker + colour correction" }
      ]
    },
    {
      "label": "timelapse.hash_chain",
      "data": {
        "frame_index": 42,
        "hash": "a3f7c2...b91e",
        "prev_hash": "d8e1f4...c03a",
        "algorithm": "SHA-256"
      }
    },
    {
      "label": "timelapse.capture",
      "data": {
        "camera_model": "Sony A7IV",
        "gps": { "lat": 51.5074, "lon": -0.1278 },
        "corrected_time": "2026-03-15T04:32:17Z"
      }
    }
  ]
}

```

24.25.3 Lossless 16-Bit EXR Intermediate Export

For archival workflows requiring maximum fidelity, the engine exports processed frames as OpenEXR files with 16-bit half-float precision per channel. The EXR format supports arbitrary metadata channels, enabling the engine to embed the hash chain, provenance data, and per-pixel quality metrics (e.g. deflicker confidence) alongside the image data. This lossless intermediate can be reimported into professional compositing software (Nuke, Fusion) for further processing without generational loss.

24.26 Panoramic & Multi-Frame Spatial

Panoramic timelapse extends the spatial extent of each frame by stitching multiple overlapping captures into a single wide-field image, then repeating this process across the temporal axis to produce a timelapse of evolving panoramas. The technique is essential for landscape, architectural, and astrophotographic work where a single focal length cannot capture the desired field of view.

24.26.1 Panoramic Stitching Timelapse

At each capture time t_n , the camera acquires a set of K overlapping frames $\{I_k[n]\}_{k=1}^K$, each related to a common reference frame by a planar homography. Feature-based stitching (Brown & Lowe, 2007) detects and matches keypoints across adjacent frames, then estimates the homography $\mathbf{H}_{k \rightarrow k+1}$ that maps points from frame k into the coordinate system of frame $k + 1$:

$$\tilde{\mathbf{p}}_{k+1} = \mathbf{H}_{k \rightarrow k+1} \tilde{\mathbf{p}}_k, \quad \mathbf{H}_{k \rightarrow k+1} = \begin{pmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & 1 \end{pmatrix} \quad (110)$$

where $\tilde{\mathbf{p}} = (x, y, 1)^\top$ denotes homogeneous coordinates. The eight degrees of freedom are estimated via RANSAC on at least four point correspondences. Chain composition $\mathbf{H}_{1 \rightarrow k} = \mathbf{H}_{k-1 \rightarrow k} \cdots \mathbf{H}_{1 \rightarrow 2}$ maps all frames into the reference coordinate system, and multi-band blending (Burt & Adelson, 1983) produces the final composite.

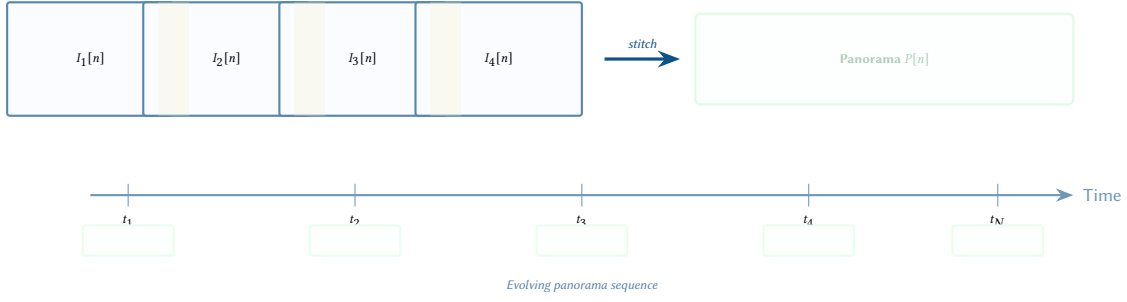


Figure 81. Panoramic stitching timelapse pipeline. At each capture time t_n , overlapping frames $I_1[n] \dots I_K[n]$ are aligned via homographies and blended into a single panorama $P[n]$. The bottom axis shows the resulting temporal sequence of panoramas.

24.26.2 Gigapixel Timelapse with Multi-Zoom Navigation

For high-resolution panoramic captures with hundreds of megapixels per composite, the engine generates tiled pyramid representations (similar to web map tile schemes) at each time step. The temporal navigator allows zoom into sub-regions while simultaneously scrubbing through time, enabling inspection of fine detail evolution within the wider panoramic context.

24.26.3 Multi-Row Panoramic for Motorised Heads

Motorised pan-tilt heads capture multi-row grids of overlapping images. The engine extends the single-row stitching model to a 2D grid, computing homographies $\mathbf{H}_{(r,c) \rightarrow (r',c')}$ between spatially adjacent tiles. Bundle adjustment jointly optimises all camera orientations, minimising reprojection error across the full grid. Temporal consistency is maintained by anchoring the bundle to the previous frame's solution, preventing inter-frame jitter.

24.27 Biological & Microscopy

Microscopy timelapse captures biological processes across both spatial depth (z-axis) and time, producing four-dimensional datasets (3D+t). The engine extends its processing model to handle z-stack acquisition, focus stacking, cell tracking, and event detection, enabling quantitative analysis of biological dynamics.

24.27.1 Z-Stack Timelapse and Focus Stacking

At each time step t_n , the microscope acquires a stack of D images at different focal depths $\{I_d[n]\}_{d=1}^D$. Focus stacking selects the sharpest region from each depth slice to produce an all-in-focus composite. The sharpness at each pixel is measured by the Laplacian focus measure:

$$\mathcal{F}_d(x, y) = |\nabla^2 I_d(x, y)|^2 = |I_d * \mathcal{L}|^2, \quad \mathcal{L} = \begin{pmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{pmatrix} \quad (111)$$

The composite selects, for each pixel, the depth slice with maximum focus measure: $d^*(x, y) = \arg \max_d \mathcal{F}_d(x, y)$. Gaussian-weighted blending across adjacent depth slices prevents sharp transitions in the depth selection map.

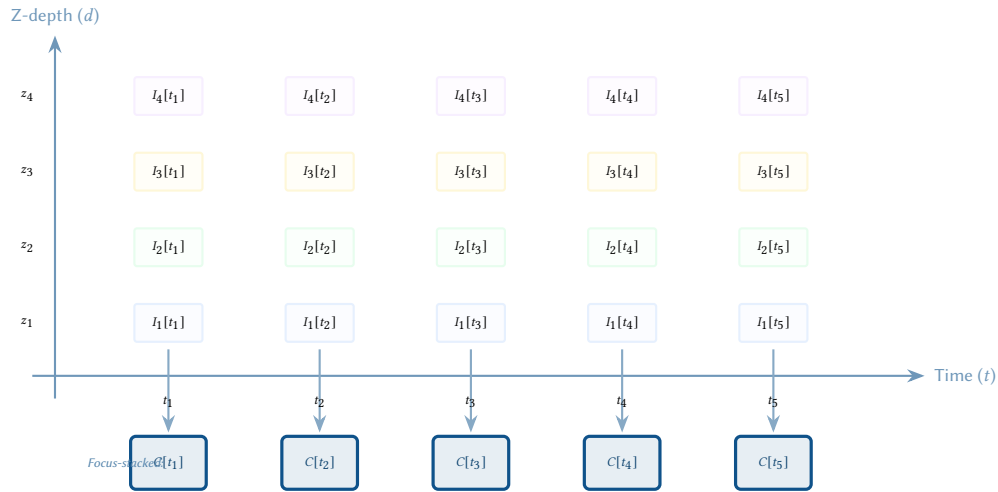


Figure 82. Z-stack timelapse as a 3D+t dataset. Each column represents one time step with $D = 4$ depth slices. The focus-stacked composites $C[t_n]$ along the bottom combine the sharpest regions from each depth, producing the final all-in-focus timelapse.

24.27.2 Cell and Organism Tracking

The engine implements a tracking-by-detection pipeline for cell and organism motion analysis. After segmentation (via thresholding, watershed, or a pre-trained U-Net), detected objects are linked across frames using the Hungarian algorithm to minimise a cost matrix based on spatial proximity and morphological similarity. Migration paths are plotted as overlaid polylines with colour-coded velocity, enabling visual assessment of chemotaxis, wound healing, or collective migration patterns.

24.27.3 Growth Rate Measurement

For calibrated microscopy setups (known $\mu\text{m}/\text{pixel}$ ratio), the engine computes object area at each time step and fits a growth model. Exponential growth $A(t) = A_0 e^{\mu t}$ is fitted via linear regression on $\ln A(t)$, yielding the specific growth rate μ . The engine annotates each frame with calibrated measurements and exports growth curves as CSV data.

24.27.4 Mitosis and Division Event Detection

Cell division events are detected by monitoring tracked objects for rapid area increase followed by splitting into two daughter cells. The engine flags frames where a tracked cell's area exceeds $1.5\times$ its running median, then verifies by checking for the appearance of two new tracks in subsequent frames. Division events are logged with timestamps and spatial coordinates.

24.28 Construction & Architecture

Construction timelapse serves both documentation and project management purposes. By aligning Building Information Model (BIM) geometry to the camera view, the engine enables automated progress tracking, safety monitoring, and activity analysis over multi-month or multi-year construction timelines.

24.28.1 BIM Model Overlay and Progress Tracking

The engine aligns a 3D BIM model to the camera's extrinsic and intrinsic parameters through a manual calibration step: the operator identifies ≥ 6 corresponding point pairs between the BIM wireframe and visible structural features in a reference frame. The resulting projection matrix $\mathbf{P} = \mathbf{K}[\mathbf{R} \mid \mathbf{t}]$ maps 3D model vertices to 2D image coordinates:

$$\tilde{\mathbf{p}} = \mathbf{P} \tilde{\mathbf{X}}, \quad \mathbf{P} = \mathbf{K}[\mathbf{R} \mid \mathbf{t}] \in \mathbb{R}^{3 \times 4} \quad (112)$$

where $\mathbf{K}$ is the 3×3 intrinsic matrix, $\mathbf{R}$ is the rotation, and $\mathbf{t}$ is the translation vector. The BIM wireframe is rendered as a semi-transparent overlay atop each frame, enabling visual comparison of planned versus actual geometry.

Camera-to-Model Calibration BIM overlay requires manual calibration of the camera-to-model projection. The operator must identify at least six corresponding point pairs between the BIM wireframe and visible structural features. Recalibration is required whenever the camera is repositioned.

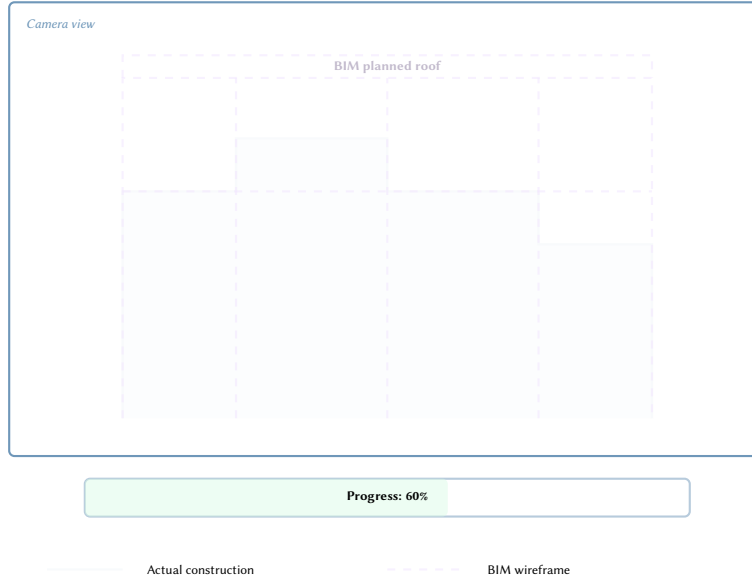


Figure 83. BIM model overlay on a construction timelapse frame. The solid fill represents the current state of construction; the dashed wireframe shows the planned BIM geometry. The progress bar estimates completion at 60% based on the ratio of occupied to planned volume.

24.28.2 Progress Percentage Estimation

The engine estimates construction progress by computing the fraction of BIM-projected area that is occupied by detected structure. A simple occupancy metric compares the as-built silhouette (extracted via background subtraction or semantic segmentation) against the full BIM projection:

$$\text{Progress}(t) = \frac{A_{\text{built}}(t)}{A_{\text{BIM}}} \quad (113)$$

This coarse metric is refined by weighting structural elements according to their cost or schedule significance as encoded in the BIM metadata.

24.28.3 Safety Zone Monitoring

The engine defines exclusion zones in the camera view (e.g. crane swing radii, active demolition areas) as polygonal regions. Personnel or equipment detected within these zones via background subtraction or object detection triggers a logged violation event with the frame number, timestamp, zone identifier, and a cropped image of the violation.

24.28.4 Crane Tracking for Activity Pattern Visualisation

Tower cranes are detected by their distinctive silhouette and tracked across frames to extract rotation angle and load position. The engine accumulates crane activity into a polar heatmap showing the angular distribution of crane operations over time, revealing workflow patterns and identifying underutilised sectors of the construction site.

24.29 Energy & Infrastructure Monitoring

Timelapse imaging of energy infrastructure enables long-term performance monitoring, fault detection, and operational analytics. The engine processes sequences from fixed cameras overlooking solar arrays, wind farms, roads, and civil structures to extract quantitative engineering data.

24.29.1 Solar Panel Shadow Coverage Tracking

Shadow coverage on solar arrays directly impacts energy output. The engine segments shadowed regions by thresholding the luminance channel against a clear-sky reference, computing the shadow fraction at each time step:

$$S(t) = \frac{A_{\text{shadowed}}(t)}{A_{\text{total}}} \quad (114)$$

where $A_{\text{shadowed}}(t)$ is the area of panels classified as shadowed at time t and A_{total} is the total panel area. The engine tracks shadow movement across the array throughout the day, correlating with solar azimuth and elevation computed from the GPS-stamped capture time.

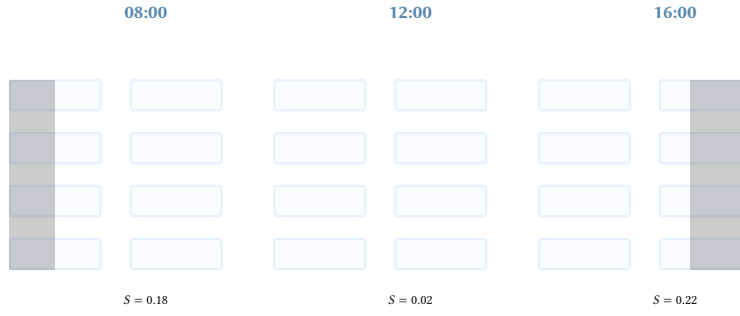


Figure 84. Solar array shadow coverage at three times of day. Dark overlays indicate shadowed panel regions. The shadow fraction $S(t)$ decreases toward solar noon and increases in the morning and afternoon as adjacent structures cast longer shadows.

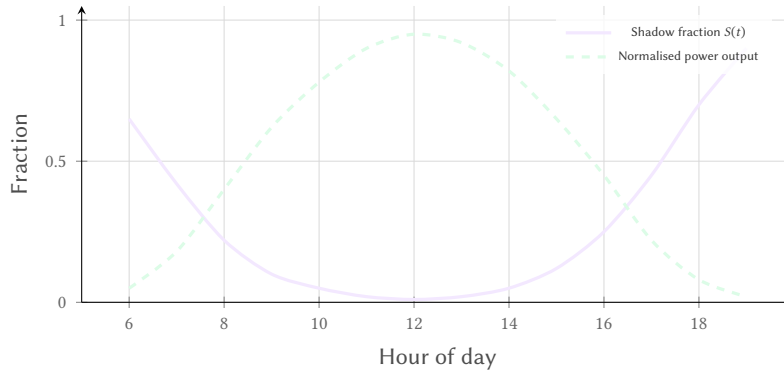


Figure 85. Shadow fraction $S(t)$ and normalised power output versus hour of day for a solar array monitored by timelapse. The inverse correlation confirms that shadow coverage is a primary driver of output reduction, enabling the engine to quantify shading losses from timelapse imagery alone.

24.29.2 Wind Turbine Rotation Speed Measurement

The engine tracks blade tip positions across frames to estimate turbine rotation speed. Blade tips are detected by their high-contrast silhouette against the sky, and angular velocity is computed from the inter-frame angular displacement divided by the frame interval. The resulting RPM time series can be correlated with meteorological data to assess turbine performance.

24.29.3 Traffic Flow Engineering Data Export

For timelapse sequences of road infrastructure, the engine applies vehicle detection and tracking to extract traffic flow parameters: vehicle count, average speed, headway distribution, and lane occupancy. These are exported in standard traffic engineering formats for integration with transportation planning tools.

24.29.4 Long-Term Infrastructure Degradation Tracking

Multi-year timelapse sequences of bridges, dams, and buildings enable visual tracking of structural degradation. The engine computes frame-to-frame structural similarity (SSIM) maps, highlighting regions of progressive change such as crack propagation, spalling, vegetation encroachment, or discolouration. Change rates are plotted over time to flag accelerating degradation.

24.30 Thermal & Multispectral Imaging

Thermal and multispectral cameras capture information beyond the visible spectrum, enabling applications in building diagnostics, agriculture, ecology, and industrial monitoring. The engine extends its pipeline to ingest, align, and fuse imagery from multiple spectral bands.

24.30.1 Thermal Camera Timelapse with Calibrated Temperature

Radiometric thermal cameras produce per-pixel intensity values proportional to incident thermal radiation. The engine converts these raw values to calibrated temperatures by inverting the Planck radiation law. For a camera with known spectral response centred at wavelength λ , the temperature is:

$$T(x, y) = \frac{hc}{\lambda k_B \ln\left(\frac{2hc^2}{\lambda^5 L(x, y)} + 1\right)} \quad (115)$$

where h is Planck's constant, c the speed of light, k_B Boltzmann's constant, and $L(x, y)$ the measured spectral radiance. The engine applies this per-pixel inversion to produce calibrated temperature maps at each time step, enabling thermal timelapse with physically meaningful colour scales.

24.30.2 NDVI from IR-Modified Cameras

Cameras with their infrared-blocking filter removed can capture near-infrared (NIR) reflectance. The Normalised Difference Vegetation Index (NDVI) quantifies vegetation vigour:

$$\text{NDVI}(x, y) = \frac{\text{NIR}(x, y) - \text{Red}(x, y)}{\text{NIR}(x, y) + \text{Red}(x, y)} \quad (116)$$

NDVI values range from -1 to $+1$, with healthy vegetation typically yielding $\text{NDVI} > 0.3$. The engine computes per-pixel NDVI at each time step and renders false-colour maps using a diverging green–brown palette, producing timelapse sequences that visualise seasonal vegetation dynamics.

24.30.3 Multispectral Fusion Pipeline

The engine aligns and fuses imagery from multiple spectral bands (visible, thermal, NIR) captured by co-located or co-registered cameras. Geometric alignment compensates for inter-camera parallax via homography estimation, while radiometric normalisation ensures consistent intensity scales across bands. The fused composite supports arbitrary band-to-channel mapping for domain-specific visualisation.

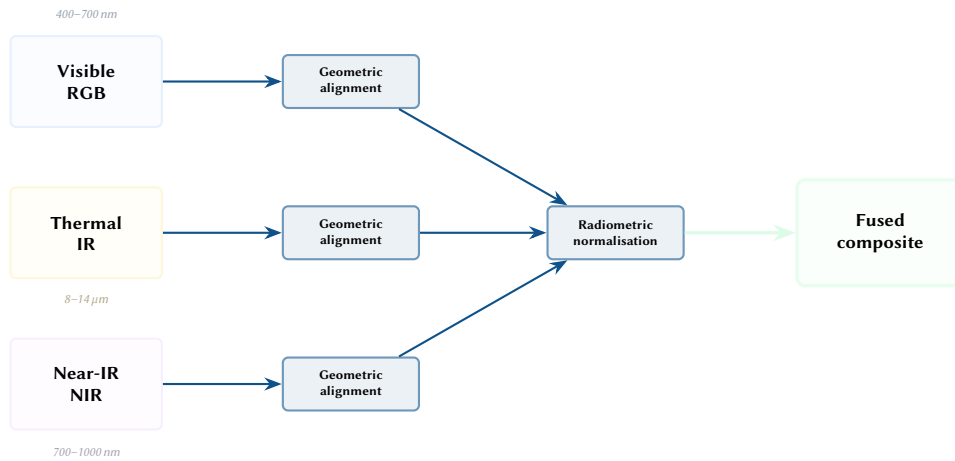


Figure 86. Multispectral fusion pipeline. Three input streams (visible RGB, thermal IR, and near-infrared) are geometrically aligned and radiometrically normalised before fusion into a composite output supporting arbitrary band-to-channel mapping.

24.30.4 Thermal Anomaly Detection

The engine detects thermal anomalies by comparing each frame's temperature map against a temporal baseline (e.g. rolling mean over the preceding week). Pixels exceeding a configurable threshold above or below the baseline are flagged and clustered into anomaly regions. This supports applications such as detecting insulation failures in buildings, overheating equipment in industrial settings, and thermal venting in geothermal monitoring.

24.31 Accessibility & Documentation

Ensuring that timelapse outputs and diagnostic visualisations are accessible to all users—including those with colour vision deficiencies—is a core design requirement. This domain introduces palette remapping, automated narration, and structured report generation.

24.31.1 Colorblind-Safe Palette Remapping

All engine visualisations (heatmaps, flow fields, diagnostic overlays) support palette switching to accommodate the three most common colour vision deficiencies: deuteranopia (red–green, most common), protanopia (red–green, less common), and tritanopia (blue–yellow, rare). The engine ships a set of perceptually uniform palettes derived from the Oklab colour space, ensuring that luminance contrast is preserved even when chromatic information is reduced.

Palette Switching for Diagnostic Outputs All diagnostic visualisations produced by the engine must support palette switching via a single configuration parameter. No colour-coded output shall rely solely on hue to convey information; luminance and pattern variation must provide redundant encoding.

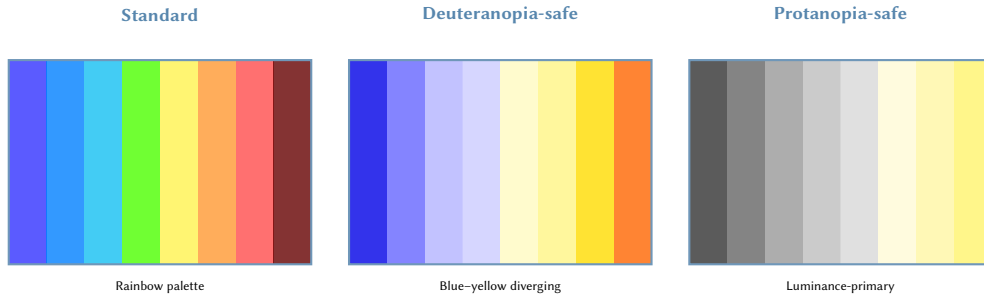


Figure 87. The same heatmap data rendered in three palettes: a standard rainbow palette (left), a deuteranopia-safe blue–yellow diverging palette (centre), and a protanopia-safe luminance-primary palette (right). All three maintain perceptual uniformity through controlled luminance progression.

24.31.2 Auto-Generated Text Narration of Scene Changes

The engine analyses temporal summaries (e.g. activity heatmaps, brightness curves, detected events) and generates structured natural-language descriptions suitable for screen readers or text-to-speech systems. For each major scene change (day–night transition, weather event, significant motion), the engine produces a timestamped narrative entry: “Frame 1200 (14:32): Cloud cover increases, mean luminance drops 35% over 40 frames.”

24.31.3 Processing Report PDF Generation

For client delivery workflows, the engine generates a structured PDF report summarising the processing pipeline, key parameters, quality metrics, and representative frame thumbnails. The report includes deflicker curves, histogram evolution plots, detected events, and configuration details, providing a complete audit trail of the processing applied to the sequence.

24.32 Creative Compositing

Creative compositing extends the engine beyond uniform temporal processing to support spatially varying time control, blending modes borrowed from image editing, and generative effects that exploit the temporal dimension for artistic output.

24.32.1 Selective Time Control per Zone

The frame is partitioned into spatial zones (e.g. sky, midground, foreground) via user-defined masks or semantic segmentation. Each zone is assigned an independent playback speed multiplier σ_z , so that the output frame at display time τ composites content from different source times:

$$O(\mathbf{x}, \tau) = \sum_z M_z(\mathbf{x}) I(\mathbf{x}, \tau \cdot \sigma_z) \quad (117)$$

where $M_z(\mathbf{x})$ is the binary mask for zone z (with $\sum_z M_z = 1$ everywhere) and σ_z is the speed multiplier. Fractional source times are handled by the same frame interpolation used elsewhere in the pipeline. This enables effects such as fast-moving clouds over a slowly evolving foreground.

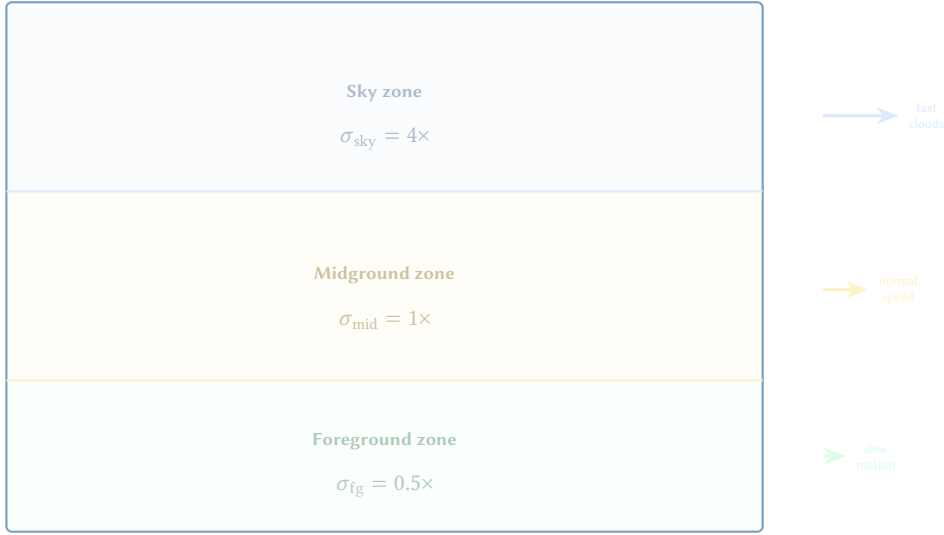


Figure 88. Selective time control per zone. The frame is partitioned into sky ($\sigma = 4\times$), midground ($\sigma = 1\times$), and foreground ($\sigma = 0.5\times$) zones. Each zone samples from a different source time, creating the effect of fast-moving clouds over a slowly evolving foreground.

24.32.2 Temporal Blending Modes

The engine supports compositing two temporal layers (e.g. the same sequence at different time offsets) using standard blending modes. For normalised pixel values $A, B \in [0, 1]$:

$$\text{Multiply : } O = A \cdot B \quad (118)$$

$$\text{Screen : } O = 1 - (1 - A)(1 - B) \quad (119)$$

$$\text{Overlay : } O = \begin{cases} 2AB & \text{if } A \leq 0.5 \\ 1 - 2(1 - A)(1 - B) & \text{if } A > 0.5 \end{cases} \quad (120)$$

$$\text{Difference : } O = |A - B| \quad (121)$$

Temporal blending enables effects such as self-shadowing (multiply with a time-shifted copy), glow accumulation (screen), and motion extraction (difference between adjacent time windows).

24.32.3 Kaleidoscope and Mirror with Temporal Evolution

The engine applies geometric transformations—reflection, rotation, and tiling—to produce kaleidoscopic patterns that evolve over time. The kaleidoscope angle, number of reflections, and centre point can be keyframed across the sequence, creating smoothly morphing symmetric patterns driven by the underlying scene motion.

24.32.4 Double Exposure Simulation

The engine composites frames from non-adjacent time ranges to simulate double (or multiple) exposure. The operator specifies two or more time windows, and the engine blends frames from each window using screen or additive blending. This produces the characteristic transparent-overlay aesthetic of film double exposure, with the temporal separation creating surreal juxtapositions of different times of day, weather conditions, or activity levels.

24.33 Temporal Bokeh & Focus-on-Change

Traditional bokeh arises from spatial depth-of-field: objects at the wrong distance appear soft while the focal plane stays sharp. This section introduces a novel analogue along the *temporal* axis. Instead of physical distance, the “depth” is temporal activity—regions that remain static over many frames accumulate blur and take on a creamy, dreamlike quality, while regions of change stay crisp and attention-grabbing. The perceptual effect is that the viewer’s eye is drawn to *what changed*, not merely to what is closest.

24.33.1 Temporal Bokeh

For each pixel (x, y) , the engine computes the temporal variance across a window of N frames:

$$\sigma_t^2(x, y) = \frac{1}{N} \sum_{n=1}^N (I[n](x, y) - \mu_t(x, y))^2 \quad (122)$$

where $\mu_t(x, y) = \frac{1}{N} \sum_n I[n](x, y)$ is the temporal mean. Low variance indicates a static region; high variance indicates motion or change. The temporal bokeh kernel width is set inversely proportional to temporal activity:

$$w(x, y) = w_{\max} \cdot \left(1 - \frac{\sigma_t(x, y)}{\sigma_{\max}} \right) \quad (123)$$

where $w_{\max}$ is the maximum blur radius (user-specified) and $\sigma_{\max}$ is clamped to the observed maximum temporal standard deviation. The output at frame n is the normalised Gaussian-weighted temporal average:

$$O[n](x, y) = \frac{\sum_k G(k; w(x, y)) \cdot I[n+k](x, y)}{\sum_k G(k; w(x, y))} \quad (124)$$

where $G(k; w)$ is a Gaussian kernel with standard deviation proportional to w . Static pixels receive a wide kernel and blur into soft temporal streaks; active pixels receive a narrow kernel (approaching a delta function) and pass through essentially unchanged.

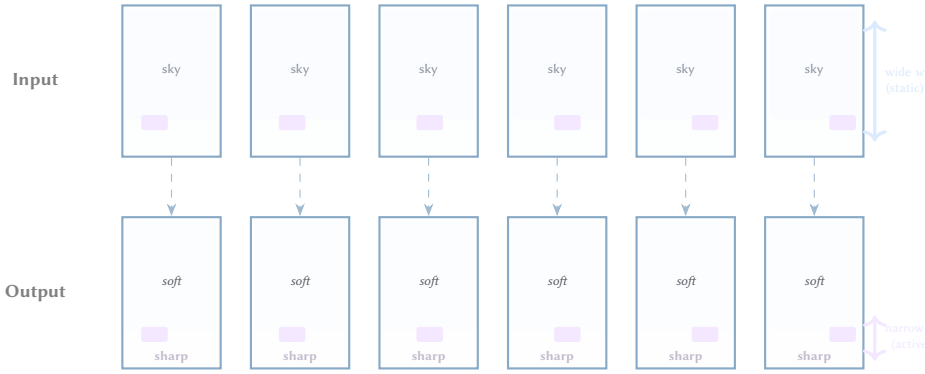


Figure 89. Temporal bokeh concept. A row of input frames (top) contains a static sky and a moving vehicle. In the output (bottom), the static sky accumulates temporal Gaussian blur and becomes soft and dreamy, while the moving vehicle—which has high temporal variance—passes through with minimal blur, remaining sharp and visually prominent. The blur kernel width w varies per-pixel as a function of temporal activity.

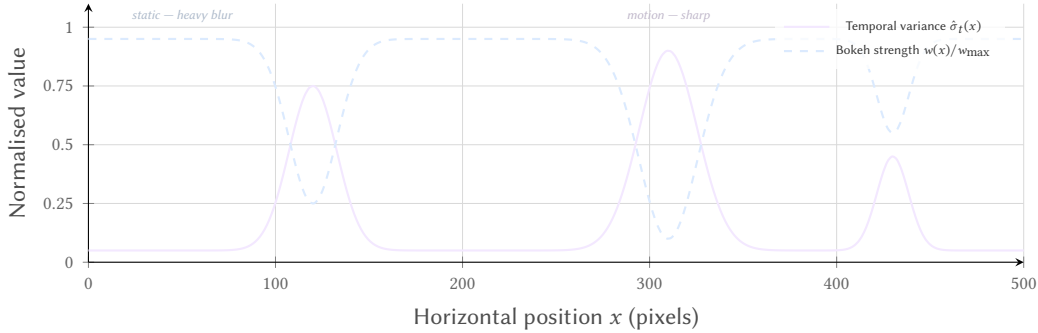


Figure 90. Temporal variance cross-section at a fixed scanline. High temporal variance (solid, purple) indicates regions where objects moved; low variance indicates static background. The bokeh strength curve (dashed, blue) is the inverse: static regions receive heavy temporal blur while active regions remain sharp. The cross-over behaviour ensures the viewer's attention is drawn to change.

24.33.2 Selective Temporal Sharpening

A temporal unsharp mask is applied selectively to high-activity regions to further enhance the perceptual contrast between static and dynamic content. The temporally sharpened output is:

$$I'[n](x, y) = I[n](x, y) + \alpha_t \cdot (I[n](x, y) - O[n](x, y)) \cdot [\sigma_t(x, y) > \tau_\sigma] \quad (125)$$

where $O[n]$ is the temporally blurred version from eq. (124), α_t is the sharpening strength, and τ_σ is the activity threshold. The indicator function $[\cdot]$ restricts sharpening to active regions, preventing noise amplification in static areas.

24.33.3 Temporal Depth-of-Field

By analogy with spatial rack focus, the engine defines a *focus time* t_{focus} and applies progressive temporal blur to frames distant from it:

$$w_{\text{dof}}[n] = w_{\text{max}} \cdot \min\left(1, \frac{|n - t_{\text{focus}}|}{r_{\text{dof}}}\right) \quad (126)$$

where r_{dof} is the temporal depth-of-field radius (in frames). Frames at $n = t_{\text{focus}}$ are perfectly sharp; frames at $|n - t_{\text{focus}}| \geq r_{\text{dof}}$ receive maximum blur. Animating t_{focus} across the sequence produces a *temporal rack focus* effect—the viewer’s temporal attention is smoothly guided through the sequence, with the “in-focus” moment sweeping forward in time.

24.33.4 Temporal Frequency Separation

The temporal dimension of the sequence can be decomposed into frequency bands analogous to spatial frequency separation in retouching. Along the time axis at each pixel, the low-frequency component captures slow changes (seasonal drift, diurnal lighting cycles) while the high-frequency component captures rapid events (traffic, pedestrians, weather transients):

$$I_{\text{low}}[n](x, y) = (I * G_{\sigma_t})[n](x, y) \quad (127)$$

$$I_{\text{high}}[n](x, y) = I[n](x, y) - I_{\text{low}}[n](x, y) \quad (128)$$

where $*$ denotes temporal convolution and G_{σ_t} is a Gaussian kernel along the frame index axis with bandwidth parameter σ_t . The two bands can be processed independently—for example, applying colour grading only to I_{low} (changing the diurnal palette) while leaving I_{high} untouched, or boosting I_{high} to exaggerate transient motion. Multiple bandpass filters can further separate the signal into fine-grained temporal frequency bands (e.g., sub-hourly vs. multi-hour vs. multi-day variation).

24.34 Object Detection & Event Intelligence

Applying per-frame object detection to timelapse sequences transforms the pixel-level data into a structured semantic event log. This domain integrates YOLO-class detectors (Redmon et al. 2016) with temporal tracking algorithms to enable class-driven narrative automation, rare event discovery, and semantic trail rendering.

24.34.1 Per-Frame Event Tagging

A detector such as YOLOv8 or RT-DETR is run on each frame (or on a decimated subset at interval s for efficiency), producing timestamped bounding boxes with class labels and confidence scores. These detections are aggregated into a machine-readable event log:

```
{
  "events": [
    {
      "frame": 142,
      "timestamp": "2024-07-15T08:22:14Z",
      "detections": [
        { "class": "crane", "confidence": 0.94,
          "bbox": [120, 45, 380, 290] }
      ]
    },
    {
      "frame": 305,
      "timestamp": "2024-07-15T09:14:50Z",
      "detections": [
        { "class": "person", "confidence": 0.87,
          "bbox": [510, 320, 580, 480] },
        { "class": "vehicle", "confidence": 0.91,
          "bbox": [50, 280, 210, 390] }
      ]
    }
  ]
}
```

The event log doubles as a chapter marker file: video editors can import it to create jump points at significant detection events.

24.34.2 Object Persistence Tracking

Individual detections are linked across frames using a multi-object tracker such as DeepSORT (Wojke et al. 2017) or ByteTrack (Zhang et al. 2022). Each tracked object k receives an entry time $n_{\text{entry}}^{(k)}$, exit time $n_{\text{exit}}^{(k)}$, and trajectory. The persistence duration is:

$$D_k = \frac{n_{\text{exit}}^{(k)} - n_{\text{entry}}^{(k)}}{f_{\text{capture}}} \quad (129)$$

where f_{capture} is the capture frame rate (e.g., one frame per four minutes). This enables annotations such as “Vehicle #14 parked for 47 frames (3.2 hours)” or “Crane present for 1,200 frames (83 days),” providing a quantitative activity log for construction or urban monitoring projects.

24.34.3 Rare Event Highlighting

Objects or events appearing in fewer than $p\%$ of frames (typically $p = 1$) are flagged as rare events and auto-bookmarked. The rarity score for class c is:

$$R(c) = 1 - \frac{|\{n : c \in \mathcal{D}[n]\}|}{N} \quad (130)$$

where $\mathcal{D}[n]$ is the set of detected classes in frame n . High-rarity detections—a fox crossing a construction site at dawn, a lightning strike, an unusual vehicle—are surfaced as highlights in the event log, enabling efficient review of multi-thousand-frame sequences.

24.34.4 Class-Based Automatic Speed Ramping

Object class presence drives narrative speed curves. The playback speed multiplier at frame n is:

$$v(n) = v_{\text{base}} \cdot (1 + \lambda \cdot C(n))^{-1} \quad (131)$$

where $C(n) = \sum_c w_c \cdot [c \in \mathcal{D}[n]]$ is the weighted interesting-class count (with user-specified per-class weights w_c) and λ controls sensitivity. When people or animals are detected the playback slows to let the viewer observe; empty frames accelerate. The resulting speed curve is exported as a draft narrative YAML for user review and refinement.

24.34.5 Object Count Transitions

The engine monitors the count of each class c across frames and detects threshold crossings—moments when $|\mathcal{D}_c[n]|$ transitions across a user-defined boundary (e.g., parking lot empty $\rightarrow$ full, or crane count 0 $\rightarrow$ 1). These transitions are marked as scene boundaries in the event log and can drive automatic chapter segmentation.

24.34.6 Semantic Object Trails

Unlike generic optical flow trails (section 24.5), semantic trails are rendered per object class with distinct colours. The composite trail image T accumulates tracked centroids:

$$T(x, y) = \sum_k \text{col}(c_k) \cdot \sum_{n=n_{\text{entry}}^{(k)}}^{n_{\text{exit}}^{(k)}} G((x, y) - \mathbf{p}_k[n]; \sigma_{\text{trail}}) \quad (132)$$

where $\mathbf{p}_k[n]$ is the centroid of tracked object k at frame n , c_k is its class, and $\text{col}(c)$ maps class to colour (e.g., vehicle headlights rendered in red, pedestrians in blue, clouds in white). The result is a single composite image summarising all motion in the sequence, semantically labelled by object type.

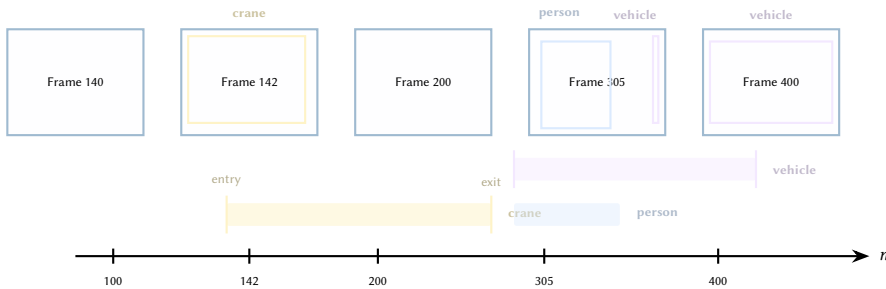


Figure 91. Per-frame object detection with event timeline. Top: selected frames with detection bounding boxes annotated by class (crane, person, vehicle). Bottom: class presence bars on the timeline axis, with entry and exit markers. The event log records every detection with frame index, timestamp, class, bounding box, and confidence.



Figure 92. Semantic object trails. Each object class is rendered with a distinct colour on a single composite image: vehicle trajectories in purple, pedestrian paths in blue, cloud movement in slate. Unlike generic flow trails, these are semantically labelled, enabling class-specific analysis of motion patterns across the entire sequence.

24.35 Attention & Narrative Intelligence

Computational attention models and multi-signal analysis enable the engine to automatically identify what is interesting in a timelapse sequence and generate optimal narrative pacing. This domain combines visual saliency, change detection, object presence, and photometric signals into a unified interestingness score that drives playback speed, crop positioning, and depth-of-field decisions. The narrative engine generates suggestions; it never overrides explicit user keyframes from the narrative YAML.

24.35.1 Saliency-Based Attention Heatmap

A visual saliency model (Itti & Koch, 2001) is run per frame to predict fixation probability across the image. The saliency map $S[n](x, y) \in [0, 1]$ highlights regions that would attract a human viewer's gaze—high contrast edges, faces, isolated motion, colour pop-out. The engine uses saliency for three purposes: (i) auto-positioning crops to maximise salient content within the output aspect ratio, (ii) weighting the speed ramping signal so that frames with concentrated saliency slow down, and (iii) guiding the temporal depth-of-field focus time t_{focus} (section 24.33) toward temporally salient moments.

24.35.2 Narrative Tension Curve

Multiple frame-level signals are combined into a single normalised *interestingness* score $I(n)$:

$$I(n) = w_{\Delta} \hat{\Delta}(n) + w_C \hat{C}(n) + w_Y \left| \frac{d\hat{Y}}{dt} \right|(n) + w_E \left| \frac{d\hat{E}}{dt} \right|(n) \quad (133)$$

where $\hat{\Delta}(n)$ is the normalised change magnitude (eq. (53)), $\hat{C}(n)$ is the normalised interesting-object count (eq. (131)), $|d\hat{Y}/dt|(n)$ is the normalised luminance rate of change, and $|d\hat{E}/dt|(n)$ is the normalised CIELAB colour shift rate. All signals are min-max normalised to $[0, 1]$ before weighting. The weights w_{Δ} , w_C , w_Y , w_E are user-tuneable (defaulting to equal).

The narrative speed is the inverse of tension:

$$v(n) = v_{\min} + (v_{\max} - v_{\min}) \cdot (1 - I(n)) \quad (134)$$

High interestingness produces slow playback; low interestingness produces fast playback. The speed curve is smoothed via PCHIP interpolation (section 6) to avoid perceptually jarring speed transitions.

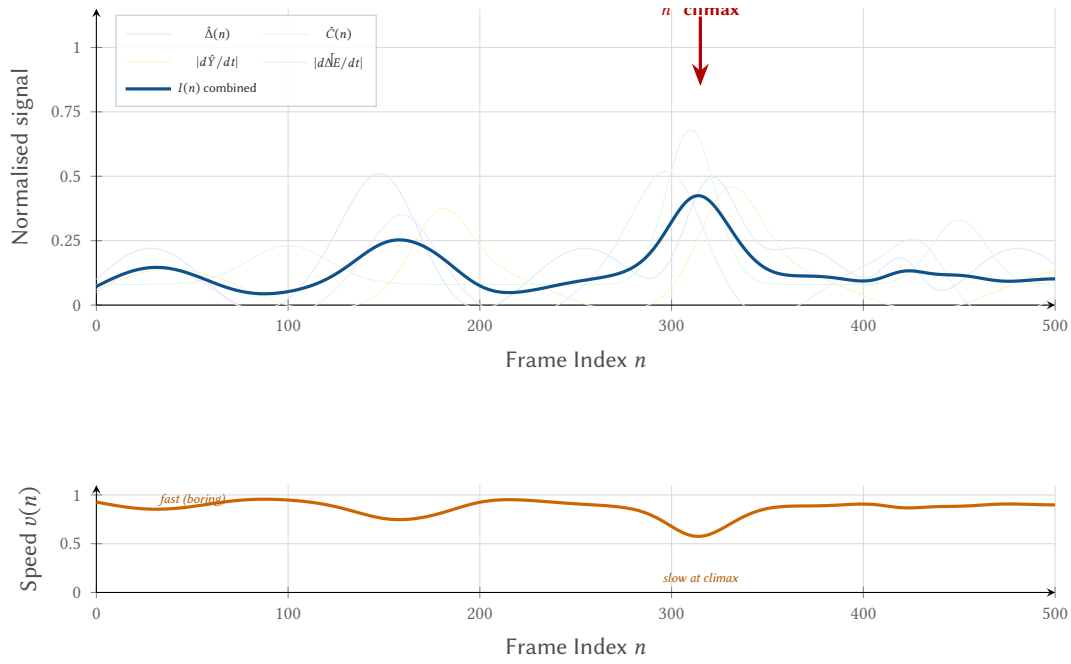


Figure 93. Narrative tension curve (top) and resulting speed curve (bottom). Individual signals—change magnitude $\hat{\Delta}$, object count $\hat{C}$, luminance rate $|d\hat{Y}/dt|$, and colour shift rate $|d\hat{E}/dt|$ —are shown as thin coloured lines. The combined interestingness $I(n)$ (thick line) peaks at the detected climax n^* . The speed curve (bottom, orange) is the inverse: playback slows at interesting moments and accelerates through static intervals.

24.35.3 Automatic Climax Detection

The climax frame is identified as the global maximum of the interestingness score:

$$n^* = \arg \max_n I(n) \quad (135)$$

The narrative engine anchors the final output around n^* : the approach phase decelerates gradually (building anticipation), the climax region plays at $v_{\min}$ (holding the viewer’s attention), and the exit phase re-accelerates (graceful resolution). A narrative arc template—exposition, rising action, climax, falling action—is mapped onto the interestingness curve to produce a cinematically satisfying pacing profile.

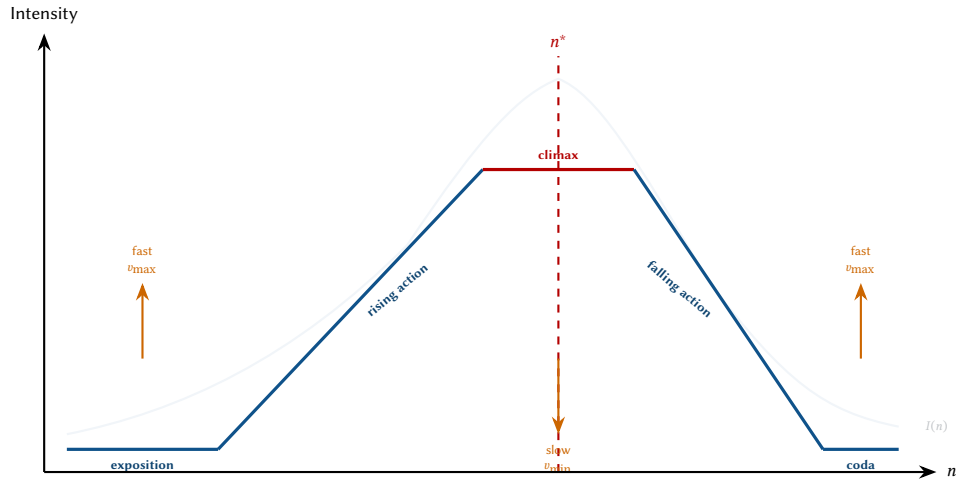


Figure 94. Narrative arc template mapped onto the interestingness curve. The grey curve shows the raw interestingness $I(n)$; the coloured overlay shows the five-act narrative template. Playback speed (orange arrows) is highest during exposition and coda, lowest at the detected climax n^* . The template adapts its shape to the actual interestingness distribution, ensuring the narrative arc feels natural regardless of the underlying scene dynamics.

24.36 Performance & Quality Extensions

The following extensions improve the core pipeline’s performance and quality independently of the feature domains above.

Adaptive Gaussian smoothing. Replace the fixed-width boxcar with a Gaussian kernel whose σ adapts to the local luminance gradient, providing stronger suppression in stable regions while preserving rapid transitions.

GPU-accelerated correction. The per-pixel correction loop (section 20) is embarrassingly parallel and would benefit from GPU dispatch via wgpu or CUDA, reducing per-frame processing time from milliseconds to microseconds.

HDR / PQ output. Extending the pipeline to output 10-bit HEVC with the PQ (Perceptual Quantizer, SMPTE ST 2084) transfer function would exploit the full 16-bit internal precision without the dithering lossy step.

Optical flow stabilisation. Integrating geometric stabilisation before photometric correction would enable the engine to handle handheld sequences, computing affine or homographic warps to align successive frames.

Extended XMP metadata. Expanding the XMP sidcar to include white balance (`crs:Temperature`, `crs:Tint`) and tone curve corrections would enable full non-destructive round-tripping for the chromatic and tonal adjustments.

25 Conclusion

By treating the timelapse as a formal signal processing problem rather than a basic video conversion task, we achieve a level of temporal coherence and narrative control that matches professional cinema standards.

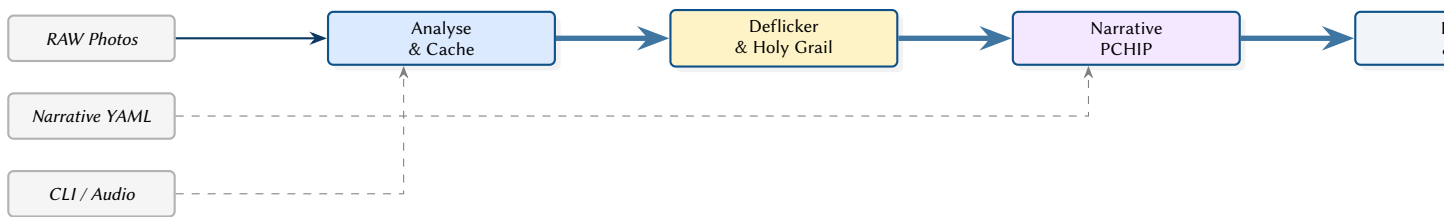


Figure 95. End-to-end workflow summary. Three inputs feed a four-phase pipeline producing four output categories: video with optional audio, XMP sidcars with HDR stacking, NLE manifests (XMEML/CSV), and diagnostics.

The key contributions of this report are:

1. A formal multiplicative noise model for mechanical stochastic flicker, grounded in the physics of aperture and shutter assemblies (section 1).
2. A CIE 1931 photometric normalisation framework that ensures physically correct luminance estimation in the linear domain (section 2).
3. A temporal smoothing operator with sigma-clipped robust estimation, exponential confidence smoothing, and spatiotemporal stability filtering (sections 3, 4, 4.3 and 4.4).
4. A log-linear Holy Grail transform with deadband thresholding for seamless day-to-night transitions (section 5).
5. A Fritsch–Carlson PCHIP interpolator providing mathematically guaranteed monotonicity for narrative speed ramping (section 6).
6. Dual temporal blur kernels (Gaussian and linear) with optional optical-flow guided warping for ghost-free motion blur (sections 9.2 and 10).
7. Narrative audio time-stretching with pitch-preserved sinc interpolation via rubato (section 11).
8. Automatic HDR bracket detection from EXIF temporal clustering and EV spread analysis, with Adobe XMP stacking metadata (section 12.4).
9. Multi-format NLE export: XMEML for FCP/Premiere, CSV with SMPTE timecodes for DaVinci Resolve (sections 12.5 and 12.6).
10. Full project persistence with BLAKE3-based partial re-rendering for near-instantaneous iterative workflows (section 18).
11. Stochastic RPDF dithering theory with quantitative justification for the choice over TPDF (section 7).
12. A multi-stage pipeline architecture with LRU caching providing a provable 100% hit rate for the temporal convolution window (section 16).

The resulting engine demonstrates that principled signal processing and careful numerical engineering can deliver results that rival commercial solutions costing thousands of dollars, while maintaining full transparency through diagnostic exports and formal documentation.

References

- [1] G. Farnebäck, “Two-frame motion estimation based on polynomial expansion,” in *Image Analysis: 13th Scandinavian Conference, SCIA 2003*, ser. Lecture Notes in Computer Science, vol. 2749, Springer, 2003, pp. 363–370. DOI: [10.1007/3-540-45103-X_50](https://doi.org/10.1007/3-540-45103-X_50)